

A Comparative Study of Current Open-source Infrastructure as a Service Frameworks

Theo Lynn, Graham Hunt, David Corcoran, John Morrison and Philip Healy
Irish Centre for Cloud Computing, Dublin 9, Ireland

Keywords: Cloud Computing, Open Source, IaaS, Openstack, Cloudstack, Opennebula, Eucalyptus.

Abstract: With the growth of cloud computing in recent years, several commercial and open source IaaS frameworks have emerged. The development of open source IaaS solutions offers a free and flexible alternative to commercial cloud services. The main contribution of this paper is to provide a qualitative comparative of current open-source IaaS frameworks. Existing research papers examining open source IaaS frameworks have focused on comparing OpenStack with a small number of alternatives. However, current research fails to adequately compare all major open source frameworks in a single study and notably lacks the inclusion of CloudStack. Our research paper provides the first overview of the five main open source cloud IaaS frameworks – OpenStack, CloudStack, OpenNebula, Eucalyptus and Nimbus. As such, this review provides researchers and potential users with an up to date and comprehensive overview of the features of each solution and allows for an easy comparison between the open source solutions.

1 INTRODUCTION

Cloud Computing technologies have seen significant adoption in recent years by enterprises, researchers and individuals. It is forecast that the Cloud Computing industry will reach a market size of \$241 billion by 2020 (Reid and Kilster, 2011). Cloud Computing has become an important tool in delivering Infrastructure as a Service (IaaS) for users that require a high level of flexibility and management of their software stack.

The last eight years has seen the rapid proliferation of vendors in the IaaS market, which includes major enterprise players such as Amazon (Amazon, n.a.), Microsoft (Microsoft, n.a.), Google (Google, n.a.), and IBM (Google, n.a.). While these services may offer flexibility to the user, commentators cite cost, lack of portability and lack of interoperability as drawbacks to enterprise IaaS (Mahjoub et al., 2011; Zhang et al., 2013). With each vendor promoting its own infrastructure and incompatible standards, users can suffer from vendor lock-in (Mahjoub et al., 2011; Zhang et al., 2013; Wen et al., 2012).

In some environments it may be preferable for an organization to develop their own cloud, based on an open source framework. Open source frameworks offer an alternative to enterprise clouds by offering

the freedom to modify the source code and build a cloud that is pluggable and open to extensions while reducing costs and avoiding vendor lock-in (Zhang et al., 2013; Wen et al. 2012; Bist et al., 2013). The development of open source solutions is of particular importance for the further proliferation of private and hybrid clouds (Sefraoui, 2012).

The decision to select the most appropriate open source solution can be a challenge for organizations given each framework's specific characteristics. While previous studies (Zhang et al., 2013; Dukarić, R. and Jurić, 2013; Endo et al., 2010; Yaday, 2013) have compared these frameworks, most have focused only on OpenStack and OpenNebula, neglecting CloudStack or have focused on comparing frameworks with different hypervisors (Ristov et al., 2013; Cordeiro, 2010).

The purpose of this paper is to provide a qualitative review of the top five open source IaaS frameworks and present a comparative analysis to aid in framework selection decisions. The frameworks included in this paper have been chosen based on literature reviews and perceived industry acceptance. The open source frameworks detailed in this paper are OpenStack, CloudStack, OpenNebula, Eucalyptus and Nimbus.

The paper is structured as follows. In Section II, we present a background to the study providing an

overview of the benefits of Cloud Computing adoption and discuss the different service models. Section III provides a qualitative review concerning the five chosen frameworks and examines their components. This section also provides a short overview of closed vs. open source IaaS solutions.

Drawing on the review presented in Section III, Section IV provides a comparative analysis of the identified frameworks based on selected criteria. Finally, Section V will discuss our conclusions and recommendations for future developments.

2 BACKGROUND

The National Institute of Standards and Technology (NIST) defines Cloud Computing as “a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction” (Mell and Grance, 2011). Today it is the most widely cited and accepted definition. This definition can be expanded to include five additional characteristics of cloud computing. These characteristics have also become widely accepted and cited; they are on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service. The NIST definition of cloud computing outlines three service models: software as a service (SaaS); platform as a service (PaaS); and infrastructure as a service (IaaS). In addition, it outlines four deployment models: private cloud; community cloud; public cloud; and hybrid cloud.

2.1 Service Models

Cloud Computing is typically classified according to a variety of service models. In recent years, service models such as Storage as a Service (Fielder et al., 2012) and Business Process as a Service (Lynn et al., 2014) have emerged. However, the most common differentiation of service models remains as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS), all of which have been given a definition by NIST.

Software as a Service is when the supplier provides working applications running on their own cloud infrastructure, typically these applications would be accessed through a web browser or some other type of web interface or application (Mell and Grance, 2011).

Platform as a Service is when the supplier provides the user with the tools required for deploying an app onto the cloud infrastructure (Mell and Grance, 2011).

Finally, Infrastructure as a Service, the focus of this paper, is when the supplier provides the consumer with processing, memory, storage, networks, and other computing resources (Mell and Grance, 2011).

3 OPEN SOURCE IaaS FRAMEWORKS

Enterprise Solutions (or closed frameworks) typically supply servers and storage to their customers using their own infrastructure through a public cloud development (Zhang et al., 2013). Open source IaaS frameworks take advantage of open source code which can be modified by users to create a single functional package that can be applied to a network of servers and storage to produce IaaS (Zhang et al., 2013). While open source solutions are typically used to develop private clouds they are also suitable for hybrid and public cloud development models (Salih and Zang, 2012; Ristov et al., 2013; Wen et al., 2012). Recent acquisitions such as Citrix’s purchase of CloudStack (Keeps, 2014) suggest that we are likely to see further development of public clouds based on open source frameworks, leading to improved interoperability between different clouds.

The main player offering enterprise solutions is Amazon Web Services, with growing competition from competitors including Microsoft, Google, and IBM (Gartner, 2014). The remainder of this paper will discuss open source frameworks.

Five major projects, OpenStack, Apache CloudStack, OpenNebula, Eucalyptus, and Nimbus dominate the market for Open Source IaaS. A survey conducted in July 2014, by Linux.com, of 550 Open Source cloud computing experts found that OpenStack, CloudStack, OpenNebula, and Eucalyptus are the most popular Open Source IaaS options with all others receiving less than 2% of the result (Williams, 2014). OpenStack was favoured with 63 per cent of the vote, followed by CloudStack, OpenNebula, and Eucalyptus, respectively. OpenStack was also found to be the most popular overall Open Source Cloud Computing project. One of the reasons suggested for OpenStack’s popularity is that the strength of the OpenStack community and the prestige and size of the companies who back OpenStack has given it a

significant advantage over its alternatives (Voras et al., 2013). Another reason suggested is that with the maturation of the market there has been a focus on the bigger projects available in search of a de facto standard (Voras et al., 2013).

An alternative view is that no one Open Source IaaS framework will dominate the market, as each framework is suited to different end use scenarios or applications. For example it is proposed that OpenNebula is more suited to security-sensitive and smart-city scenarios while OpenStack is more suited for time-constrained scenarios (Kostantos, 2013).

At this stage in the market's maturation, the likelihood of new entrants is slim (Voras et al., 2013). Considering the lead the five mentioned projects have over their alternatives, we have decided to focus our analysis on these projects for the purpose of this paper. In the remainder of this section we provide a short discussion of the qualitative features of the different open source IaaS frameworks.

3.1 OpenStack

Announced in 2010, OpenStack is an entirely open sourced IaaS project for private and public clouds. Initially developed by Rackspace Hosting and NASA, it has since developed into a global collaboration of developers and cloud computing technologists. With over 4500 members, 850 companies, and support from major tech industry companies, OpenStack has a very powerful foundation. The OpenStack Open Source Cloud Mission is "to produce the ubiquitous Open Source Cloud Computing platform that will meet the needs of public and private clouds regardless of size, by being simple to implement and massively scalable" (Openstack, n.a.).

OpenStack has three core official software programs, Compute (Nova), Object Storage (Swift),

and Image Service (Glance).

Nova is the computing engine behind OpenStack. It is the cloud computing fabric controller and the central element of the IaaS system (Openstack, n.a.). It manages all the compute resources of the OpenStack cloud. It includes an API (nova-api) server that accepts requests for creating virtual machines (VM), and their local or remote disks (Baset, 2012), and associated metadata in a VM. Other processes within Nova include:

- nova-schedule – takes a VM request and determines where it should be placed.
- nova-compute – creates and terminates VM instances through a hypervisors' API. It includes support for Xen, XenServer/XCP, KVM, VMware vSphere, LXC, QEMU, UML, Hyer-V, etc.

Swift offers cloud storage software enabling the storage and access of objects across different nodes using a simple API. It is designed to be extremely scalable in size and capacity.

Glance provides image services for OpenStack. It provides a catalogue and repository of VM images, also allowing them to be used as templates when deploying new VM instances (Kumar et al., 2014).

Other official OpenStack programs include Cinder, Neutron, Horizon, Keystone, Ceilometer, Heat, and Trove.

Cinder is the block storage component of OpenStack. It provides persistent block level storage devices for use with OpenStack compute instances.

Neutron is OpenStack Networking; it provides networking capabilities to OpenStack and manages networks and IP addresses (Kumar et al., 2014).

Horizon is the dashboard behind OpenStack. It provides a graphical interface for users and administrators.

Keystone provides identity services to OpenStack. It is a means of access to the OpenStack services available to different users.

Ceilometer provides billing services to individual users of the cloud. It also keeps an account of the users' usage of different components of OpenStack.

Heat uses a template based orchestration mechanism to launch multiple composite cloud applications. The templates are in the form of text files that are treated like code and define what resources are necessary for the code.

Trove is a database as a service for OpenStack. It provides functionality for both relational and non-relational database engines.

The latest OpenStack release, Icehouse, was released in April 2014. This release saw

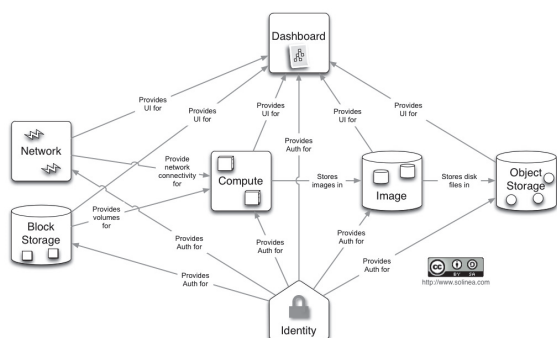


Figure 1: Conceptual OpenStack Architecture, Source: Openstack, n.a..

approximately 350 new features and 2,902 bug fixes added. Features added included support for rolling upgrades, the new feature discoverability that improves workflow, mandatory testing for external drivers, automated scaling of additional resources across the platform.

3.2 CloudStack

CloudStack was developed by Cloud.com and in 2010 released the majority of the project's source code. In 2011 CloudStack were acquired by Citrix who subsequently released the remainder of CloudStack's code. In April 2012 Citrix donated CloudStack to the Apache Software Foundation (ASF). It was accepted into the Apache Incubator and then graduated from the Apache Incubator in March 2013 to become a Top Level Project of ASF (Cloudstack, n.a.) CloudStack doesn't have quite the same foundation of users as OpenStack but a number of large companies do use Apache CloudStack including Apple, British Telecom, Dell, Nokia, and Fujitsu.

CloudStack consists of the management server and the resources to be managed. The management server is the CloudStack software that manages the cloud resources and allocates resources in the cloud deployment. It provides a web interface for administrators and end users, as well as API interfaces for CloudStack API and also the Amazon EC2 interface. The management server also manages assignment of guest VMs, assignment of IP addresses, allocates storage during VM instantiation, and manages snapshots, disk images, and ISO images. It is a single point of configuration for the IaaS cloud and during deployment the user informs it of the resources to be managed.

The CloudStack deployment is organised with hosts contained within clusters, clusters contained within pods, pods contained within zones, and zone contained within regions. Regions are the largest available units in the CloudStack deployment.

Regions essentially consist of a number of available zones, with zones roughly being the equivalent of a data enter. Regions can be used to provide fault tolerance and achieve higher availability and scalability. Zones, as mentioned typically correspond to a single data centre, but a data centre can also contain multiple zones. Pods often represent a single rack. Clusters are a way to group hosts; hosts in a cluster will all have identical hardware and run the same hypervisor. Hosts are single computers and provide the resources that run guest VMs. Figure 2 provides an outline of the

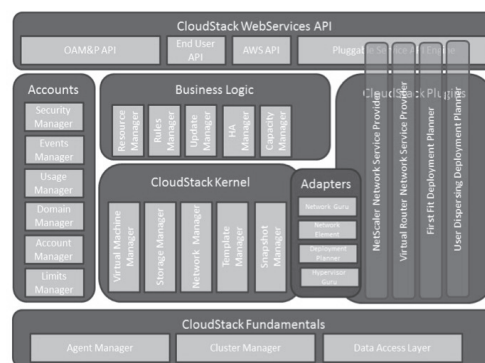


Figure 2: CloudStack Architecture, Source: Cloudstack.

conceptual architecture of a simple CloudStack deployment, note that does not include regions since these would not be necessary for a simple smaller scale deployment (Cloudstack).

The latest version of Apache CloudStack is 4.4.0 and was released in August 2014 and had a number of updates including updating to the latest version of Java.

3.3 OpenNebula

OpenNebula was initially released in 2008 and now operates as an open source project. While it is most commonly used for private cloud, it also supports public and hybrid clouds. The OpenNebula mission is "to become the simplest cloud enabling platform for the enterprise" and their purpose is "to bring simplicity to the private and hybrid enterprise cloud". OpenNebula has a large user base with telecommunication companies (e.g. Telefonica), system integrators (e.g. Logica), supercomputing centers (e.g. SARA), etc. (Kostantos et al., 2013). Additionally, in March 2010 the main authors of OpenNebula founded C12 Labs to provide a value-added professional service, which many enterprises require for internal adoption, and to enhance OpenNebulas long-term sustainability (Yadav, 2013).

The OpenNebula architecture is a classical cluster with a front end and a set of cluster nodes to run the VMs. At least one physical network is needed for connecting all cluster nodes with the front end (OpenNebula, n.a.). This architecture is flexible and modular, making it easier to integrate multiple storages, network infrastructures and hypervisor technologies than using OpenStack (Wen, 2012).

The OpenNebula software consists of three layers: tools, drivers, and core. An example of an

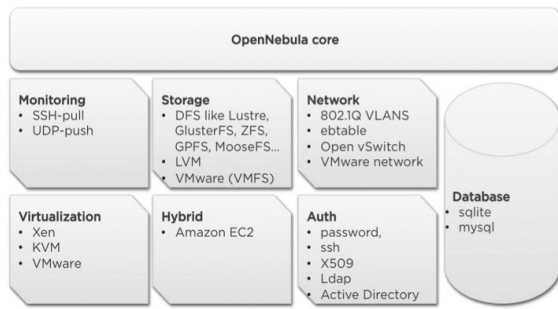


Figure 3: OpenNebula Architecture, Source (OpenNebula, n.a.).

architecture reference model for IaaS clouds is shown in Figure 3.

The **Tools** layer provides interfaces to communicate with users and allows users to manage VMs through the interfaces, which include command line interface (CLI) and libvirt API (Wen, 2012). This layer also contains a scheduler that manages the functionality of the core layer (Wen, 2012).

The **Drivers** layer contains components that communicate directly with the underlying operating system and capture the underlying infrastructure as an abstract service (Wen, 2012).

The **Core** layer (Figure 3) contains the components used to perform user requests and control resources. In this layer, disk images for VMs are stored using datastores (referred to as Image Repositories in previous releases). The monitoring subsystem gathers information on the hosts and the VMs, such as basic performance indicators and capacity consumption, and it also contains the authentication system, which comes with a built-in user/password authentication driver and the choice from SSH Authentication, X509 Authentication, and LDAP Authentication also.

The latest OpenNebula update is version 4.8.0, Lemon Slice, released on the 12th of August 2014. The release saw improvements to the hybrid model, changes to virtual networks that can now include any combination of ranges to accommodate any address distribution, and several other improvements throughout every other OpenNebula component.

3.4 Eucalyptus

Eucalyptus was developed by the University of California-Santa Barbara as an open source Infrastructure as a service and was released in 2008. It allows for the installation of private and hybrid clouds. Eucalyptus target is hybrid installations with

its compatibility with Amazon Web Service AWS. Eucalyptus' scalability is limited in comparison to some of the alternative open source IaaS solutions and, as such, has lost some of the popularity it had in the earlier days of its release (Sefraoui et al., 2012). The introduction of Scalable Object Storage has helped improve the scalability of Eucalyptus.

Eucalyptus has six components (one being an optional component) (Ristov et al., 2013): Cloud Controller (CLC), Scalable Object Storage (SOS), Cluster Controller (CC), Storage Controller (SC), VMware Broker components and Node Controller (NC). These components are divided into three layers: Cloud Layer, Cluster Layer and Cloud Controller.

Cloud Controller is contained in the Cloud Layer and is the entry-point for administrators, developers, project managers, and end users. It queries the other components in Eucalyptus for information and makes requests to the Cluster Controllers. It is responsible for exposing and managing the underlying virtualized resources and offers a web-based administrative interface.

Scalable Object Storage is also contained in the Cloud Layer; this is the Eucalyptus equivalent to Amazon Web Services Simple Storage Service (S3). SOS allows for the implementation of scale-out storage that implements the S3 interface. Eucalyptus also provides a basic storage solution known as Walrus, suitable for smaller cloud deployments.

Cluster Controller is contained in the Cluster Layer. It communicates with the SC and NC and manages the execution of VM instances on specific NCs. The CC must have network connectivity to the machines running the CLC and the machines running the NC.

Storage Controller is also contained in the Cluster Layer. It communicates with the CC and the NC and manages the VM volumes and snapshots from volumes in the cloud. This component functions similarly to the Amazon Elastic Block Store.

The **VMware Broker** is the final component in the Cluster Layer and is the only optional component. It mediates interactions between the CC and VMware and can connect directly to ESX/ESXi hosts or to vCenter Server.

Node Controller is the sole component in the Node Layer. The Node Controller is written in C, hosts the VM instances and manages the virtual network endpoint (Eucalyptus, n.a.).

The latest release of Eucalyptus is version 4.0.1, a maintenance update for version 4.0.0 that was released on the 30th of May 2014. Some of the

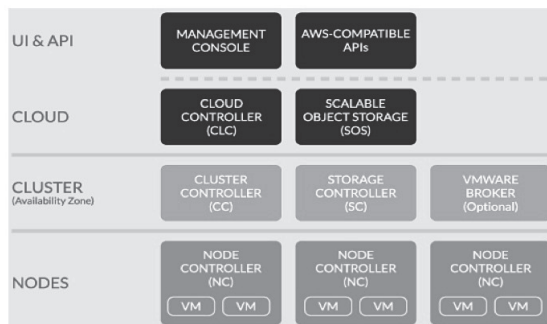


Figure 4: Eucalyptus Architecture, Source: Eucalyptus, n.a..

changes in this release include the possibility to have multiple user-facing services, the possibility to use different object storage backends, and more improvements throughout most of the Eucalyptus software.

3.5 Nimbus

Nimbus is an open source IaaS project that was initially released in 2009. Nimbus' mission is "to evolve the infrastructure with emphasis on the needs of science". Nimbus is built with three goals in mind:

- Enable resource owners to provide their resources as an infrastructure cloud.
- Enable cloud users to access infrastructure cloud resources more easily.
- Enable scientists and developers to extend and experiment with both sets of capabilities.

The main infrastructure components are the Workspace Service site manager, a web services resource framework (WSRF) based remote protocol implementation, an EC2 based remote protocol implementation of their SOAP and Query APIs, Cumulus, RM API, the cloud client, the reference client, the Workspace Pilot, the workspace-control agent, and the metadata server.

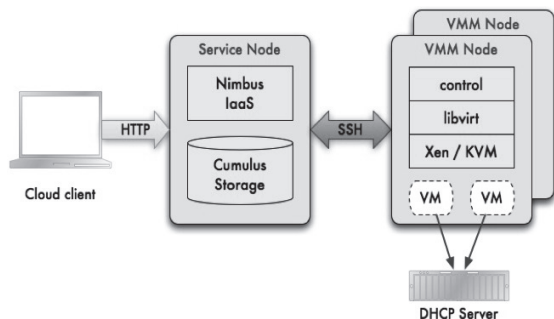


Figure 5: Nimbus Architecture, Source: Nimbus, n.a..

The **Workspace Service** is a site VM manager that different remote protocol frontends can invoke. The **metadata server** responds to HTTP queries from VMs. The **Cloud Client** is the component that enables users to launch instances quickly. The **workspace-control** is a program that can start, stop and pause VMs, implement VM image reconstruction and management, connect the VMs to the network, and deliver contextualization information.

4 COMPARISON OF OPEN SOURCE SOLUTIONS

While the IaaS frameworks discussed in Section III are designed to allow users to manage their own virtual infrastructures, they have different features that should be considered when selecting a framework. This will differ depending on the expected use case and organization. These qualitative features are summarized in Table 1. The set of criteria outlined in Table 1 are derived from a review of the supporting framework documentation and related literature. Based on this review 19 criteria emerged as the main discussed points when comparing open source IaaS frameworks. These criteria are outlined below:

- **Philosophy:** Concerns the main use cases for the frameworks deployment (Wen et al., 2012).
- **Suitability:** Different frameworks will better service the needs of different users. As a result, large commercial organizations are likely to adopt a different framework than research institutes.
- **Architecture:** The architecture of the platform provides details on how the framework was built and how it operates (Mahjoub et al., 2011)
- **API Support:** Cloud APIs are application programming interfaces that are used to build applications (Wen et al., 2012; Endo et al., 2010; Semolinski and Thain, 2010).
- **Amazon Support:** Given that AWS is the predominant enterprise cloud framework, compatibility with such public clouds is an important factor for many users (Wen et al., 2012).
- **Cloud Implementation:** The cloud deployment models (Mell & Grance, 2011) that are suitable for the IaaS framework.
- **Hypervisor:** A hypervisor also known as a VM Manager (VMM) is used to describe a technique to allow multiple guests to run concurrently on a

Table 1: Comparison of Open Source IaaS Solutions.

Capability/features	OpenStack	CloudStack	Eucalyptus	OpenNebula	Nimbus
Established Origin	2010 Rackspace, NASA, Dell, Citrix, Cisco, Canonical etc.	2010 Cloud.com	2008 Santa Barbara university, Eucalyptus System Company	2008 European Union	2009 University of Chicago
Philosophy	Offers Cloud Computing services		Mimic Amazon EC2	Private, highly customizable cloud	Cloud tailored to scientific researchers
Suitability	Enterprises, service providers and researchers	Enterprises, service providers and researchers	Large commercial enterprises, Research institutions	Large commercial companies and public institutions	Research institutions
Architecture	Integration of OpenStack object and OpenStack compute	Hierarchical with four main components: - Management Server, - Availability Zone, - Pod, - Computer Nodes.	Hierarchically grouped from CLC via the CC to the NC ; - Hierarchical - Five components - Minimum two servers	Three modules contain all components; - Centralized - Three components - Minimum two servers	Three modules contain all components; - Centralized - Three components - Minimum two servers
API Support	Native API, Amazon EC2 API, CloudFiles REST API.	Amazon EC2 API, S3	Amazon EC2 API,	Native API in Ruby and JAVA. XML-RPC API for interfaces creation. OGF OCC1 & Amazon EC2 APIs.	EC2 APIs, S3 APIs, JAVA client APIs.
Amazon Support	EC2, S3	EC2, S3	EC2, S3, EBS, IAM, AMI	EC2, EBS, AMI	EC2, S3
Cloud Implementation (deployment)	Public Hybrid Private	Public Hybrid Private	Private Hybrid	Private Hybrid	Private Community
Hypervisor	KVM, Xen, VMware ESX, ESXi, Hyper-v, LXC, QEMU, UML, PowerVM, Bare metal	VMware, Oracle VM, KVM, XEN	KVM, Xen, VMware	KVM, Xen, VMware ESX, ESXi	Python, Bash, Ebttables, Libvirt, KVM, Xen
Programming Language Community	Python +++++	Java ++++	Java, C, Python +++	Java, Ruby and C++ +++	Java, Python ++
Release Frequency	<4 months	4 months	>4 months	>6 months	<4 months
Ease of use	+++++	++++	++	+++	+++
Supported OS	Linux, Windows, Requires x86 Server	Depending on the Hypervisor and hardware - Mac OS X, Asianux, CentOS, Debian, DOS, Fedora, FreeBSD, Novell Netware, Oracle Enterprise Linux, Ubuntu, Red Hat Enterprise Linux, Sun Solaris, SUSE Linux Enterprise, Windows.	Linux (Ubuntu, Fedora, CentOS, OpenSUSE et Debian)	CentOS, Debian, Fedora, RHEL open-SUSE, SLES, and Ubuntu.	Most Linux distributions
Storage	Object and block storage supported. Volumes are persistent (data retained until the volume is deleted, independently from the VM). File storage is supported through Swift (organizing the files in containers).	Supports for iSCSI, NFS, SMB/CIFS; support for OpenStack Swift and Amazon S3	Support for iSCSI, EBS, Amazon S3. Hardware support for industry-standard Storage Hardware.	Hardware support for Fibre Channel, iSCSI, NAS shared storage, SCSI / SAS / SATA. Non-shared and shared file systems (NFS, LVM with CoW, VMFS, etc.).	
Networking	VLAN NO VLAN Public IP's Private IP's SDN IDS Load- balance Firewalls VPN; OpenStack Compute	VLAN, Public IP,	VLAN NO VLAN Public IP's Private IP's; DHCP server on the cluster controller	VLAN NO VLAN Public IP's Private IP's Ebttables OVSswitch; Manual configuration	DHCP server installed on nodes
User Interface	Web interface (i.e. Dashboard) and Command line interface to deploy VMs and a console to manage the VMs.	Web interface and Command Line Interface (CLI)	euca2ools (CLI)	Web interface and Command Line interface (CLI)	Web-Services, specifically: Nimbus Web
Security	API includes protection against DoS attacks or faulty clients. The project concept is introduced by Nova, allowing administrators to manage other user accounts and the project resources. Keystone used for identity management.	CloudStack Security Groups	The Cloud Controller generates a public/private key code pairing for user authentication	Authentication by passwords, secure shell and RSA key code pairings Lightweight Directory Access Protocol ; Authentication framework based on passwords, SSH RSA key-pairs or LDAP. Various administration roles. Multi-tenancy for public clouds.	Public Key Infrastructure
Error Robustness	Replication	Replication	Separate clusters reduce likelihood of correlated errors; Cluster controller's separation	Permanent database to store information about hosts, networks and virtual machines; Database backend (registers virtual machine information)	Regular check and backup of worker nodes; Periodic verification of cloud nodes
Load Balancing	The Cloud Controller	TCP Load Balancer	The Cloud Controller	Nginx	Le Context Broker
Licensing	ApacheLicence Version2	ApacheLicence Version2	BSD-Licence	ApacheLicence Version2	ApacheLicence Version2
Document Support	+++++	+++++	+++	+++	++

- host machine (Mahjoub et al., 2011; Wen et al., 2012; Von Laszewski et al., 2010).
- **Programming Language:** Concerns the scripting language in which the framework is written.
 - **Community:** A community may consist of individuals, teams or organizations that are users, developers or service providers. While all may not contribute to the development, the size of the community is a good indication of future developments of the framework (Wen et al., 2012).
 - **Release Frequency:** The strength of the community will influence the regularity of updates to the open source frameworks. Typically these range from between 4-6 month release cycles.
 - **Ease of Use:** The complexity of setting up the system and managing VMs. This can be impacted by the supporting documentation available (Wen et al., 2012). Easy of management is crucial for successful implementing Infrastructure as a service (Voras et al., 2013).
 - **Supported OS:** Concerns the operating systems that are implemented on the machines.
 - **Storage:** Concerns the support for storage technologies such as network-attached storage, direct-attached storage and backup technologies (Voras et al., 2013). How data is stored on the virtualized pools is critical to achieving flexibility, scaling and ease of use (Wen et al., 2012; Voras et al., 2013).
 - **Network:** Cloud frameworks must give attention to virtual networks to ensure optimal performance (Voras et al., 2013). Networks can be considered a means of using services deployed on the VMs and managing the cloud environment (Voras et al., 2013; Von Laszewski et al., 2013). This includes support for VLAN, and firewalls.
 - **Interface:** Refers to how the user will access and manage their resources, typically through a command line interface (CLI) or browser interface (Mahjoub et al., 2011; Wen et al., 2012).
 - **Security:** Given the perceived security treat of cloud computing, the protection of data is imperative to any IaaS framework Lynn et al., 2014; Mahjoub et al., 2011).
 - **Error Robustness:** Concerns how the system will tolerate component failure and continue to operate effectively if there is a component failure (Mahjoub et al., 2011).
 - **Load Balancing:** In computer networking terminology, load balancing denotes how a workload is distributed across multiple machines to optimize resource utilization (Mahjoub et al., 2011).
 - **Licensing:** Refers to the restrictions of the user on distributing and modifying the software with or without the payment or royalties (Wen et al., 2013). The most common license of open source IaaS frameworks is Apache License Version 2.0 (Openstack, n.a.; Cloudstack, n.a., Von Laszewski et al., 2012).
 - **Documentation Support:** The level of documentation available to users will impact on the ease of use for setup and management. One of the challenges is keeping documentation updated in line with frequent new releases

5 CONCLUSIONS

In this paper, we present an up to date qualitative review of the main open source infrastructure as a service frameworks. Building on this, we deliver a means of comparison between these frameworks based on a set of criteria developed to evaluate open source frameworks. The paper adds to the literature by providing a comparison of five open source solutions based on the latest releases. We believe that the evaluation will to help potential adopters understand the functions of these frameworks and aid in the early stage adoption decision-making process. This work aims to illustrate the difference offerings for each of the IaaS frameworks and that user requirements may be significantly different, impacting on their framework selection.

In future research, the criteria outlined in this paper will be used to expand this study to compare open and closed IaaS frameworks. We also look to develop performance criteria that can be combined with the existing paper to develop a comprehensive quantitative and qualitative comparison tool.

ACKNOWLEDGEMENTS

The research work described in this paper was supported by the Irish Centre for Cloud Computing and Commerce, an Irish national Technology Centre funded by Enterprise Ireland and the Irish Industrial Development Authority.

REFERENCES

- Amazon Web Services Homepage. <http://aws.amazon.com/>.
- Baset, S. (2012). Open Source Cloud Technologies. *Proceedings of the Symposium on Cloud Computing – SoCC'12*, 28-27.
- Bist, M., Wariya, M., & Agarwal, A. (2013, February). Comparing delta, open stack and Xen Cloud Platforms: A survey on open source IaaS. In *Advance Computing Conference (IACC), 2013 IEEE 3rd International*, 96-100.
- CloudStack Homepage. <http://cloudstack.apache.org/>.
- Cordeiro, T. D., Damalio, D. B., Pereira, N. C. V. N., Endo, P. T., de Almeida Palhares, A. V., Gonçalves, G. E., Mångs, J.-E. (2010). Open Source Cloud Computing Platforms. *2010 Ninth International Conference on Grid and Cloud Computing*, 366–371.
- Dukarić, R., & Jurić, M. B. (2013). A Taxonomy and Survey of Infrastructure-as-a-Service Systems. *Lecture Notes on Information Theory*, 1(1), 29–33.
- Endo, P. T., Gonçalves, G. E., Kelner, J., & Sadok, D. (2010, May). A survey on open-source cloud computing solutions. In *Brazilian Symposium on Computer Networks and Distributed Systems*, 3–16.
- Eucalyptus Homepage. <https://www.eucalyptus.com/>.
- Fielder, A., Alleweldt, F., Kara, S., Brown, I., Weber, V., & McSpedden-Brown, N. (2012) Cloud Computing Study. For the European Parliament's Committee on Internal Market and Consumer Protection. IP/A/IMCO/ST/2011-18.
- Gartner. (2014). *Magic Quadrant for Cloud Infrastructure as a Service*. Available: <http://www.gartner.com/technology/reprints.do?id=1-1UKQQA6&ct=140528&st=sb>. Last accessed 30th Sept 2014. Last accessed 30th Sept 2014.
- Google Cloud Platform Homepage. <https://cloud.google.com/compute/>.
- IBM IaaS Homepage. <http://www.ibm.com/cloud-computing/ie/en/iaas.html>.
- Keeps, B. (2014). *Cloudstack execs take the money and run or were they pushed*. Available: <http://www.forbes.com/sites/benkepes/2014/09/04/on-the-three-year-anniversary-cloudstack-execs-take-the-money-and-run-or-were-they-pushed/>. Last accessed 30th Sept 2014.
- Kostantos, K., Kapsalis, A., Kyriazis, D., Themistocleous, M., & Rupino de Cunha, P. (2013). Open-Source IaaS Fit For Purpose: A Comparison Between OpenNebula and OpenStack, *International Journal of Electronic Business Management*, 11(3), 191-201.
- Kostantos, K., Kapsalis, A., Kyriazis, D., Themistocleous, M., & Rupino, P. (2013). OPEN-source iaas fit for purpose: a comparison between opennebula and openstack, *11(3)*, 192–202.
- Kumar, R., Gupta, N., Charu, S., Jain, K., & Jangie, S. (2014) Open Source Solution for Cloud Computing Platform Using OpenStack. *International Journal of Computer Science and Mobile Computing*, 3(5), . 89-98.
- Lynn, T., O' Carroll, N., Mooney, J., Helfert, M., Corcoran, D., Hunt, G., Van Der Werff, L., & Morrison, J. (2014) Towards a Framework for Defining and Categorising Business Process-As-A-Service (BPaaS), *Proceedings of the 21st International Product Development Management Conference*.
- Mahjoub, M., Mdhaftar, A., Halima, R. Ben, & Jmaiel, M. (2011). A Comparative Study of the Current Cloud Computing Technologies and Offers. *2011 First International Symposium on Network Cloud Computing and Applications*, 131–134.
- Mell, P., & Grance, T. (2011) The NIST definition of cloud computing. Recommendations of the National Institute of Standards and Technology, Washington, D.C. (NIST Special Publication 800-145).
- Microsoft Azure Homepage. <https://azure.microsoft.com>.
- Nimbus Homepage. <http://www.nimbusproject.org/>.
- OpenNebula Homepage. <http://www.opennebula.org/>.
- OpenStack Homepage. <http://www.openstack.org/>.
- Reid, S., Kilster, H., "Sizing the Cloud", Forrester Research Report, 2011.
- Ristov, S., Gusev, M., & Donevski, A. (2013). OpenStack cloud security vulnerabilities from inside and outside. In *CLOUD COMPUTING 2013, The Fourth International Conference on Cloud Computing, GRIDS, and Virtualization*, 101-107.
- Salih, N. K., & Zang, T. (2012). Survey and comparison for Open and closed sources in cloud computing. *arXiv preprint arXiv:1207.5480*.
- Seffraoui, O., Aissaoui, M., & Eleuldi, M. (2012). Comparison of multiple IaaS Cloud platform solutions. *7th WSEAS International Conference on Computer Engineering and Applications, (Milan-CEA 13)*. 978-1000.
- Sempolinski, P., & Thain, D. (2010). A Comparison and Critique of Eucalyptus, OpenNebula and Nimbus. *2010 IEEE Second International Conference on Cloud Computing Technology and Science*, 417–426.
- Vaquero, L. M., Rodero-Merino, L., Caceres, J., & Lindner, M. (2009) A break in the clouds: towards a cloud definition, *ACM SIGCOMM Computer Communication Review*, 39(1), 50-55.
- Von Laszewski, G., Diaz, J., Wang, F., & Fox, G. (2012, June). Comparison of multiple cloud frameworks. In *Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on*, 734-741.
- Voras, I., Orlic, M., & Mihaljevic, B. (2013). The Effects of Maturation of the Cloud Computing Market on Open Source Cloud Computing Infrastructure Projects, *Proceedings of the ITI 2013 35th International Conference on Information Technology Interfaces*. 101-106.
- Wen, X., Gu, G., Li, Q., Gao, Y., & Zhang, X. (2012). Comparison of open-source cloud management platforms: OpenStack and OpenNebula. *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery, (Fskd)*, 2457–2461.
- Williams, A. (2014). *The Top Open Source Cloud Projects of 2014* Available: <http://www.linux.com/news/enterprise/cloud-computing/784573-the-top-open>

source-cloud-projects-of-2014. Last accessed 30th Sept 2014.

Yadav, S. (2013). Comparative Study on Open Source Software for Cloud Computing Platform: Eucalyptus, Openstack and Opennebula, *International Journal of Engineering and Science*, 3(10), 51-54.

Zhang, Z., Wu, C., & Cheung, D. W. (2013). A survey on cloud interoperability: taxonomies, standards, and practice. *ACM SIGMETRICS Performance Evaluation Review*, 40(4), 13-22.



Apache CloudStack: Open Source Infrastructure as a Service Cloud Computing Platform

Rakesh Kumar¹, Kanishk Jain², Hitesh Maharwal³, Neha Jain⁴, Anjali Dadhich⁵

^{1,2,3,4,5}Department of Information Technology & JECRC, Jaipur, India

Abstract— Cloud computing is changing the way IT is delivered in enterprises around the world, as well as the whole world's leading open source cloud computing platform. Apache CloudStack, helps you implement a cloud computing service in your enterprise and it is a turnkey open source cloud management as well as flexible cloud orchestration platform for deploying reliable and also scalable public and private Infrastructure-as-a-Service (IaaS) in cloud computing environments. Apache CloudStack was mainly designed to deploy and manage a large number of virtual machines (VMs) and the software-side infrastructure required to support it. Apache CloudStack is a top-level project of the Apache Software Foundation (ASF) which provides a management server and agents for hypervisor hosts so that you can run an Infrastructure-as-a-Service (IaaS) cloud. In this research paper, we are discussing introduction, benefits, background, features, use cases and purposes of Apache CloudStack. Further discussing deployment architecture, components, API, pros and cons of Apache CloudStack and lastly discuss about some new features supported by latest release of Apache CloudStack 4.4. This paper show importance of Apache CloudStack as a Cloud provider and give the best solution for service providers, web content providers, SaaS providers, and enterprises.

Keywords— *Apache CloudStack; Cloud Computing; Cloud infrastructure; Hypervisor; IaaS; JIRA; Management Server; Virtual Machines (VMs)*

I. INTRODUCTION

Apache CloudStack [1], [16] is an open source cloud computing software, which is used to build private, public and hybrid Infrastructure as a Service (IaaS) clouds by pooling computing resources. Apache CloudStack [1], [17] manages computing, networking as well as storage resources. Apache CloudStack uses multiple hypervisors such KVM, vSphere and XenServer/XCP for virtualization purpose. Apache CloudStack [1], [21] also supports the Amazon Web Services (AWS) API, apart from its own APIs.

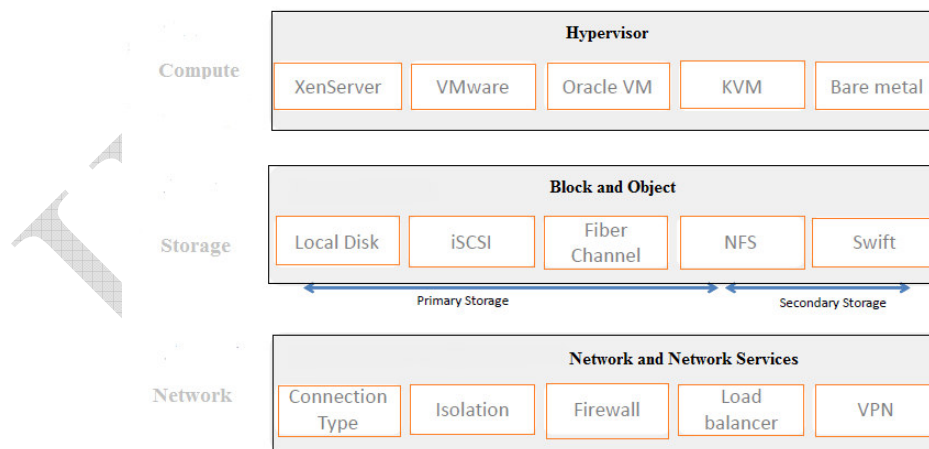


Fig. 1 Open Flexible Platform of Apache CloudStack

Apache CloudStack [8], [27] is a turnkey solution that contains the entire stack of features most organizations want with an Infrastructure-as-a-Service (IaaS) cloud such as Network-as-a-Service (NaaS), compute orchestration, user and account management, a full and open

native API, resource accounting, and a first-class User Interface (UI). Apache CloudStack [9], [26] also provides an API that's compatible with AWS EC2 and S3 for organizations which help to deploy hybrid clouds and some benefits of Apache CloudStack [7] such as:

- It mainly focus on solving business problems
- It automates deliver services quickly and helping to reduce IT operations costs
- It allows IT to provide standardized workloads, which ensure consistency
- It enhances visibility into resource allocation
- It increases the server/admin ratio and delivers benefits of scale, if deployed globally

II. APACHE CLOUDSTACK BACKGROUND

CloudStack [10], [30] was developed by Cloud.com, and Cloud.com released CloudStack as free software under the GNU General Public License, version 3 (GPLv3) in May 2010. Citrix purchased [10] Cloud.com in July 2011. In April 2012, Citrix donated CloudStack to the Apache Software Foundation (ASF). Apache CloudStack [10] became a Top-Level Project (TLP) of ASF On March 20, 2013. Some notable users of Apache CloudStack such as Amdocs, Autodesk, NTT, Zynga, China Telecom, Dell, Walt Disney, Huawei, Orange and Tata.

III. APACHE CLOUDSTACK KEY FEATURES AND USE CASES

Apache CloudStack [22] supports broad hypervisor such as XenServer, VMware, OracleVM, vSphere and KVM. Apache CloudStack [1] support scalable architecture, Multi-node installation as well as load balancing makes it highly available. Virtual networking ability is an important feature of Apache CloudStack, and also MySQL replication is useful for maintaining high availability.

Apache CloudStack [5], [7] can be used for multiple purposes such as:

- Enterprises can stage development and testing in a consistent way easing the development as well as publishing processes for applications
- Web content providers can offer scalable and elastic web infrastructure
- SaaS providers can support true multi-tenant software hosting
- Service Providers can attempt virtualized hosting in a cloud computing configuration

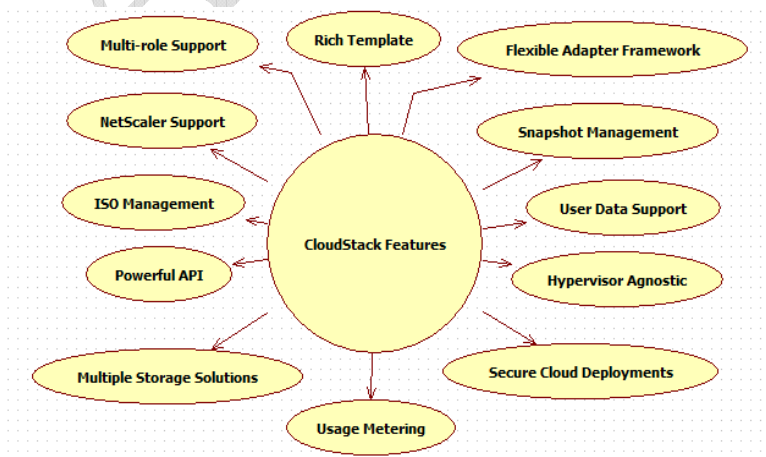


Fig. 2 Features of Apache CloudStack

Service providers, organizations [1] use Apache CloudStack to set up an on-demand as well as elastic Infrastructure as a Service (IaaS). Apache CloudStack [1], [29] can be used to set up an on premise private cloud behind the organization's firewall for internal purposes such as gaining better control over infrastructure. It can also be used to some purposes like AWS insourcing and traditional enterprise workloads.

IV. APACHE CLOUDSTACK DEPLOYMENT ARCHITECTURE

Apache CloudStack [1], [2], [15] installation consists the Management Server (Apache CloudStack software that manages cloud resources) and the cloud infrastructure. When you set up as well as manage a Apache CloudStack cloud, you provision resources such as storage, hosts, and IP addresses into the Management Server [20].

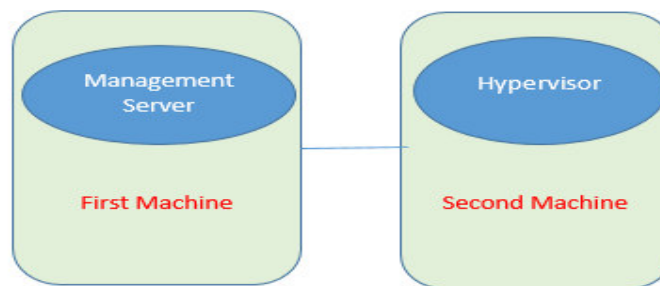


Fig. 3 Basic Deployment

A single machine [10] can act as both the Management Server and the hypervisor host when they using the KVM hypervisor, in smallest deployment. The management server controls cloud resources [1], as well as the administrator can manage and interact with the management server by using a UI or APIs.

V. APACHE CLOUDSTACK COMPONENTS

A host [1] is a computer which provides the computing resources like the CPU, storage, memory, networking, etc. to run the virtual machines (VMs) and also every host has a hypervisor installed to manage the virtual machines (VMs). Primary storage is coupled with a cluster which stores the disk volumes for all the virtual machines (VMs) running on hosts in that specific cluster.

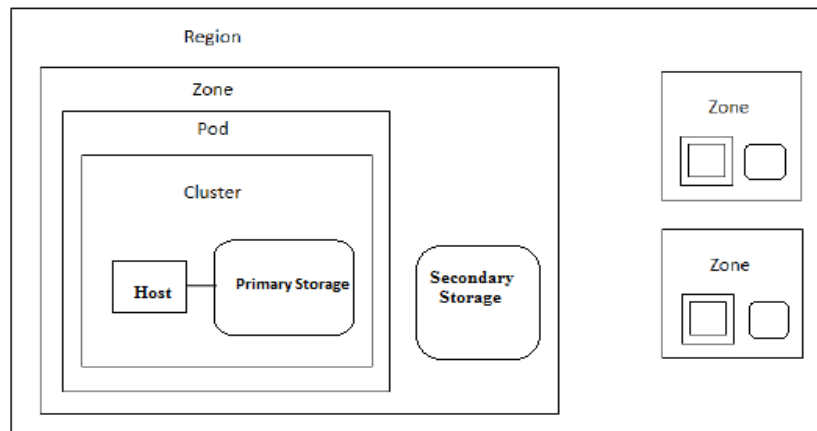


Fig. 4 A region with multiple zones

A cluster [3] contains of one or more hosts and one or more primary storage servers, and also every cluster can consist of one or more nodes. A pod [3], [6] is simply a dedicated rack of hardware, and also each pod can consist of one or more clusters. A host [24] is a single compute node which can consist of one or more virtual machines (VMs). A zone [3] simply corresponds to a single data centre and also it is permissible to have multiple zones in a data centre. Pod is contained within zones as well as every zone can contain one or more pods, and also zones can be public or private. Apache CloudStack [19] supports two types of networking basic networking, which is used for AWS style networking and advanced networking, which is used for more sophisticated network topologies.

VI. APACHE CLOUDSTACK API

The Apache CloudStack API [1], [4] supports three access roles such as root admin API, domain admin API and user API. The root admin API [4] can access all the features in addition to both virtual as well as physical resource management. The domain admin API can access the virtual resources which belong to the administrator's domain. User API can access the features that allow the management of the user's virtual machines (VMs), storage as well as network.

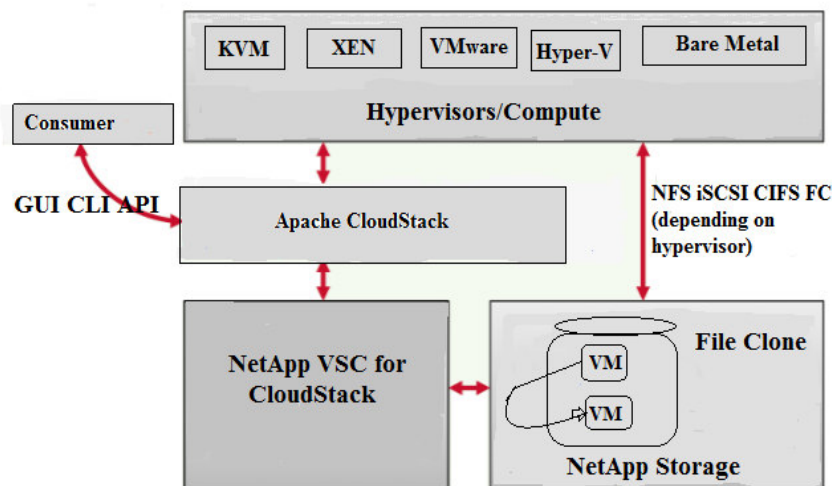


Fig. 5 Apache CloudStack

VII. APACHE CLOUDSTACK PROS AND CONS

Apache CloudStack [6], [28] abridgments complexities of dealing with compute, network, and storage. It aims at abstracting as well as unifying heterogeneous IT infrastructure into uniform services, through a common UI and user experience.

A. Apache CloudStack Pros

Apache CloudStack [18] gives featured solution designed to manage a cloud through an easy-to-use web interface, command line tools and a RESTful API. The RESTful API allows easy integration for other tools and automation frameworks. Some Apache CloudStack pros [6] such as:

- Easy to configure, use, and maintain
- Enterprise level support
- Unified management

- Allow plugin framework
- It is a mature product

B. Apache CloudStack Cons

To use the Apache CloudStack API [25], knowledge of Java, PHP, HTTP GET/POST, query strings, XML, JSON, URL of the CloudStack server, and also API key as well as secret key is necessary. Some Apache CloudStack cons [6] such as:

- Flexibility and modularity not as customizable
- It only supports Fiber Channel as primary storage through the hypervisor
- Backup and restore is not solved sufficiently and arguably

Apache CloudStack 4.4 [11] includes the some new features such as:

- Resize root disk
- Monitoring VR services
- Improvements of Hyper-V support
- Primary storage plug-in for root disks
- Added per primary storage overprovisioning
- Windowsification of CloudStack Management Server
- Region level Guest networks as well as VPC deployment
- VMware Support for DRS (Distributed Resource Scheduler)
- Support Distributed routing and network ACL with OVS plug-in

Apache CloudStack [13], [23] is a software designed to deploy and also manage big networks of virtual machines (VMs), as well as highly scalable Infrastructure as a Service (IaaS) cloud computing platform. All new features and bugs in Apache CloudStack 4.4.0 [12] have been tracked in Jira, and also have a standard naming convention of "CLOUDSTACK-NNNN" where "NNNN" represents the issue number such as CLOUDSTACK-3968.

JIRA [14] is a commercial software product, developed since 2002 by Atlassian, Inc. JIRA is written in Java and it provides bug tracking, issue tracking and project management functions.

VIII. CONCLUSIONS

Of all the technology trends but, cloud computing is one of the more interesting and least controversial, in my opinion. Cloud computing is most revolutionary types of technology entering corporate information technology strategies today. The latest innovations in cloud computing technology are making our business applications even more mobile and also collaborative, much similar to popular consumer apps such as Facebook and Twitter. Cloud computing will continue to expand and come to dominate information transactions because it provides many advantages, allowing users to have easy, instant, and individualized access to tools and information they need wherever they are, locatable from any networked device. Large businesses are far less likely to put most of work in the cloud anytime soon because of control and security issues, so further research is going on related to most efficient solution of security issues and increase the performances in dynamic reconfiguration in an IaaS Cloud Computing.



REFERENCES

- [1] <http://www.opensourceforu.com/2013/12/cloudstack-overview-open-source-iaas-platform/>
- [2] https://cloudstack.apache.org/docs/enUS/apache-cloudstack/4.0.2/html/API_Developers_Guide/deployment-architecture-overview.html
- [3] http://support.rightscale.com/09-Clouds/CloudStack/CloudStack_Reference_Architecture#Overview
- [4] <http://cloudstack.apache.org/docs/api/>
- [5] <http://cloudstack.apache.org/software/features.html>
- [6] <https://communities.netapp.com/community/netapp-blogs/the-raised-floor/blog/2014/06/23/apache-cloudstack-citrix-cloudplatform-introduction-part-i-of-iii>
- [7] <http://cloudstack.apache.org/software.html>
- [8] <http://cloudstack.apache.org/>
- [9] <http://kangaroot.net/blog/cloudstack-open-source-cloud-computing>
- [10] http://en.wikipedia.org/wiki/Apache_CloudStack
- [11] <http://cloudstack-release-notes.readthedocs.org/en/latest/about.html>
- [12] http://cloudstack-release-notes.readthedocs.org/en/latest/known_issues.html
- [13] <https://issues.apache.org/jira/browse/CLOUDSTACK/?selectedTab=com.atlassian.jira.jira-projects-plugin:summary-panel>
- [14] <http://en.wikipedia.org/wiki/JIRA>
- [15] Apache CloudStack Architecture pdf; Alex Huang, Software Architect, Citrix Systems
- [16] Getting to Know Apache CloudStack pdf; Joe Brockmeier, PPMC Member Apache CloudStack, jzb@apache.org
- [17] Apache CloudStack Cloud Computing pdf; Leverage the power of CloudStack and learn to extend the CloudStack environment; Navin Sabharwal Ravi Shankar
- [18] Apache CloudStack 4.0.0-incubating CloudStack API Developer's Guide pdf
- [19] Apache CloudStack 4.0.0-incubating CloudStack Installation Guide pdf
- [20] Apache CloudStack 4.0.2 CloudStack Installation Guide pdf
- [21] Taxonomy, Classification & Implementation of open source cloud computing platforms; CS 692 R&D Project Report; Arpit Malani (10305901); May 16, 2012 pdf
- [22] Barrie Sosinsky, "Cloud Computing Bible", Published by Wiley Publishing, Inc.; 10475 Crosspoint Boulevard; Indianapolis, IN 46256 www.wiley.com
- [23] CloudStack Documentation Release 4.3.0; Apache CloudStack; May 28, 2014 pdf
- [24] CloudStack Advanced Installation Guide; For CloudStack Version 3.0.0 – 3.0.2; Revised August 16, 2012 1:33 AM Pacific pdf
- [25] CloudStack Installation Documentation; Release 4.3, Apache Software Foundation; June 30, 2014 pdf
- [26] CloudStack Overview; Written by: Chiradeep Vittal, Alex Huang @ Citrix Revised by: Gavin Lee, Zhennan Sun @ TCloud computing pdf
- [27] Comparing Open Source Private Cloud (IaaS) Platforms; Lance Albertson OSU Open Source Lab Associate Director of Operations; lance@osuosl.org / @ramereth pdf
- [28] Building Clouds on Apache CloudStack: An Introduction; Linux CloudOpen 23 Oct 2013; Giles Sirett CEO ShapeBlue, Giles.sirett@shapeblue.com; Twitter: @ShapeBlue pdf
- [29] Hiren Parmar, Tushar Champaneria "Comparative Study of Open Nebula, Eucalyptus, Open Stack and Cloud Stack"; International Journal of Advanced Research in Computer Science and Software Engineering; Volume 4, Issue 2, February 2014; ISSN: 2277 128X
- [30] Rakesh Kumar, Neha Gupta, Shilpi Charu, Kanishk Jain, Sunil Kumar Jangir; "Open Source Solution for Cloud Computing Platform Using OpenStack"; International Journal of Computer Science and Mobile Computing; Vol. 3, Issue. 5, May 2014, pg.89 – 98; ISSN 2320-088X

Benchmarking the Performance of OpenStack and CloudStack

Aaron Paradowski, Lu Liu, Bo Yuan
Faculty of Business, Computing and Law
University of Derby, Derby, U.K. DE22 1GB
l.liu@derby.ac.uk

Abstract— With the rapid increase of growth and development in cloud computing it is clear to see that it is becoming a common trend within the industry leading to the adoption by businesses and Information Technology (IT) users alike. Coupled with this trend comes a vast amount of academic research into the cloud computing industry covering most aspects but still leaving some untouched. There is, however, a gap in the academic research for this specific area with current studies focusing on very specific performance issues without considering a general overview of performance. This paper demonstrates the importance of understanding the need to benchmark cloud platforms in order to gain a performance overview of the platforms which one may be integrating into a production environment. This paper investigates the performance of two open source cloud platforms, OpenStack and CloudStack. The performance testing is strictly focused on the platforms themselves other than the underlying elements. Therefore, they have both been tested on a mutual hypervisor and mutual available resources. The experiment results demonstrate the advantages of performing cloud management platform benchmarking.

Keywords—Cloud; Benchmark; OpenStack; CloudStack

I. INTRODUCTION

Several sources, including trend reports identified within this study show cloud computing has gained an increased interest over the recent years at a rapid rate. It is, therefore, important to have relevant and meaningful academic research into the many areas of cloud computing which led to this study.

The virtualization paradigm has evolved over time to offer more defined levels both in terms of service and deployment.[1] In line with these changes the platforms of which manage these cloud services have also changed, developed and emerged bringing greater elasticity to the way in which cloud platforms are managed at all levels. Whilst the platforms have evolved with new ones also emerging it is clearly important to have an understanding of the performance of these platforms.

This paper aims to analyse current performance research in relation to cloud computing at varying levels and fill a gap in the research around the specific area of benchmarking the performance of two open source cloud platforms, OpenStack and CloudStack, both of which are products of Citrix however written in different programming languages.

Firstly, in this paper an in depth literature review was conducted not only to review the previous overlapping studies

but also to cover the broader issues, such as the cloud offerings, the available software and subsequently the compatibility of such software with the two platforms in question.

Secondly, having identified the previous studies and their findings, this paper fills the void in the current research by defining the ways in which the cloud platforms can be benchmarked. And several key performance metrics is identified to enable the cross comparison of the platforms against these criteria.

Then, in order to evaluate the platforms (i.e. OpenStack and CloudStack), the mutually suitable hypervisors for both platforms is evaluated. Whilst both platforms have a large dependency on the hypervisor of which they run on it is not only important for the hypervisor selection to be mutual for both platforms to ensure fair testing but it is just as important to ensure that the selected hypervisor is evaluated in terms of its standard and industry recognition.

After performed the performance evaluation of the platforms by using a mutual hypervisor against the set of identified criteria, we highlighted some interesting results to make recommendations. This is important to ensure the research has been meaningful and served a purpose not only in terms of simply benchmarking the platforms in question but also helping to identify further improvements.

In general, this study also aims to serve as a base point in order for performance benchmarking to be completed on more platforms. This in turn will help to create a bigger picture in relation to the performance benchmarking of a wide variety of platforms which can then be compared against once such research has been conducted.

This paper is structured as follows. In section 2, we present the findings of a comprehensive literature review. We demonstrate the deployment methods of the systems in section 3. In the following section 4, we show the performance evaluation testing results. Section 5 discusses performance trends and suggested improvements. Finally, we conclude in section 6 and highlight aims and objective of the paper outlines the further work.

II. LITERATURE REVIEW

There are many existing lines of research, simulations and performance testing around cloud platforms both in public and private environments.[2, 3] Although this paper specifically evaluates OpenStack and CloudStack, two open source cloud

computing solutions, there are also many other cloud computing solutions available all of which offer similar solutions with slight varying differences from features and performance all the way to pricing. It is important not to disregard other products and previous research and it is even more important to ensure a firm understanding is in place of previous findings with a critical viewpoint to enable the structure of this research to be the most informative at the time.

A. Platform evaluation studies

In 2011 [4] conducted a study on evaluating open-source cloud computing solutions in which they developed an extensive list of possible criteria for which solutions may be evaluated against. In this study they try and set a standard based on research into the important areas within cloud computing and categorize this list into several main categories which include support, security, network, management, storage and virtualization. Each criterion they have identified fall into one of these categories.

This study is related in some capacity to the one being conducted however it is more of evaluation criteria for the general features of a product versus the performance which this research looks to measure. The criteria were also specifically designed for enterprise deployments, something this project doesn't have the capacity nor the scope for yet the categories make use for an interesting possible criteria scope in this project.

As with all research there are different avenues which can be taken regarding the analysis of a system. This could be feature analysis all the way through to performance with sub divisions for example performance in a specific scenario vs. overall software performance. A study by [5] investigates a comparison between three open source platforms; Eucalyptus, OpenNebula and Nimbus in which although they test the systems this is purely for the functionality of the features they have to offer. Typically feature sets of software could be pulled from the vendors' literature however they demonstrate the need to testing the flexibility and extent of such features to perform a detailed cross comparison. Features can relate to performance however this still leaves an open gap in research for correlating the features they have identified with the performance of the system. After all, what use is a system with lots of features which performs badly.

Similarly, [6] also perform a comparison of open source cloud platforms but this time the comparison is between Eucalyptus, OpenNebula, Abicloud and Nimbus. Much like [5] this research compares the features of the platforms in generalities such as its intended use, license information, programming language, interoperability, security and the platforms computability to interact with current cloud platform offerings such as Amazon EC2.

It is important to understand that in contrast to other studies such as those mentioned above, the research to be conducted in this project will provide a general overview of performance as oppose to features which will hopefully in turn correlate to future feature/performance related studies. An interesting feature of this study is to consider the base language as a comparison point. If other future studies are conducted on

platforms with matching base languages this could serve as a point for correlation to help identify performance trends. Such studies are yet to conclude on a final point and lead into further fields of research.

Interestingly, although for different platforms, both [6] and [6] conclude on their studies outlining the feature differences in platforms they also outline areas for further work from feature improvements through to future studies now they have identified feature differences in the platforms. It is here where we have identified the main gap for a performance related study at a high overview level to bring some academic research into this currently untouched area.

B. Test methodologies

In the above previous test methodologies were identified in the various overlapping studies however when concluding on their effectiveness for this study not only are they unsuited to the way in which this study looks to measure performance but they are also measured against a specific application or application layer as opposed to measuring the performance of a platform. The study by [7] showed the importance of performing the study on the platform you wish to test against as oppose to a similar setup and considering this to be suitable as a scaled solution to the setup you propose your results for. The particular example included a study in which a cluster was tested and considered to be a cloud computing solution.

As with all platforms although there is a reliance on the underlying hardware for performance there also serves a link between the hardware and the software in the way this can effect performance with the kernel code being the interlaying feature between the various layers of software and hardware. Obviously, as with all test methodologies, especially in a comparison situation it is important they are performed on identical hardware with the closest configuration possible.

As well as the configuration of the test bed it is equally as important to consider the way in which to test the platforms to give the general high level performance overview of these two platforms. A study on Virtual Machine Monitors (VMM), a term for a hypervisor, was undertaken by [8] in which they used performance monitoring tools to successfully monitor the behavior of the machines running on the hypervisor. This however gives rise to concern particularly for this study in that the test methodology used primarily analyses the virtual machines performance in relation to the hypervisor as oppose to the platform of which is used to manage the machines. This wouldn't be suitable for this study as it still needs to test the cloud platforms basic tasks such as machine provisioning times, deployment times and deletion times and the effects on the hypervisors performance per se.

The contributions of this paper are not only to fill this gap in the research but also to contribute to providing a benchmarking set of criteria for cloud platforms. This will allow further studies to be conducted on additional platforms helping to further fill this void in the current research area. This coupled with the conclusions drawn from the performance testing of the platforms will give good ground for the importance and understanding of such platforms.

III. DEPLOYMENTS OF THE SYSTEMS

Both OpenStack and CloudStack are deployed and configured as virtual instances using Oracle VM VirtualBox 4.2.10 on a host machine configured with an Intel Core i7-2630QM running with full Virtualization Technology (VT) enabled alongside 8GB DDR3 memory and 700GB of hard drive storage. The choice of VirtualBox to run the platforms was down to both platforms, “Dev” Cloud and Stack, respectively offer system build options for this particular type 2 hosted hypervisor of which our tests will be conducted on.

OpenStack requires a different installation configuration from CloudStack, which is to be expected with them both being different products; however it is required to be installed as a virtual machine (VM) on the XCP hypervisor. To do this the XCP is installed as an instance using Virtual Box and subsequently once this completed the OpenStack could be created as a VM running within it.

To maintain a fair test as both system builds are configured in a single virtual machine in line with the development deployment options for both platforms each will have allocated 4 CPUs and 4GB of RAM with a single file 100GB virtual disk to both virtual machines making the comparison fair. The exact machine specifications will be detailed in the relevant proceeding sections.

The OpenStack deployment has been carried out in keeping with the recommended deployment infrastructure for development and testing purposes. The entire machine resources essentially pool from those of the XCP hypervisor and are distributed according. OpenStack was deployed on top of the XCP hypervisor allowing for this to take place whilst ensuring the mutual hypervisor remained consistent.

Although having a similar architecture the devcloud installation to install the CloudStack development/testing environment follows a more simplified installation process in comparison to OpenStack. The process allows for one partially pre-configured VirtualBox virtual machine file to be downloaded followed by a few subsequent configurations before a fully operational devcloud environment is ready for testing/development purposes.

IV. PERFORMANCE EVALUATION

To perform a fair testing evaluation across the two platforms the tests needed to be run on the same specification virtual machines. OpenStack uses flavors to define machine build specification. The main settings defined within the flavor include the number of CPU's, the amount of RAM and also the hard disk storage space. In CloudStack this is split out slightly differently with the CPU and RAM being allocated under a different flavor to the disk. To make this a fair test the below offerings were created on each platform. The names, tiny, small and medium shown on table 1 and table 2 were maintained in keeping with the default naming of flavors/offerings. All tests were performed twenty times to identify any anomalies when plotting the trends onto graphs.

In the proceeding sections a variety of the flavorings above are used to test both the platforms performance in performing a variety of tasks for different variations of flavors to identify

their performance in relation to particular machine specifications and if this has any correlation on the performance of the system.

All deployment and deletion test time tests were times from the moment the cloud platform was instructed to deploy/delete the VM to the moment it was reported in the console as running/deleted. For the CPU performance measurements a command was used to capture the average performance of all cores on the hypervisor. These tests were run following the deployment and deletion tests all of which took no longer than 30 seconds however to ensure a safety net was created the command logged the CPU performance over a period of a minute collecting the data every second and reporting the average.

Table 1 - CPU and memory flavor specifications

CPU and memory flavours			
Name displayed on results	Cloud Name	Platform	Memory
1 CPU, 100MB RAM	tiny		100MB
1 CPU, 512MB RAM	small		512MB
2 CPU, 1024MB RAM	medium		1024MB

Table 2 - hard disk size flavors

Hard disk flavours		
Name displayed on results	Cloud Platform Name	Hard disk
1GB HDD	tiny	1GB
5GB HDD	small	5GB
20GB HDD	medium	20GB

To ensure fair testing and there being no load on the environments each virtual machine was deleted following the test meaning the no virtual machine were running and the cloud platforms were not managing any instances which would in turn effect the performance.

4.1 Processor/RAM effect on deployment times

With processor and memory specifications varying on machines in any environment, whether that is domestic or commercial in a cloud for example, the specification of the machine will largely depend on its purpose and the requirements of the platform(s) it is required to run. To identify any possible correlation between deployment times and machine specification (processor and memory) this study tested the three machine processor/ram specifications with the same hard disk size.

As shown in Fig. 1 above, although there are some clear differences between the three instances, albeit with a couple of anomalies, it is clear to see that there is a correlation between the processor and memory specifications versus the machine deployment time when a larger number of CPUs are allocated. Although, that being said, the scale is miniscule resulting in the machine deployment time difference only being within a few seconds as the instance specification increases. This shows the possibility of the dependence on available CPUs within the host as to the time taken to deploy the machine as both tests with 1 CPU showed similar correlation although they had varying amounts of memory.

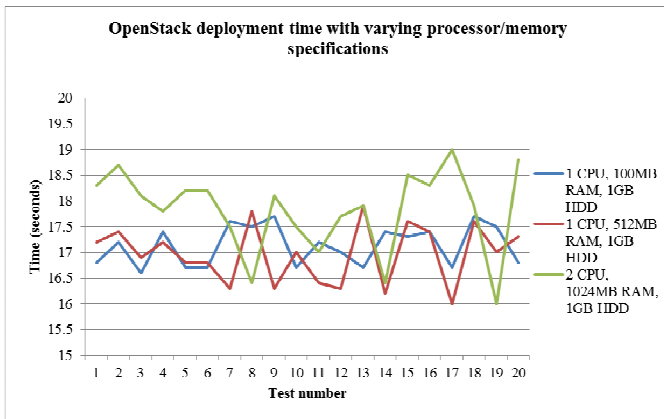


Fig. 1. OpenStack machine deployment times with varying processor and memory specifications

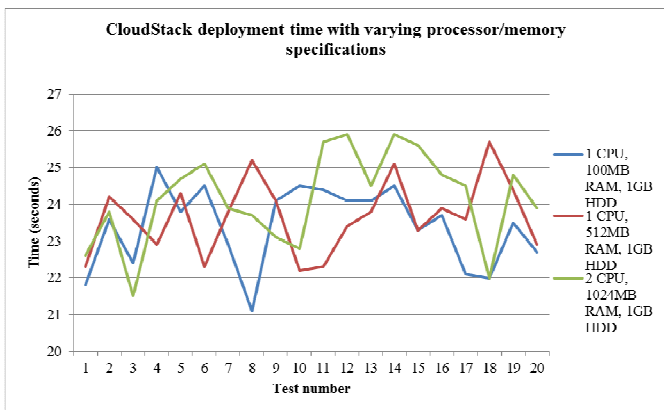


Fig. 2. CloudStack machine deployment times with varying processor and memory specifications

Interestingly, as shown above in Fig. 2 CloudStack shows little to no correlation in processor and memory specifications versus deployment time. It could be said this may be expected with what should only be a quick calculation of required resources versus the available resources therefore explaining the lack of correlation. However there was some correlation shown on the OpenStack platform.

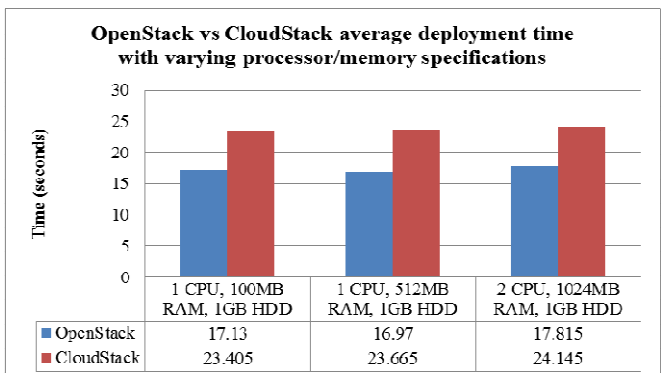


Fig. 3. Average OpenStack and CloudStack machine deployment times with varying processor and memory specifications

Fig. 4 helps to further identify the increasing deployment time in OpenStack as the number of processors increase albeit a difference in milliseconds and taking into account the anomalies there still lies the difference. Across all tests

OpenStack shows a more efficient machine deployment time in comparison to CloudStack.

4.2 Processor/RAM effect on deletion times

Having run the above tests in the prior section of this report whilst using the same variable of processor and memory configurations in this section the deletion times can now be tested to see if there is a link between the time taken to delete a virtual instance versus the specification of the instance.

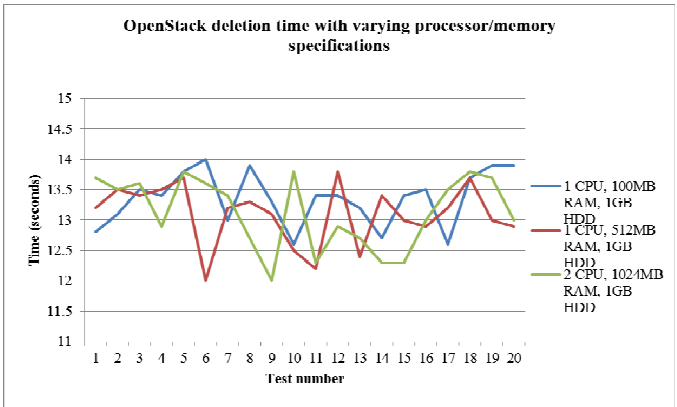


Fig. 4. OpenStack machine deletion times with varying processor and memory specifications

Interestingly, as Fig. 3 shows, there is no correlation between the processor and RAM specification in the time taken to delete the machine on the OpenStack platform. This doesn't tie inside the correlation demonstrated for deployment times, particularly with an increased number of processors. The results are all within two seconds of one and other.

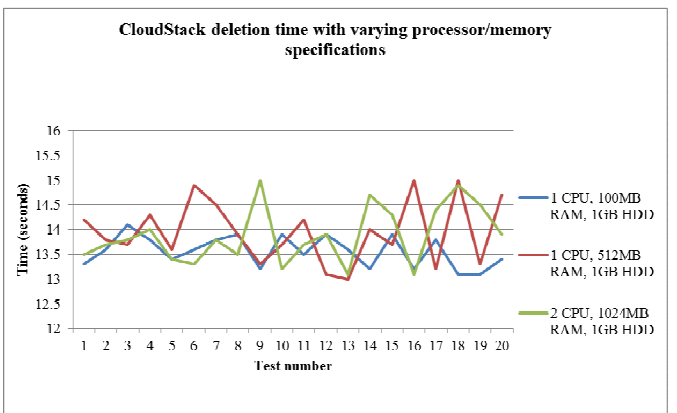


Fig. 5. CloudStack machine deletion times with varying processor and memory specifications

Fig. 5 shows that again CloudStack does not show correlation nor similarity to the performance of the cloud platform in relation to deleting instances of varying specifications. Although all instances are deleted only within a few seconds of one and other, similar to OpenStack, and there are no large outsider anomalies. The fluctuation of the results for all instance sizes demonstrates the lack of correlation between instance specification and deletion time.

In Fig. 6 below, this performance is also evidenced in the averages and it is also clear to see that OpenStack

demonstrated faster deletion times across all specification variations in comparison to CloudStack. A similar conclusion to that drawn from the testing in section 4.1.

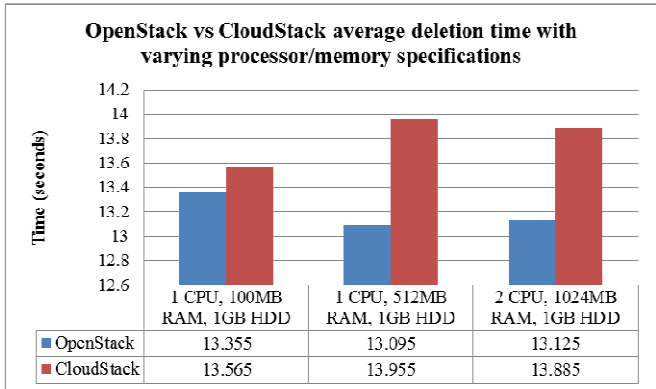


Fig. 6. Average OpenStack and CloudStack machine deletion times with varying processor and memory specifications

4.3 Hard drive size effect on deployment times

Having already discovered some correlation in the previous tests around the effect of processor and memory specifications on certain cloud platform performance tasks, now the effect of hard drive size can be testing in comparison to deployment time.

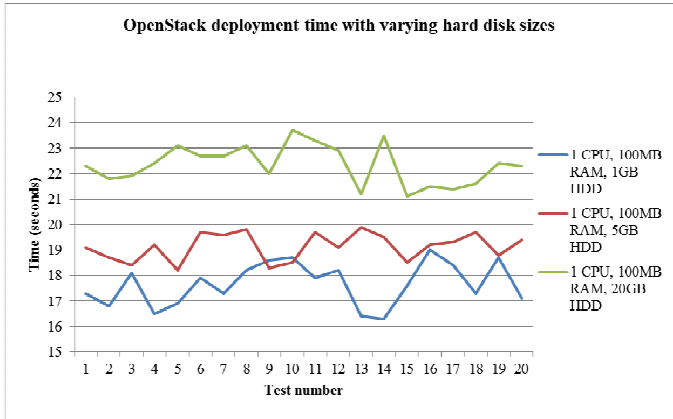


Fig. 7. OpenStack machine deployment times with varying hard disk sizes

Fig. 7 shows OpenStack provides a very strong correlation between the disk size when taking into account the time taken to create the virtual machine instance. There is particularly a noticeable difference as the hard drive size increases from 5GB to 20GB showing the definite increase of deployment time in relation to the hard disk size.

Whilst the there is still a correlation with the CloudStack tests as shown above in Fig. 8, it does not demonstrate the relative correlation shown in the same test for OpenStack, particularly the difference in deployment times for the varying hard drive sizes. OpenStack showed a slight increase in deployment time between a 1GB hard disk and a 5GB hard disk whilst showing a greater difference between a 5GB and 15GB hard disk, in relation to the size increase. CloudStack however shows the differences the opposite way round. It is unlikely that all twenty results for the 5GB hard disk are anomalies. That being said the correlation still shows that hard

disk size increases deployment time, just not in the relative relation as demonstrated by OpenStack.

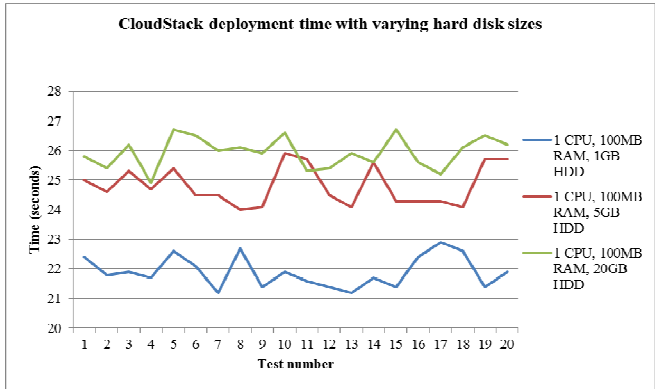


Fig. 8. CloudStack machine deployment times with varying hard disk sizes

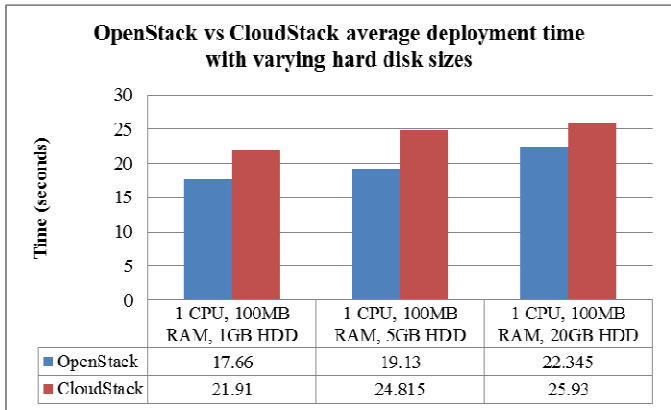


Fig. 9. Average OpenStack and CloudStack machine deployment times with varying hard drive sizes

Fig. 9 shows a clear average performance variation between the two platforms, again with OpenStack performing the better of the two. The first test shows a difference of over 4 seconds between the two platforms which then increases at a similar rate as the hard drive size increases. The two platforms both demonstrate a relation between deployment time and hard drive size unlike the deployment time with varying processor and memory specifications.

4.4 Hard drive size effect on deletion times

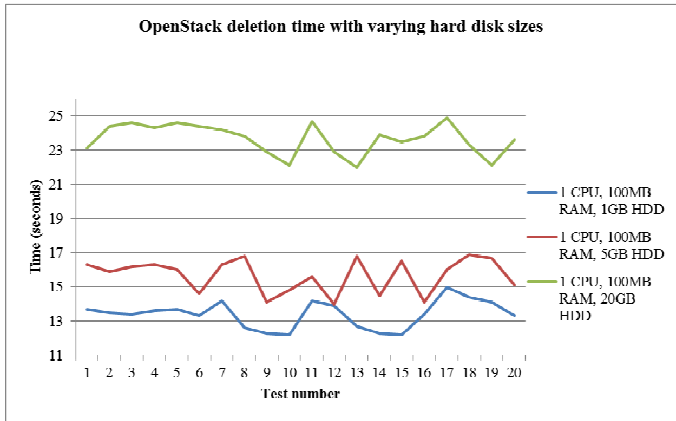


Fig. 10. OpenStack machine deletion times with varying hard disk sizes

Similarly to the processor and memory tests performance tests are run to identify deletion times with varying hard disk sizes. It was clear from the previous set of tests that the hard drive size affected the deployment times.

Fig. 10 below shows the same correlation in deletion time between the disk sizes as it did in deployment time in section 4.3. This not only shows the relative relationship between disk size and deletion times but also that there's a relationship between deployment and deletion with varying hard disk sizes with increased times when a larger hard disk is provisioned.CloudStack deletion times in relation to hard disk size show a stronger and more relative correlation to that of the deployment times identified in the experiment in section 4.3. The results shown in Fig. 11 identify a clear effect on the systems performance as the same processing power for machine deletion takes an extended amount of time which increases as the hard disk size increases.

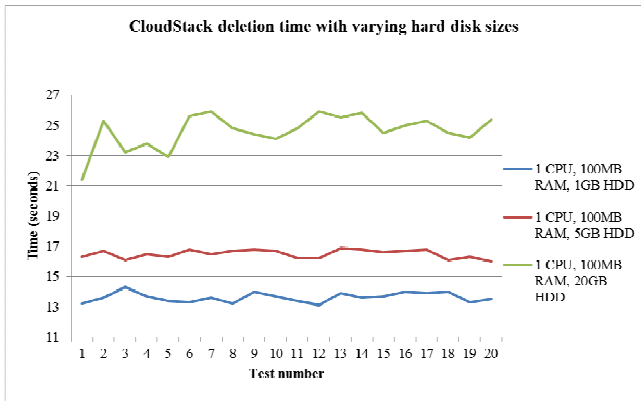


Fig. 11. CloudStack machine deletion times with varying hard disk sizes

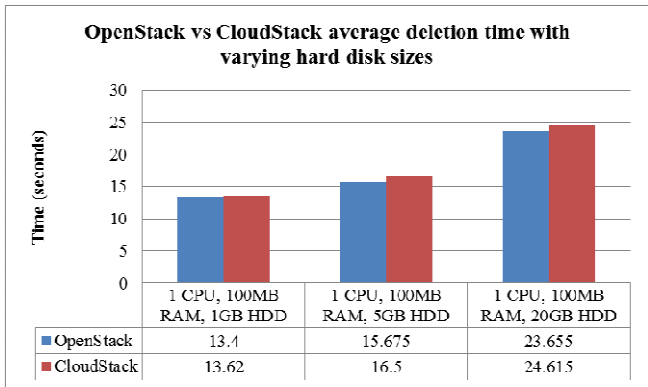


Fig. 12. Average OpenStack and CloudStack machine deletion times with varying hard drive sizes

Again, following the trend that has been identified in the sub section summaries throughout these tests, Fig. 12 demonstrates OpenStack's better performing averages in comparison to CloudStack. This time however, the difference is miniscule and although there is a difference it could also be argued that the platforms perform at similar levels in this particular benchmark test.

4.5 Hard drive size deployment effect on host CPU utilisation

In sections 4.1 through to 4.4 tests were conducted to evaluate the platforms performance in deploying and deleting instances with varying processor/RAM and hard disk configurations. It was clearly evidenced from the results that hard disk configurations are the only variable of the two that demonstrated any performance correlation. Taking this into account in this section and section 4.6 tests are conducted on hard drive size and the effect it places on the average host CPU utilisation. The average was used over a set period of time as it is possible that the deployment process utilises several processes combined for successful deployment and therefore this aggregates the statistics for all processes running on the host which at that particular moment in time will only be all machine deployment processes.

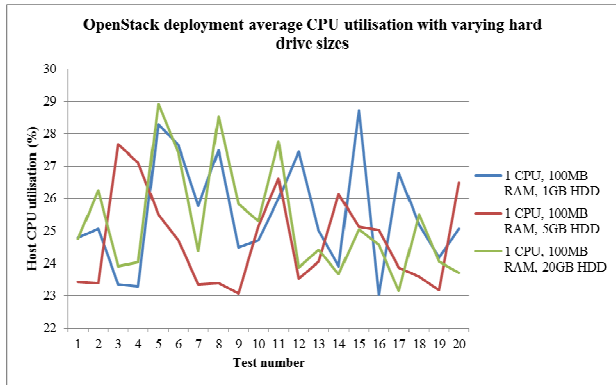


Fig. 13. OpenStack machine deployment average CPU utilisation with varying hard drive sizes

As shown in Fig. 13, there are some clear fluctuations demonstrated on the host performance usage when machines are deployed, none of which are in a relative relation to the machine hard disk size. It is therefore not possible to define any correlation between the hard drive size and host CPU utilisation during deployment for OpenStack. All fluctuations however do fall between 23% and 29% with none going above this which could demonstrate a host CPU limit for machine deployments.

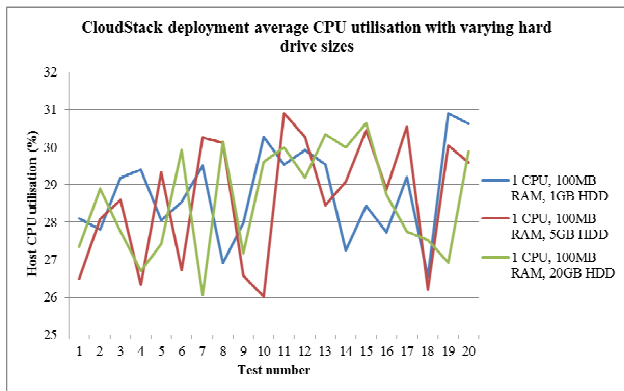


Fig. 14. CloudStack machine deployment average CPU utilisation with varying hard drive sizes

Similarly to OpenStack as shown in Fig. 14, CloudStack also demonstrates no correlation between machines hard disk

size and the host CPU utilisation. Results peak and through for all hard disk sizes throughout the tests but with a higher lower and upper boundary of 26% and 31%.

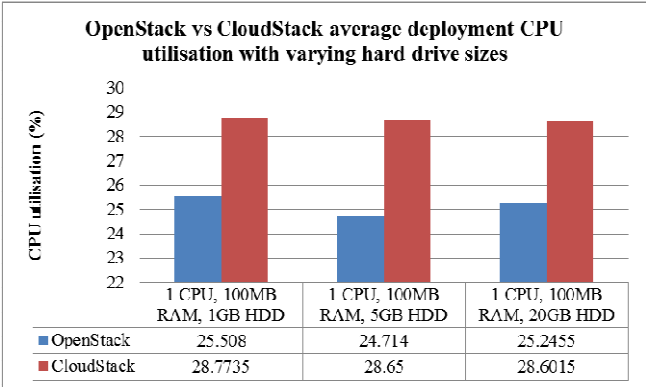


Fig. 15. Average OpenStack and CloudStack machine deployment CPU utilisation with varying hard drive sizes

As the findings in the two previous sections demonstrated, there is no correlation between the hard drive size and the host CPU utilisation during deployment. This is further evidenced by the averages for both platforms as shown in Fig. 15. The averages are all within 1 second of one and other on either platform. Although, the averages here do demonstrate OpenStack’s less resource demanding way of deploying machines as also demonstrated it its deployment time tests for both hard drive size and processor and memory configurations in the previous sections.

4.6 Hard drive size deletion effect on host CPU utilisation

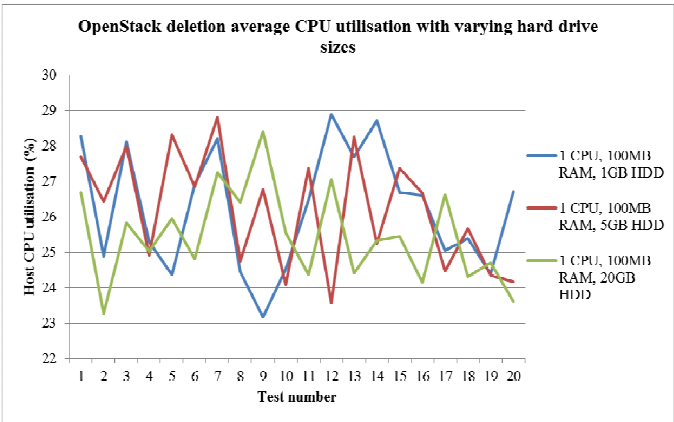


Fig. 16. OpenStack machine deletion average CPU utilisation with varying hard drive sizes

The results for the deletion average CPU utilisation for OpenStack shown in Fig. 16 also show no correlation, similar to that of the deployment tests in the previous section. These results tie in with the previous identified possibility that CPU utilisation may be limited for deployment and also deletion although this is not documented in the platforms online documentation.

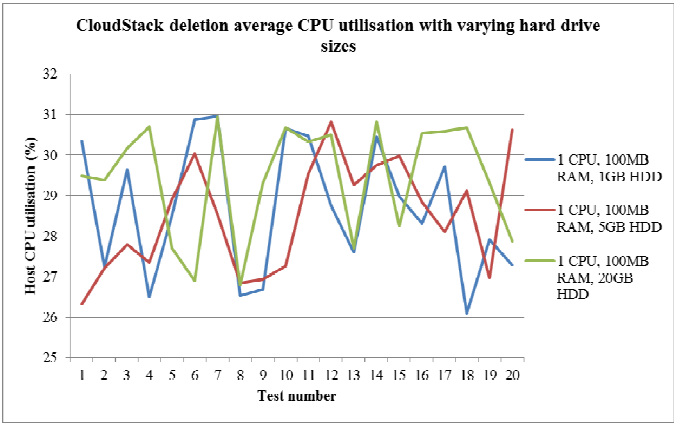


Fig. 17. CloudStack machine deletion average CPU utilisation with varying hard drive sizes

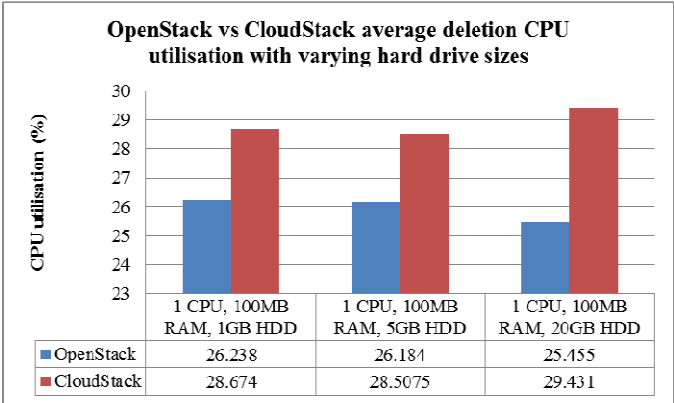


Fig. 18. Average OpenStack and CloudStack machine deletion CPU utilisation with varying hard drive sizes

Again, Fig. 17 shows as expected having previously conducted three other tests of a similar nature, CloudStack has no correlation between the host average CPU utilisation and the hard disk size when completing the management task of deleting a virtual machine instance. All results fluctuate but in a limit of 26% to 31% with no anomalies outside of this range.

Having already summarised throughout these two tests the findings for both platforms are further evidenced in Figure 23. This figure further supports the findings in that there is no distinct variation in average CPU utilisation when deleting virtual machines with varying hard disk sizes.

4.7 Experiment discussions

Having outlined the performance trends in relation to the given scenarios in section 4 and its subsections any identified performance trends and rooms for improvement in the platforms can be outlined, particularly in relation to one and other with no previous similar benchmarking being available for such cloud platforms.

When reviewing the results and identifying the performance of the systems it is crucial to keep in mind the scale of difference between results. Across both platforms and all tests with results having differences between 0.1 and 11.2 seconds the scale on which performance can be measured needs to be reduced. The identified performance against the

benchmarks needs to be relative to the variation in the test results.

Generally as an overall consensus it is clear to see that OpenStack offers the best performance out of both the cloud platforms with reduced times taken in the basic tasks one would be undertaking from within the console on a cloud computing management platform. Whilst undertaking the studies it was clear to see that OpenStack had the more basic and less glamorous user interface console of the two platforms however it is unclear if this serves any relation to the enhanced performance it offers in completing tasks. OpenStack also demonstrates a fairly consistent performance even at its increased performance level over CloudStack. With both platforms being developed by Citrix and released under the Apache license it would normally be assumed that there's a fair chance the platforms were developed similarly using the same programming language. However, this isn't the case. OpenStack is written in Python whilst CloudStack is written in Java which could help to demonstrate a potential reasoning behind the performance variations.

With the performance testing having identified some areas of improvements for CloudStack it is important that these are fed back for investigation and improvement. With CloudStack being a community driven platform for maintenance and refinements the proposed plan to have these further investigated and possibly implemented in new revisions will be to submit a summary of the findings to the CloudStack community. The CloudStack dedicated Apache web page has a published maintainers guide for different areas and given the areas for which the performance needs to be investigated and software possibly revised it is possible for the results of this study to be submitted to the following: Chiradeep Vittal (Storage/Volumes), Nitin Mehta (Templates) and Rohit Yadav (DevCloud) [9].

The main identified trends outlined in each of the summary subsections in section 4 show that although CPU and memory configurations may increase as machines are deployed these have little to no effect on the deployment and deletion times on both platforms. Hard drive size however is where both platforms performance show a clear relationship in terms of the time taken to complete the task in relation to the size of the hard disk. As the hard disk size increases, so does the time taken to complete the task.

OpenStack demonstrated a more relative correlation to the size of the increase leading to a relative increase in time taken also. CloudStack on the other hand, although time taken increased with the increased size of hard disk it did not always increase at a relative rate to the hard drive size increase. In addition to this the project has now opened up further avenues of research particular around ways for improving the CloudStack platforms performance in the areas identified and also in benchmarking performance of other cloud platforms.

V. CONCLUSION AND FURTHER WORK

This paper has examined the performance variations between the two cloud platforms, OpenStack and CloudStack. Through the study it can be clearly identified that the

performance of OpenStack supersedes that of CloudStack when measured against the benchmark criteria. This is evidenced and summarised throughout the tests performed in section 4 where not only did OpenStack perform tasks in less time than that of CloudStack on all tasks it also showed more relative correlation in its results, particularly relating to hard disk sizes. By doing this, the study has successfully fulfilled the original aim in benchmarking the two open source cloud platforms.

There are many future studies which can now follow from this study. These studies however are not limited to strictly testing more platforms against the same criteria. Having identified the limitations of the testing environment this could also lead to repeating this study in a real world data centre environment to identify any possible differences in results creating correlation links between the development and production platforms. Future work could also help contribute to an increased range of benchmarking criteria and a definitive selection for all future studies creating a solid set of testing criteria allowing for relative comparisons between a range of cloud management platforms.

ACKNOWLEDGMENT

The work reported in this paper has been supported by the Sino-UK Higher Education Research Partnership for PhD Studies, Natural Science Foundation of Jiangsu Province of China (BK20130528), RLTF e-learning Cloud Programme, Visiting Research Fellow Program of Tongji University (8105142504), National Natural Science Foundation of China Program (61202474 and 61272074) and China 973 Fundamental R&D Program (2011CB302600).

REFERENCES

- [1] L. Liu, O. Masfary, and N. Antonopoulos, "Energy Performance Assessment of Virtualization Technologies Using Small Environmental Monitoring Sensors," *Sensors*, vol. 12, pp. 6610-6628, 2012.
- [2] J. Li, Y. Jia, L. Liu, and T. Wo, "CyberLiveApp: A Secure Sharing and Migration Approach for Live Virtual Desktop Applications in a Cloud Environment," *Future Generation Computer Systems*, Elsevier Science, vol. 29, pp. 330-340, 2013.
- [3] L. Liu, D.-A. DaSilva, N. Antonopoulos, Z. Ding, and Y. Zhan, "Achieving Green IT Using VDI in Cyber Physical Society," *Journal of Internet Technology*, vol. 14, pp. 413-424, 2013.
- [4] I. Voras, B. Mihaljevic, M. Orlic, M. Pletikosa, M. Zagar, T. Pavic, *et al.*, "Evaluating open-source cloud computing solutions," in *MIPRO, 2011 Proceedings of the 34th International Convention*, 2011, pp. 209-214.
- [5] P. Sempolinski and D. Thain, "A comparison and critique of eucalyptus, opennebula and nimbus," in *Cloud Computing Technology and Science (CloudCom), 2010 IEEE Second International Conference on*, 2010, pp. 417-426.
- [6] S. Wind, "Open source cloud computing management platforms: Introduction, comparison, and recommendations for implementation," in *Open Systems (ICOS), 2011 IEEE Conference on*, 2011, pp. 175-179.
- [7] V. Goswami, S. Patra, and G. Mund, "Performance analysis of Cloud with queue-dependent virtual machines," in *Recent Advances in Information Technology (RAIT), 2012 1st International Conference on*, 2012, pp. 357-362.
- [8] J. Che, Q. He, Q. Gao, and D. Huang, "Performance measuring and comparing of virtual machine monitors," in *Embedded and Ubiquitous Computing, 2008. EUC'08. IEEE/IFIP International Conference on*, 2008, pp. 381-386.
- [9] Apache. (2013, January 27, 2014). *Apache CloudStack Documentation*. Available: <https://cloudstack.apache.org/docs/zh-CN/index.html>

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/372618959>

Comparative Study of the Support of Academic Clouds Based on Apache CloudStack and Proxmox VE Platforms

Conference Paper · January 2021

DOI: 10.5220/0012064300003431

CITATIONS

3

READS

225

3 authors:



Vasyl Oleksiuk

Ternopil Volodymyr Hnatiuk National Pedagogical University

53 PUBLICATIONS 284 CITATIONS

SEE PROFILE



Olesia Oleksiuk

Ternopil Regional Communal Institute of Postgraduate Pedagogical Education

13 PUBLICATIONS 131 CITATIONS

SEE PROFILE



Oleg Spirin

Institute for Digitalisation of Education of the National Academy of Educational S...

100 PUBLICATIONS 651 CITATIONS

SEE PROFILE

Comparative Study of the Support of Academic Clouds Based on Apache CloudStack and Proxmox VE Platforms

Vasyl P. Oleksiuk^{1,2}^a, Olesia R. Oleksiuk³^b and Oleg M. Spirin^{2,4}^c

¹*Ternopil Volodymyr Hnatiuk National Pedagogical University, 2 M. Kryvonosa Str., Ternopil, Ukraine*

²*Institute for Digitalisation of Education of the National Academy of Educational Sciences of Ukraine, 9 M. Berlynskoho Str., Kyiv, 04060, Ukraine*

³*Ternopil Regional Municipal Institute of Postgraduate Education, 1 V. Hromnytskogo Str., Ternopil, 46027, Ukraine*

⁴*University of Educational Management, 52A Sichovykh Striltsiv Str., Kyiv, 04053, Ukraine*
fi

Keywords: Cloud Computing, Academic Cloud, Apache CloudStack, Proxmox VE, Google Workspace.

Abstract: Cloud technologies provide users with efficient and secure tools for data management, computing, storage and other services. The article analyzes the projects for the introduction of cloud technologies in education and identifies the main advantages and risks in creating a cloud infrastructure for the university. Such startups contribute to the formation of a new paradigm of education. It involves the virtualization of education, the introduction of mobile and blended learning, ie the combination of cloud computing with modern learning concepts. In this paper, we compare some aspects of deployment and support in our experience in improving the academic cloud for the training of a bachelor's degree in computer science. This is through the integration of the Proxmox VE platform into existing computing power by deploying the Proxmox VE system. In the study, we reveal some technical and methodological aspects of the organization of the educational process using this corporate cloud platform. The scheme of the organization of physical components of cloud infrastructure (nodes, virtual networks, routers, domain controller, VPN-server, backup system of students' virtual machines) is given. All characteristics of this environment and possibilities of their application are studied.

1 INTRODUCTION

Today, many universities are creating their own cloud-based learning environments (CBLE). Although there is currently no single concept for CBLE, scientists understand it as similar concepts (Glazunova and Shyshkina, 2018; Korotun et al., 2020; Bykov et al., 2020; Vakaliuk, 2021; Spirin et al., 2022). In general, it can be understood as an IT system consisting of cloud services and providing learning mobility, group collaboration of teachers and students to achieve educational goals (Lytvyanova, 2018).

As the analysis of the literature shows, many university CBLE are usually deployed according to the hybrid model (Shyshkina, 2016; Markova et al., 2019; Vorozhbyt, 2017; Vakaliuk et al., 2021). One of the most important components in the structure of this environment is the private academic cloud (Glazunova,

2015). It is now deployed according to the most productive IaaS service model. A hybrid cloud is a cost-effective way to solve the problem of insufficient computing resources. The private academic cloud allows the university to meet the peak demand of students and faculty through a combination of local infrastructure and one or more public clouds (Wang et al., 2020).

Various commercial and free platforms are used in universities to build private academic clouds (AL-Mukhtar and Mardan, 2014; Ilin et al., 2016; Amiel et al., 2020). A productive method of deploying private academic clouds is to use solutions from leading cloud vendors such as Google, Microsoft, Amazon and others. Google offers researchers, universities, faculties, faculty and students grants and loans for teaching and research. In particular, leading European educational institutions can access Google Cloud within the Internet2 project (Internet2, 2023). Unfortunately, these opportunities are not currently available for our country (Ukraine). For example, Microsoft Educator Grant is a program designed specif-

^a  <https://orcid.org/0000-0003-0789-0272>

^b  <https://orcid.org/0000-0001-5659-4774>

^c  <https://orcid.org/0000-0002-9594-6602>

ically to provide access to Microsoft Azure to college and university professors teaching advanced courses. As part of this program, faculty teaching Azure in their curricula are awarded subscriptions to support their course (Microsoft, 2023). In general, these programs are very useful and productive. However, they are usually provided on a temporary basis and therefore cannot completely replace the cloud-based IT infrastructure of universities and colleges.

Among the free platforms for cloud infrastructure deployment, CloudStack, Openstack, Proxmox VE, and Eucalyptus are the most suitable. Each of them has its advantages and disadvantages. There are many attempts to compare these platforms. The authors of articles comparing such platforms state (Microsoft, 2023; Fylaktopoulos et al., 2016):

- OpenStack has large community, offers wide integration with storage, network and compute technologies, but is too complex to deploy and configure;
- Eucalyptus the longest-standing open source project is banking on its very tight technical ties to Amazon Web Services (AWS). The platform is configurable, but not very customizable;
- Proxmox is open source platform. It can provide easy way to deployment cloud infrastructure. But it is not very suitable as a platform for a private cloud in the CLBE;
- CloudStack has well rounded GUI, can provide an advanced cloud infrastructure, but it is very GUI centric built on single Java core.

We have been deploying a private academic cloud based on the Apache CloudStack platform for the past 8 years. For the last 2 years we have been deploying an academic cloud based on Proxmox VE as well. Apache CloudStack deployment is housed in the infrastructure building. It contains a management server, 4 hosts, 4 primaries and 1 secondary storage. We decided to use hypervisors instead of containers. This is because the former are more versatile. In addition, the use of hypervisors is safer than containers. To save computing resources, we have installed primary storage on the hosts. We used VLANs to distribute traffic across individual networks. These networks can be allocated to groups or individual students.

We have so far rented one refurbished physical server based on a used Intel Xeon processor with 32 Gb of RAM for deployment of Proxmox VE. This was also done because Ukraine is currently at war and it is difficult to say whether servers will be available as they are in university buildings.

In general, our private academic cloud provides (Fylaktopoulos et al., 2016):

- Development and Execution of student virtual machines;
- Aggregation of computing resources of several hosts;
- VM migration between repositories;
- VM connections to each other through guest networks;
- Launching VMs within other VMs;
- Integration with Active Directory;
- Distribution of student accounts according to their academic groups.

There are many advantages and disadvantages to using Apache CloudStack and Proxmox VE as an academic cloud platforms. There are some problems happens in the process private academic cloud using.

The purpose of the article is to compare some tasks in maintenance of academic clouds based on Apache CloudStack and Proxmox VE platforms.

The following tasks are required to achieve the goal of the research:

1. Analysis of maintenance tasks of academic clouds in foreign and Ukrainian universities.
2. Definition of maintenance tasks of academic clouds based on Apache CloudStack and Proxmox VE platforms.
3. Describe and compare methods for solving these tasks.

We used a set of research methods such as theoretical (analysis of scientific, technical literature, experience; generalization of experience of using cloud computing in education) and empirical (modelling, designing, developing of scripts).

2 THE PRIVATE CLOUDS MAINTENANCE TASKS

As the experience of cloud infrastructure maintenance shows, this is an ongoing process. It requires constant attention from engineers, network system administrators, teachers, and student involvement. Scientists describe the experience of deploying an academic private cloud, including determining the performance of hypervisors and storage (Spirin et al., 2018). The biggest challenge for researchers was the transition from a prototype of an academic cloud to a productive one. In this context, they addressed the problem of load balancing, elastic hypervisors, security

threats. Storage backup tasks are also important for such clouds.

Rao and Nayak (Rao and Nayak, 2014) offer the concept of backup to cloud storage. They say that both the cloud provider and the cloud consumers have to take comprehensive steps to ensure appropriate configurations, hardening of the CBLE, appropriate design and development, appropriate interoperability, and adequate testing.

Rusyn et al. (Rusyn et al., 2019) have developed an effective method of deduplication and distribution of data in cloud storage during the creation of backups. Researchers have developed an intelligent system for such deduplication and tested it.

Tian et al. (Tian et al., 2020) also studied cloud data backup. The authors propose a scheme for data separation and backup and encryption. They state that their own scheme resolves the conflict between data security and the survivability of the IT infrastructure with the help of encrypted backup.

The Apache CloudStack cloud infrastructure redundancy model developed by Paul Angus is very useful for our study. It creates a vendor agnostic API and UI in CloudStack for end users. The author's Framework abstracts the specifics of solutions, such that through the use of a plugin, a 3rd party solution can deliver backup and recovery solutions (Khmelevsky and Voytenko, 2015).

Here are some tasks for servicing our sample private academic cloud:

1. Making changes to the cloud infrastructure.
2. Creating and verification of the academic cloud performance model.
3. Designing and realization an academic clouds' backup model.

Let's consider these tasks in more detail.

3 MAKING CHANGES TO THE CLOUD INFRASTRUCTURE

Maintenance of our developed cloud infrastructure involves the implementation of tasks such as:

- changing the parameters of the components of the cloud infrastructure – zones, clusters, hosts, storages;
- creating and routing of virtual networks for individual groups or students;
- creating and modifying templates of compute offering services that determine the performance of VM;

- creating and modifying network offering service templates such as VPN, DHCP, DNS servers, Firewall, Load Balancer and others;

- creating projects for VM sharing by students.

This task also involves creating student and faculty accounts. We authenticate users of the academic cloud from a centralized database – LDAP-directory (Microsoft Active Directory). This approach makes it possible to use single registration data to access all hybrid IT infrastructure services. We used CloudStack domains to distribute students according to academic groups. Adding users to them is possible in automatic (using links at the first successful authentication) and manual mode. Unfortunately, due to the incompatibility of our users' logins with the Apache CloudStack platform, we had to choose the manual mode. To reduce the technical work involved in finding LDAP directory entries, we have created several queries to filter user account data.

Proxmox VE also provides authentication based on the LDAP directory (Active Directory). To do this, cloud administrator need to specify some parameters such as socket to connect to the domain controller, user data for reading data from the directory, some user attributes, filters to restrict access.

To log in to the system, the administrator can add it manually, for example using the web interface. However, it is possible to automatically synchronize the directory with the *pveum realm sync <realm>* command. Like Apache CloudStack domains, Proxmox VE can create groups. For these groups, cloud administrator can define privileges for access to cloud infrastructure.

Apache CloudStack users automatically have access to their own VMs. This is good for students to complete individual tasks. The creation and providing permission to VMs isn't a trivial task in the Proxmox VE Platform. We offer two approaches to do this. The first involves the administrator creating copies of VMs and giving students access to these VMs. Permissions may vary depending on the operations to be performed with these VMs. For example, cloud administrators can provide students with administrative access to such VMs. Another approach involves students cloning their own machines from the templates. It requires providing some rights. They are displayed as a table 1.

Creating permissions that match the last row of the table requires defining *<students.login>* - *<VM_ID>* pairs. Finally, permission is granted through the command:

```
pveum acl modify /vms/<WM_ID> <username>
    -role Administrator
```

Table 1: Permissions are required to clone student VMs from templates.

Permissions	Path	Propagate
PVEAuditor	/	+
PVEDatastoreUser	/storage	+
PVETemplateUser	/pool	+
PVEVMUser	/nodes	+
Administrator	/vms/VM_ID	–

Later, using a script, we obtained a list of users from the LDAP directory and set the appropriate IDs for them. It is important that when cloning VM, students must indicate the ID provided by the teacher for VM.

3.1 The Academic Cloud Performance Model

Performance calculations are needed to understand the number of VMs that can run in the cloud. It is also needed to understand the computing performance of these machines, which is required to create their templates.

Both platforms use CPU and memory redundancy depending on the number of VMs. VE allows cloud administrators to do this more flexibly when running containers instead of VMs.

When creating the service offering templates, we compared the characteristics of the hardware hosts (CPU frequency, RAM) with the minimum guest OS requirements and the number of students. To do this, we used the inequality:

$$FRQ = N_{st} * F_{OS} < FRQ_{hosts},$$

where FRQ – the total frequency of VMs processors; N_{st} – amount of students; F_{OS} – the minimum frequency is recommended for the guest OS; FRQ_{hosts} – total frequency of hardware host processors. The last value can be found from the ratio:

$$FRQ_{hosts} = \sum_{i=1}^n (N_{ci} F_{ci}),$$

where N_{ci} – the number of cores in the processor of the i -th host, F_{ci} – CPU frequency of the i -th host.

Unlike Apache Cloudstack, Proxmox VE does not have performance templates. Appropriate parameters must be set for each VM during its installation. If a student clones a VM from a template, he/she cannot change its performance settings. Instead, when using the VM, these parameters are available for change. However, the last two formulas are also valid for the Proxmox VE platform. In order to compare the computing capabilities of the studied cloud platforms, we will consider the hardware characteristics of the respective servers (table 2).

Table 2: Main characteristics of academic cloud's hardware.

N	N_{ci}	F_{ci}	MEM_{ci}	FRQ_{hosts}
Host0	4	3200	16384	12800
Host1	4	3100	24576	12400
Host2	4	3100	16384	12400
Host3	4	3700	32768	14800
CloudStack	16		90112	52400
Proxmox VE	8	3500	28000	31980

As table 2 shows, the private academic cloud based on Apache CloudStack has a total frequency of about 50GHz. And the total amount of memory is about 90 Gb.

Unfortunately, we do not have the equipment to deploy identical academic clouds based on the CloudStack and Proxmox VE platforms. However, we can assume that the performance of the physical server with Proxmox VE should be equal to the computing power of the two servers with CloudStack (Host0 and Host2). Therefore, in the CloudStack deployment, we disabled Host1 and Host3. During the testing, Host1 stayed online as a management server in the CloudStack infrastructure. But it didn't run any VMs.

Regarding the frequency, two other opposite factors should be taken into account:

- table 2 shows the base frequency, and processors can run faster thanks to Turbo-Boost technology;
- hosts run other software (OS, databases, management servers, hypervisors). It also consumes resources.

It is well known that the frequency of a modern processor is not constant. It can increase or decrease depending on the mode of operation of the CPU. That's why we use Processor Base Frequency in the tables and formulas above. Processor Base Frequency describes the rate at which the processor's transistors are open and close. The KVM hypervisor measures this processor frequency. It is software for launching and executing virtual machines in the Apache CloudStack and Proxmox VE platforms.

Similarly, to determine the required amount of memory we used the inequality:

$$MEM = N_{st} * MEM_{OS} < MEM_{hosts}$$

3.2 Verification of the Academic Cloud Performance Model

To check computation performance of both clouds we loaded the same VMs on two clouds until their CPU load was 90%. There were several types of test virtual machines (table 3). These types corresponded to the VM templates that were created for the various training courses. For example, templates Lin-

uxGUI, WindowsWs were used in the course of operating systems, AdvLinux in the course of computer networks, WindowsSrv in the course of administration. The Linux OS with CLI worked as Proxmox VE containers and as regular VMs in CloudStack. We use the EVE-NG platform based on AdvLinux template for modelling in the study of computer networks. It launches its own VMs inside the main Apache CloudStack VM (Spirin et al., 2019). Such nested virtualization requires more resources.

Table 3: Some templates types of our academic cloud's performance.

Template	F_{OS}	MEM_{OS}	N_{st}	FRQ	MEM
LinuxCLI	500	500	20	10000	10000
LinuxGUI	1500	2000	20	30000	40000
WindowsWs	1000	2000	20	20000	40000
WindowsSrv	1500	2000	20	30000	40000
AdvLinux	2500	4000	20	50000	80000

The number of VMs that were loaded is shown in table 4.

Table 4: Comparison of the number of VMs running on Apache CloudStack and Proxmox VE platforms.

Template	CloudStack	Proxmox VE
LinuxCLI*	50	120
LinuxCLI	112	120
LinuxGUI	18	19
WindowsWs	20	22
WindowsSrv	16	16
AdvLinux	12	8

Comparing the data in table 2 and table 3, we can conclude that our deployments of both platforms provides about 50 VMs with Linux with command line interface (CLI), more than 40 VMs with Windows Workstation, about 35 VMs with Windows Server and OC Linux with GUI (see table 4). This test showed that the number of virtual machines loaded on Proxmox VE is slightly higher than in Apache CloudStack (except for the template AdvLinux).

To verify the performance model, we performed academic cloud testing. The experiment was performed according to the method described in the article (Algarni et al., 2018). The authors' model used traditional benchmarking systems to measure system performance and individual components like CPU, memory, cache and disk performance. Like the previous case, we compared the performance of two CloudStack hosts and one Proxmox VE. To test performance of clouds we have created 16 identical VMs (8 on Apache CloudStack and 8 on Proxmox

VE). They had equivalent 2-core processors, the same amount of RAM (2 Gb) and the size of disks (14 Gb). The same Ubuntu server 20.04 x64 was installed on all these VMs.

We have performed real-tasks tests such as:

- Ab is a tool for benchmarking Apache web server. Ab was used for measuring of time taken for the system to respond to a request from a web client. This can estimate the response efficiency of each VM by measure the number of static web page requests a server can fulfil in one second.
- The LZMA benchmark measures the amount of time consumed in compressing a file using its own compression algorithm.
- John the Ripper is a tool for measure CPU throughput. In this benchmark, the efficiency of the process management of the hypervisors is evaluated. This benchmark is a common decipher application which uses diverse ciphers such as DES and MD5.
- IOzone benchmark is used to evaluate performance of file system. IOzone generates and assesses many file operations such as read, write and random read.

Ab was used for measuring of time taken for the system to respond to a request from a web client. This can estimate the response efficiency of each VM. Our script have completed 100000 requests with concurrency level 1000 simultaneous requests. The results of all test available by the URL <https://drive.google.com/file/d/1m-3oS7cOWyCFUveBe8t4EnHOcll6ApyX>. Shared table contains the designations of VMC for VMs Cloudstack and VMP for Proxmox VE ones. The average values of ab-test are given in the table 5.

Table 5: Ab benchmark results.

Platform	Requests per sec	Transfer rate, Kb/s
CloudStack	2115,68	95150,89
Proxmox VE	1787,62	80437,84

In the Apache benchmark, CloudStack performance is 18% higher in terms of both the number of requests and transfer rate. A slightly smaller advantage of Apache CloudStack (near 7%) was shown by the LZMA test. We performed John the Ripper benchmarks for the case of coding according to DES 128 and MD5 algorithms. Because the test was run on a single CPU core, the "real" and "virtual" results were the same. We analyzed the results for the single-salt case. They indicate (table 6) that Apache CloudStack has significantly advantage under Proxmox VE (38% and 54% for DES 128 and MD5 algorithms).

Table 6: John the Ripper benchmark results

Platform	MD5 hash	DES 128 hash
CloudStack	14526,13	61055,88
Proxmox VE	10503,75	39524,38

Similar results were obtained in the performance test of virtual HDDs (figure 1).

Therefore, it is reasonable to predict that the performance of the virtual machines themselves will be higher on the Cloudstack platform. It is hoped that students will feel this in their academic research work. However, another important factor in this assessment is the functionality and usability of the interface. The Proxmox VE console interface is more advanced than Apache Cloudstack. This fact, together with a good mobile version of the platform allows you to work with the cloud from different devices.

The previous test concerned the performance of the virtual machines themselves. We decided to investigate the performance of hardware servers (hosts) with their running hypervisors. This test will allow us to evaluate the efficiency of the use of computing resources by cloud platforms. To do this we used the formula to rank hypervisors' performance proposed by Reddy and Shyamala (Reddy and Shyamala, 2016). According to it, the hypervisor's score scores of its CPU and memory.

$$Score_{HSI} = Score_{HCPU} + Score_{HMEM}$$

Each of these scores is calculated as the sum of the CPU and RAM performance of any VM in the cloud.

$$Score_{HCPU} = \sum_{i=1}^n CP_{VMi}$$

$$Score_{HMEM} = \sum_{i=1}^n MP_{VMi}$$

The CP_{VMi} and MP_{VMi} values are calculated as follows

$$CP_{VMi} = \alpha_{CPU} \times \frac{ACPU_{VMi}}{HCP_{VMs}}$$

and

$$MP_{VMi} = \alpha_{MEM} \times \frac{AM_{VMi}}{HMP_{VMs}},$$

where α_{CPU} and α_{MEM} are CPU and memory weight factors; $ACPU_{VMi}$ and AM_{VMi} are available CPU and memory resources of VMi; HCP_{VMs} and HMP_{VMs} are all VMs relative CPU(RAM) utilization with respect to host.

The formulas for calculating the values HCP_{VMs} and HMP_{VMs} are as follows

$$HCP_{VMs} = \frac{TotalCPU - \sum_{i=1}^n ACPU_{VMi}}{TotalCPU} \times 100,$$

$$HMP_{VMs} = \frac{TotalMEM - \sum_{i=1}^n AMEM_{VMi}}{TotalMemory} \times 100,$$

where $ACPU_{VMi}$ and $AMEM_{VMi}$ are available CPU(RAM) of VMi.

Alpha coefficients are the ratio of processor resources (memory) of one VM to all VMs. Since we have all 8 machines the same, for each platform $\alpha_{CPU} = \frac{1}{8}$ and $\alpha_{MEM} = \frac{1}{8}$.

We applied the above formulas and obtained the following values for the studied platforms (table 7).

Table 7: The obtained scores of Cloudstack and Proxmox VE platforms.

	AB	Izma	iozone
Score.CloudStack	77,24	68,55	349,29
Score.Proxmox	79,03	87,18	284,85

Thus, we can state that the score of both platforms based on the Apache benchmark test is almost the same (the difference is about 1.5%). Proxmox VE showed the best results in the Izma test (about 27%). The opposite result of the advantage of Apache Cloudstack is observed in the iozone test. However, we have doubts about this case, because the disk subsystems of the two platforms are significantly different (Proxmox VE uses a software raid, and Cloudstack generally contains two separate physical hosts with HDDs). Diagrams of the obtained ratings are shown in the figure 2.

So, based on the verification of the cloud platform performance model, we can say that the Apache Cloudstack and Proxmox VE have roughly equal ratings. In naked academic clouds, the performance of 2 physical Apache Cloudstack hosts is approximately equal to the performance of one Proxmox VE host. In this case, other factors of the cloud platform at the university will be decisive.

4 DESIGNING AND REALIZATION AN ACADEMIC CLOUDS' BACKUP MODEL

The backup model depends on which platform will be stored. There are general backup principles that are valid for both platforms. However, the technical implementation will be different. We had to develop scripts for the Apache Cloudstack. Proxmox VE has powerful built-in backup tools.

Experience shows that the task of backup is very important and time consuming. This is primarily due to the large amounts of student VMs data in the private academic cloud. Large companies develop a dis-

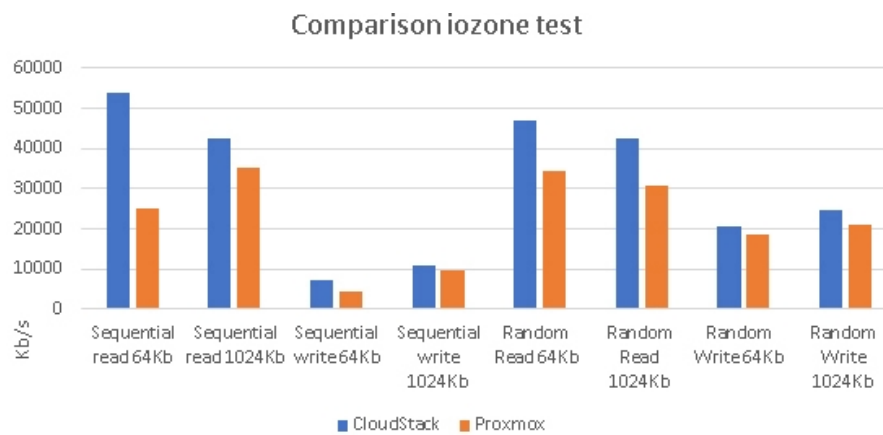


Figure 1: Comparison of the results of iotest.

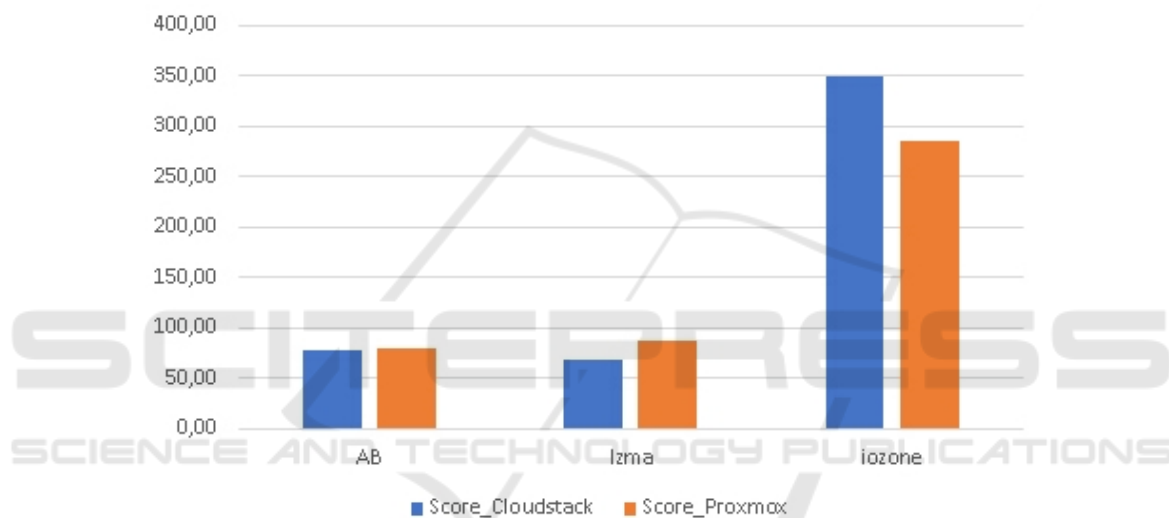


Figure 2: Comparison of the Apache CloudStack and Proxmox VE scores.

aster recovery plan in this case. But in educational institutions, IT services work to perform such tasks. Therefore, they need to develop a model, identify potential risks in the IT infrastructure, consider and implement an appropriate backup system.

The development of a backup strategy requires the definition of the main goals and objectives of the backup, tools and regulations. In general, the problem of back-up is relevant for almost all IT infrastructures. When choosing a backup method, the following criteria are important (Angus, 2019):

- backup time to the storage;
- recovery time from backup;
- the number of copies that can be stored;
- risks due to inconsistency of backups, imperfection of the backup method, complete or partial loss of backups;
- overhead costs: the level of load on the servers

when performing copying, reducing the speed of service response, etc;

- the cost of renting all services and storage.

Currently, there are 3 main backup schemes such as:

- *Full*. This type of backup creates a complete copy of all data.
- *Incremental*. In this case, only files that have changed since the previous backup are copied. The following incremental backup only adds files that have been modified since the previous backup.
- *Differential*. The backup program copies each file that has changed since the last full backup. Differential copying speeds up the recovery process (Paulsen, 2011).

To save material costs, we use almost no server equipment and powerful and high-speed network stor-

age in our academic cloud installation. Instead, we decided to use cloud services. For example, the Google Drive service within the Google Workspace for Education Fundamentals package offers virtually unlimited disk space (Oleksiuk et al., 2017). The disadvantage of such a repository is the significant time to upload or download backups. This speed will be limited by the bandwidth of the university's Internet channel. The latter requirement can be considered acceptable, as our implementation of the academic cloud is used primarily for training rather than for production.

To use Google Drive in our own scripts, we need to use the API of this service. This interface is accessible through Google Developers Console, a software developer service. First you need to create your own project. Credentials were created to access this project. We have chosen to access OAuth 2.0 accounts. OAuth is an open authorization standard that allows a user or application to give and access data without having to enter a login and password. Access tokens are used for this purpose. Each access token provides access to a specific client to specific resources and for a specified period of time (Oleksiuk et al., 2017). After adding a new project, we created new data for authentication, selected the type of application (desktop) and activated the appropriate API (Google Drive API).

Because templates and ISO images in the CloudStack infrastructure do not change, but only new ones are added, we chose the incremental method to back up the secondary storage. Its implementation was based on the use of a ready-made utility for synchronizing storage files. Unfortunately, there is currently almost no such high-quality utility like Google backup and Sync, which is developed for OC Windows. We analysed several tools such as:

- Gdrive (grive2). Google Drive client with the support for the new Drive REST API and partial sync. It can't provide continuously wait for changes in file system or in Google Drive to occur and upload.
- Gnome-online-accounts. It is system utility located within system's settings in Gnome GUI. But it can only be executed in a graphical interface.
- GoSync is a Google Drive client with GUI support for Linux. It is designed under the GNU General Public License. The client is not perfect enough, for example, it has automatic regular synchronization every 10 minutes.
- Google-drive-ocamlfuse is a FUSE (Filesystem in Userspace) filesystem for Google Drive, written in OCaml. FUSE is a free module for the kernel

of Unix-like operating systems. It allows developers to create new types of file systems available for users to mount without the root privileges of Google Drive on Linux (Yallop et al., 2018).

We used the latest utility. Here are its main features:

- full read/write access to ordinary files and folders;
- read-only access to google docs, sheets, and slides;
- multiple account support;
- duplicate file handling;
- access to trash;
- storing Unix permissions and ownership;
- support symbolic links;
- streaming through read-ahead buffers.

Some problem was that the utility requires authorization using a browser in a graphical interface. Therefore, we used an alternative authorization mode. Since we already had our own OAuth2 client ID and client secret, we specified them in the command:

```
google-drive-ocamlfuse -id
12345678.apps.googleusercontent.com
-secret abcde12345
```

As the command tries to start the browser on the server where there is no GUI we formed the necessary URL as it is written in the documentation on Google Developers Console. After going to this address, we received a verification code. This code gave access to folder synchronization to the Google Drive.

For security reasons, we decided to sync not the Apache CloudStack secondary storage itself, but a copy of it from the backup drive. So, we first synchronized local folders with the command:

```
rsync -azvh /export/secondary
/export/sync_secondary/arch_cloud
```

where */export/secondary* – the secondary storage of Apache CloudStack infrastructure;

/export/sync_secondary/arch_cloud – the local copy of this storage.

To synchronize the */export/sync_secondary/arch_cloud* folder, the following command has been added to the server task scheduler:

```
google-drive-ocamlfuse
/export/sync_secondary
```

It runs every time a server with secondary storage is loaded.

A backup of all databases is required to restore the Apache CloudStack cloud infrastructure. These are such databases:

- *Cloud*. It contains all objects of cloud infrastructure.
- *Cloud_usage*. A database that contains generalized data on resource consumption by the end user. It is used to obtain statistics and compile reports.

Since the backup of these databases is quite small, we decided to store all backups in the cloud storage (Backup_Database task, figure 3). The traditional database for the Apache CloudStack platform is MySQL. The main utility for backing up MySQL databases is mysqldump. Its syntax involves entering a login name and password. Because the shell script in Linux is written as a plain-text file, it will contain the name of the user's password (usually the root) of the database. This is a potential security risk for the entire server. In order not to leave open the data for authorization of the database user, we used the "login path" option. A "login path" is an option group containing options that specify which MySQL server to connect to and which account to authenticate as. To create or modify a login path file, we have used the mysql.config editor utility. In general, the commands for creating and archiving a database dump are as follows:

```
/usr/bin/mysqldump
--login-path=DailyBackup -u root -A >
$BACKUP_DIR/"_cloud_all_"$date_daily".sql"
```

The variable *\$date_daily* contains the current date of the archive. This allows you to see the date of archiving directly in the file name.

```
tar -czf
$BACKUP_DIR/"archive_cloud_all_$date_daily.sql.tgz"
$BACKUP_DIR/"archive_cloud_all_$date_daily.sql"
```

To upload the files to the server, we used a ready-made script from GitHub. Here is its launch:

```
upload.sh "arch_cloud/DB" "$entry" $upl_file
folder_ID "application/x-gzip"
```

where

- *arch_cloud/DB* – folder for uploading files;
- *\$entry* – full path to the file;
- *\$upl_file* – file name to download;
- *folder_ID* – Google Drive folder ID;
- *application/x-gzip* – file MIME-type.

A special refresh_token token is required to provide long-term access of the up-load.sh script to Google Drive. It can be obtained by curl-calling a URL such as:

```
curl -silent "https://accounts.google.com/o/\
oauth2/token" --data "code=<access_token>&
```

```
client_id=<client_ID>&
client_secret=<client_secret>&
redirect_uri=urn:ietf:wg:oauth:2.0:oob&
grant_type=authorization_code"
```

In general, the scheme of backup of Apache Cloudstack infrastructure is shown in figure 3.

Performing backup of primary repositories (Backup_Primary task (cloud0,cloud1,cloud2,cloud3)) has some difficulties. An analysis of Internet sources, management server databases, and storages files showed that the Apache CloudStack platform does not typically use full copies of disk templates for each VM. This means that full backups should be made to reduce the risk of inconsistencies in primary repository archives.

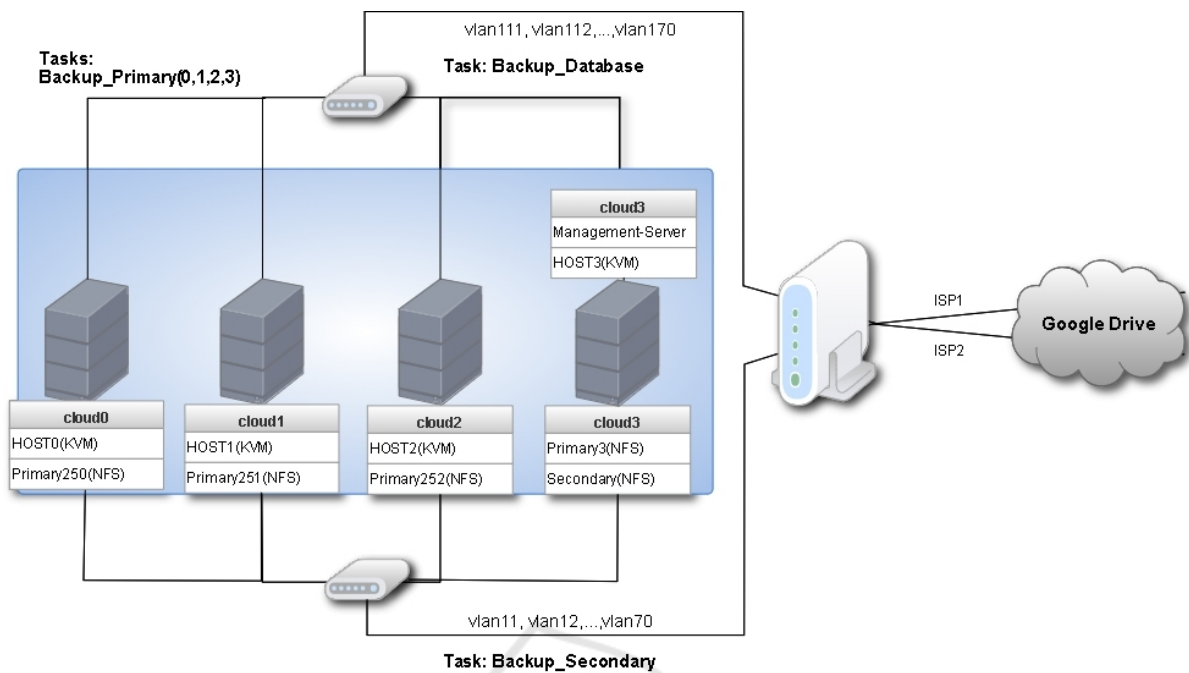
Additionally, it is necessary to prepare a cloud platform, stopping all VMs. Of course, students need to form an understanding of the need to turn off their own VMs. However, in practice this is not always possible. Therefore, it is necessary to stop all VM programmatically, by means of a script. This can be done using the API features of the Apache CloudStack platform. Using API functions allows the developer to access data about cloud infrastructure objects. It is also possible to change the state of these objects. To generate a query that contains API functions, you must specify:

- URL of the management server;
- Service construct "api?". It contains the path to a certain API-function, and indicates the beginning of the parameters that are transmitted using the GET method.
- Command. It is the name of the API-function.
- ApiKey. The key, that can be generated for each user account.
- Additional query options separated like GET queries using the "&" character.
- Response format (JSON or XML).
- Signature of the request.

Regardless of the protocol (HTTP or HTTPS) used to access the Apache Cloud-Stack API functions, the request must be signed. This allows the platform to confirm that the request was sent from a trusted accounting request that has the authority to execute the appropriate command. To sign a request, the developer must have an API key and an account secret key. They are generated by the platform administrator (OAuth, 2020).

Here is our bash-script to stop all working users' VMs

```
mysql --login-path=DailyBackup -D cloud
-e "SELECT uuid FROM vm_instance WHERE
```



ID	Node	Status	Name	Type
245	Debian-101...	stopped	CT245	lxc
246	Debian-101...	stopped	CT246	lxc
247	Debian-101...	stopped	VM254	qemu
248	Debian-101...	stopped	CT248	lxc
249	Debian-101...	running	VM249	qemu
250	Debian-101...	running	mikrotik	qemu

Figure 4: Proxmox VE backup job.

```
type = \"User\" and state = \"running\";"  
> uuid.txt
```

```
sed -i 'ld' uuid.txt
```

```
while read LINE; do php -q  
  cloudstackapi.php "$LINE" ;  
done < uuid.txt
```

In the first line we receive in a file from a database the list of user VM with a running state. The next

command clears the first line because it does not contain a VM. The third line runs the cloudstackapi.php script. It generates a signature and calls the stopVirtualMachine API.

Another way to back up the current state of the VM is to create their snapshots. The Apache CloudStack platform provides 2 types of images (Rusyn et al., 2019):

- VM Snapshot – a hypervisor-driven point-in-time

image of a virtual machine's disks. The exact mechanism of this is dependent on the hypervisor.

- Volume snapshot – a point-in-time image of a specific volume. The process usually involves taking a VM snapshot and then copying the required volume to secondary storage and the deleting the VM snapshot.

This approach requires additional space on the secondary storage or data coping on the user's local disk. Such images can be taken by students from the web interface of the Apache CloudStack platform. Performing this action and turning off their own VMs after the end of their use are important components of ICT competence of the student.

However, experience shows that not all students perform these actions. Therefore, these are also worth automating with scripts. Among the API functions of the Apache Cloudstack platform are relevant (Sneddon, 2017).

Another task of backing up our academic cloud is to estimate the time required to upload data to the cloud storage. Currently (June 2022) the sizes of our academic cloud storage is approximately as follows:

- primary250 – 132 Gb;
- primary251 – 70 Gb;
- primary252 – 126 Gb;
- primary253 – 88 Gb;
- secondary – 110 Gb.

Since we make a full copy of the primary storage, we need to download about 400 GB to the cloud storage each time. Let the speed of the Internet channel at night be 80 Mbps (10 Mbytes per second). Then it will take 11 hours to download 400*1024 MB. That's a lot. Therefore, we balanced Internet access through 2 providers. At the time of backup, our router routes hosts cloud0 and cloud1 through the first provider, and cloud2 and cloud3 through the second. In this case, a full backup takes about 6 hours. This time is also significant, but is acceptable.

Another disadvantage of our scheme is the significant time required to download backups from the Google Drive service. However, this time will be significant if the management or storage servers fail. This means that we must back up the entire OS of the management server to fast local area network storage.

As mentioned earlier Proxmox VE provides many tools, including storage and backup types. To back up all cloud infrastructure, it is advisable to install and configure Proxmox Backup Server. It is an enterprise backup solution, for backing up and restoring VMs, containers, and physical hosts. By supporting incremental, fully deduplicated backups, Prox-

mox Backup Server significantly reduces network load and saves valuable storage space. Unfortunately, we do not have the physical resources to install a separate Proxmox Backup Server. Nevertheless, we have performed some tasks from the cloud infrastructure backup model.

Stop Students' VMs. There is no need to use Proxmox API to perform this task. We already have student VM_ID (used them to establish access rights). To stop these VMs on the Proxmox host, we executed the commands `qm shutdown VM_ID` or `pct stop container_ID`. It is advisable to use such commands to enable (disable) student VMs and containers at the beginning (end) of classes.

VM Reservation. We have set up a scheduled task to back up the cloud infrastructure to an NFS share. It is connected to our university local network via an OpenVPN tunnel. This allows us to save the condition of all machines.

Standard Proxmox VE tools allow cloud administrator to perform this task flexibly. The following backup options are available (figure 4):

- host (node) to be copied;
- storage for which to perform job;
- date and time of the task;
- backup objects such as some VMs, all VMs, all except some VMs;
- compression method (LZO, GZIP, ZSTD, without compression)
- copy mode (snapshot, suspend, or turn off VM)
- email notification about the status of the job or errors during its execution

The advantage of this method is the reservation of working VMs "on the fly". Like Apache CloudStack we use VMs backup from Proxmox VE cloud infrastructure into Google Drive storage.

5 CONCLUSIONS

The private academic clouds should be used in cloud based learning environment, as they are necessary for education of future ICT specialists. Despite the availability of educational grants from leading cloud vendors, many universities are deploying their own private academic clouds. Cloud administrators have a lot of work to do to maintain and support these academic clouds. Among these tasks, one of the most important is to ensure the productivity and elasticity of the cloud. Solving them will allow them to load the maximum number of VMs in the cloud infrastructure.

In this study, the performance of academic clouds based on Apache Cloudstack and Proxmox VE was evaluated. Despite some assumptions in comparison, we can state that the platforms have approximately the same performance and therefore can provide the same amount of VM for students. This is because they both use the same KVM hypervisor. As our experience has shown, the process of cloud deployment based on Proxmox VE is technically simpler and faster. This is even though we have installed Apache CloudStack many times and Proxmox VE for the first time.

An important task in the maintenance of the academic private cloud is the backup of its components. To solve it effectively, you need to use different backup schemes such as full, incremental, differential. To save data, it is advisable to use both cloud and local storage. In any case, administrators should determine how long it will take to build and restore the entire cloud infrastructure. It is also advisable to use the API functions of the cloud platform. This will automate some maintenance tasks.

To back up Apache Cloudstack, we had to use API and develop our own tools. In any case, this process requires large repositories and fast networks. If the first requirement is not met, it is advisable to use cloud storage. We did so and copy the backups to Google Drive. Backup of Proxmox VE infrastructure is technically simpler. To ensure the full functionality of this process, it is advisable to install a Proxmox Backup Server. All of the above factors indicate that it is time to migrate our academic cloud to the Proxmox platform. Maybe for a while we'll use them both together.

REFERENCES

- AL-Mukhtar, M. M. A. and Mardan, A. A. A. (2014). Performance Evaluation of Private Clouds Eucalyptus versus CloudStack. *International Journal of Advanced Computer Science and Applications*, 5(5). <https://doi.org/10.14569/IJACSA.2014.050516>.
- Algarni, S. A., Ikbali, M. R., Alroobaea, R., Ghiduk, A. S., and Nadeem, F. (2018). Performance Evaluation of Xen, KVM, and Proxmox Hypervisors. *International Journal of Open Source Software and Processes (IJOSSP)*, 9(2):39–54. <https://doi.org/10.4018/IJOSSP.2018040103>.
- Amiel, T., ter Haar, E., Vieira, M. S., and Soares, T. C. (2020). Who Benefits from the Public Good? How OER Is Contributing to the Private Appropriation of the Educational Commons. In Burgos, D., editor, *Radical Solutions and Open Science: An Open Approach to Boost Higher Education*, Lecture Notes in Educational Technology, pages 69–89. Springer Singapore, Singapore. https://doi.org/10.1007/978-981-15-4276-3_5.
- Angus, P. (2019). CloudStack Backup and Recovery Framework. <https://www.slideshare.net/ShapeBlue/cloudstack-backup-and-recovery-framework>.
- Bykov, V., Mikulowski, D., Moravcik, O., Svetsky, S., and Shyshkina, M. (2020). The use of the cloud-based open learning and research platform for collaboration in virtual teams. *Information Technologies and Learning Tools*, 76(2):304–320. <https://doi.org/10.33407/itlt.v76i2.3706>.
- Fylaktopoulos, G., Goumas, G., Skolarikis, M., Sotiropoulos, A., and Maglogiannis, I. (2016). An overview of platforms for cloud based development. *SpringerPlus*, 5(1):38. <https://doi.org/10.1186/s40064-016-1688-5>.
- Glazunova, O. (2015). *Theoretical and methodological bases for the design and application of an e-learning system for future IT specialists in an agrarian university*. Thesis for the degree of Doctor of Pedagogical Sciences, specialty 13.00.10 - "Information and Communication Technologies in Education", Institute of Information Technologies and Learning Tools of the NAPS of Ukraine, Kyiv, Ukraine. <https://lib.iitta.gov.ua/9212/>.
- Glazunova, O. G. and Shyshkina, M. (2018). The Concept, Principles of Design and Implementation of the University Cloud-based Learning and Research Environment. In Ermolayev, V., Suárez-Figueroa, M. C., Yakovyna, V., Kharchenko, V. S., Kobets, V., Kravtsov, H., Peschanenko, V. S., Prytula, Y., Nikitchenko, M. S., and Spivakovsky, A., editors, *Proceedings of the 14th International Conference on ICT in Education, Research and Industrial Applications. Integration, Harmonization and Knowledge Transfer. Volume II: Workshops*, Kyiv, Ukraine, May 14-17, 2018, volume 2104 of *CEUR Workshop Proceedings*, pages 332–347. CEUR-WS.org. https://ceur-ws.org/Vol-2104/paper_158.pdf.
- Ilin, D. Y., Volovich, M. E., and Filatov, V. V. (2016). Analysis of CloudStack Platform Suitability for Management of Different Cloud Infrastructure Configurations. *Cloud of Science*, 3(3):433–443. http://web.archive.org/web/20180422194515/http://cloudofscience.ru/sites/default/files/pdf/CoS_3.433.pdf.
- Internet2 (2023). Google Cloud Services (GCP). <https://www.internet2.edu/products-services/cloud-services-applications/google-cloud-platform/>.
- Khmelevsky, Y. and Voytenko, V. (2015). Hybrid Cloud Computing Infrastructure in Academia. In *WCCCE 2015 - the 20th Western Canadian Conference on Computing Education*. May 8-9, 2015, Vancouver Island University (VIU), Nanaimo, British Columbia, Canada. <https://doi.org/10.13140/RG.2.1.4082.6647>.
- Korotun, O. V., Vakaliuk, T. A., and Soloviev, V. N. (2020). Model of using cloud-based environment in training databases of future IT specialists. *CTE Workshop Proceedings*, 7:281–292. <https://doi.org/10.55056/cte.360>.
- Lytvynova, S. H. (2018). Cloud-oriented learning environ-

- ment of secondary school. *CTE Workshop Proceedings*, 5:7–12. <https://doi.org/10.55056/cte.129>.
- Markova, O. M., Semerikov, S. O., Striuk, A. M., Shalatska, H. M., Nechypurenko, P. P., and Tron, V. V. (2019). Implementation of cloud service models in training of future information technology specialists. *CTE Workshop Proceedings*, 6:499–515. <https://doi.org/10.55056/cte.409>.
- Microsoft (2023). Student developer resources. <https://azure.microsoft.com/en-us/resources/students/>.
- OAuth (2020). Using OAuth 2.0 to Access Google APIs. <https://developers.google.com/identity/protocols/oauth2>.
- Oleksiuk, V., Oleksiuk, O., and Berezitskyi, M. (2017). Planning and Implementation of the Project “Cloud Services to Each School”. In Ermolayev, V., Bassiliades, N., Fill, H., Yakovyna, V., Mayr, H. C., Kharchenko, V. S., Peschanenko, V. S., Shyshkina, M., Nikitchenko, M. S., and Spivakovsky, A., editors, *Proceedings of the 13th International Conference on ICT in Education, Research and Industrial Applications. Integration, Harmonization and Knowledge Transfer, ICTERI 2017, Kyiv, Ukraine, May 15-18, 2017*, volume 1844 of *CEUR Workshop Proceedings*, pages 372–379. CEUR-WS.org. <https://ceur-ws.org/Vol-1844/10000372.pdf>.
- Paulsen, K. (2011). 12 - Archives, Backups, and Linear Tape. In Paulsen, K., editor, *Moving Media Storage Technologies*, pages 327–377. Focal Press, Boston. <https://doi.org/10.1016/B978-0-240-81448-3.00012-5>.
- Rao, U. H. and Nayak, U. (2014). Data Backups and Cloud Computing. In *The InfoSec Handbook: An Introduction to Information Security*, pages 263–288. Apress, Berkeley, CA. <https://doi.org/10.1007/978-1-4302-6383-8.13>.
- Reddy, P. V. V. and Shyamala, K. (2016). New scoring formula to rank hypervisors’ performance complementing with statistical analysis using DOE. *Future Generation Computer Systems*, 61:54–65. <https://doi.org/10.1016/j.future.2016.02.012>.
- Rusyn, B., Pohreliuk, L., Vysotska, V., and Osypov, M. (2019). Method of data dedublication and distribution in cloud warehouses during data backup. *Information systems and networks*, 6:1–12. <https://doi.org/10.23939/sisn2019.02.001>.
- Shyshkina, M. (2016). The Hybrid Cloud-based Service Model of Learning Resources Access and its Evaluation. In Ermolayev, V., Spivakovsky, A., Nikitchenko, M. S., Ginige, A., Mayr, H. C., Plexousakis, D., Zholtkevych, G., Burov, O., Kharchenko, V. S., and Kobets, V., editors, *Proceedings of the 12th International Conference on ICT in Education, Research and Industrial Applications. Integration, Harmonization and Knowledge Transfer, Kyiv, Ukraine, June 21-24, 2016*, volume 1614 of *CEUR Workshop Proceedings*, pages 241–256. CEUR-WS.org.
- Sneddon, J. (2017). Mount Your Google Drive on Linux with google-drive-ocamlfuse. <https://www.omgubuntu.co.uk/2017/04/mount-google-drive-ocamlfuse-linux>.
- Spirin, O., Oleksiuk, V., Balyk, N., Lytvynova, S., and Sydorenko, S. (2019). The blended Methodology of Learning Computer Networks: Cloud-based Approach. In Ermolayev, V., Mallet, F., Yakovyna, V., Kharchenko, V. S., Kobets, V., Kornilowicz, A., Kravtsov, H., Nikitchenko, M. S., Semerikov, S., and Spivakovsky, A., editors, *Proceedings of the 15th International Conference on ICT in Education, Research and Industrial Applications. Integration, Harmonization and Knowledge Transfer. Volume II: Workshops, Kherson, Ukraine, June 12-15, 2019*, volume 2393 of *CEUR Workshop Proceedings*, pages 68–80. CEUR-WS.org. https://ceur-ws.org/Vol-2393/paper_231.pdf.
- Spirin, O., Oleksiuk, V., Oleksiuk, O., and Sydorenko, S. (2018). The Group Methodology of Using Cloud Technologies in the Training of Future Computer Science Teachers. In Ermolayev, V., Suárez-Figueroa, M. C., Yakovyna, V., Kharchenko, V. S., Kobets, V., Kravtsov, H., Peschanenko, V. S., Prytula, Y., Nikitchenko, M. S., and Spivakovsky, A., editors, *Proceedings of the 14th International Conference on ICT in Education, Research and Industrial Applications. Integration, Harmonization and Knowledge Transfer. Volume II: Workshops, Kyiv, Ukraine, May 14-17, 2018*, volume 2104 of *CEUR Workshop Proceedings*, pages 294–304. CEUR-WS.org. http://ceur-ws.org/Vol-2104/paper_154.pdf.
- Spirin, O. M., Vakaliuk, T. A., Ievdokymov, V. V., and Sydorenko, S. I. (2022). Criteria for selecting a cloud-based learning management system for a higher education institution. *Information Technologies and Learning Tools*, 89(3):105–120. <https://doi.org/10.33407/itlt.v89i3.4958>.
- Tian, J., Wang, Z., and Li, Z. (2020). Low-cost data partitioning and encrypted backup scheme for defending against co-resident attacks. *EURASIP Journal on Information Security*, 2020(1):7. <https://doi.org/10.1186/s13635-020-00110-1>.
- Vakaliuk, T. (2021). Structural model of a cloud-based learning environment for bachelors in software engineering. *Educational Technology Quarterly*, 2021(2):257–273. <https://doi.org/10.55056/etq.17>.
- Vakaliuk, T., Spirin, O., and Kontsedailo, V. (2021). Formation of digital competence of CS bachelors in the use of cloud-based learning environments. *Educational Technology Quarterly*, 2021(3):388–401. <https://doi.org/10.55056/etq.26>.
- Vorozhbyt, A. V. (2017). Creation of multimedia content of the cloud-based learning environment in technical lyceum. *CTE Workshop Proceedings*, 4:59–63. <https://doi.org/10.55056/cte.325>.
- Wang, B., Wang, C., Song, Y., Cao, J., Cui, X., and Zhang, L. (2020). A survey and taxonomy on workload scheduling and resource provisioning in hybrid clouds. *Cluster Computing*, 28:2809–2834. <https://doi.org/10.1007/s10586-020-03048-8>.
- Yallop, J., Sheets, D., and Madhavapeddy, A. (2018). A modular foreign function interface. *Science of Computer Programming*, 164:82–97. <https://doi.org/10.1016/j.scico.2017.04.002>.

Comparison of Multiple Cloud Frameworks

Gregor von Laszewski, Javier Diaz, Fugang Wang, Geoffrey C. Fox

Pervasive Technology Institute, Indiana University

Bloomington, IN 47408, U.S.A.

e-mail: {laszewski, javier.diazmontes, kevinwangfg, gcfexchange}@gmail.com

Abstract—Today, many cloud Infrastructure as a Service (IaaS) frameworks exist. Users, developers, and administrators have to make a decision about which environment is best suited for them. Unfortunately, the comparison of such frameworks is difficult because either users do not have access to all of them or they are comparing the performance of such systems on different resources, which make it difficult to obtain objective comparisons. Hence, the community benefits from the availability of a testbed on which comparisons between the IaaS frameworks can be conducted. FutureGrid aims to offer a number of IaaS including Nimbus, Eucalyptus, OpenStack, and OpenNebula. One of the important features that FutureGrid provides is not only the ability to compare between IaaS frameworks, but also to compare them in regards to bare-metal and traditional high-performance computing services. In this paper, we outline some of our initial findings by providing such a testbed. As one of our conclusions, we also present our work on making access to the various infrastructures on FutureGrid easier.

Cloud, Grid; Nimbus; Eucalyptus; OpenStack; OpenNebula; RAIN; FutureGrid

I. INTRODUCTION

Cloud computing has become an important tool to deliver infrastructure as a service (IaaS) to users that require a great deal of customization and management of their own software stacks. In addition, we observe that users demand further abstraction and expect platforms as a service (PaaS) to be readily available to conduct higher-level development efforts. Together IaaS and PaaS can provide potent solutions not only to business users, but also to the educational and scientific computing communities. We observe that in the scientific community we can distinguish a number of typical user categories based on their usage in regards to scale, virtual machines used, computational tasks, and the number of users served by the services. In particular we are interested in the following use cases.

First, we distinguish users that demand very few images but want to run them for a long period of time with guarantees on high-availability. Such environments are targeted to support what is today termed the “long tail of science”. In this case, many millions of scientific users with moderate computing needs benefit from the delivery of less compute intense services.

Second, we distinguish scientists requiring a large number of resources to conduct actual calculations and analyses of data to be exposed to their community. This is the traditional high-performance computing use case.

Third, we identified a class of users with requirements in-between the two previous cases. These users have modest but still significant demand of compute resources. Hence our use cases motivate three classes of infrastructures:

1. Traditional high performance computing (HPC) services offered as part of traditional HPC centers.
2. Hosting services for production such as services that cater to a community, including gateways, Web servers and others. Such services may interface with services that run on traditional HPC resources.
3. Access to moderate compute resources to support many moderate calculations to its many users that demand it.

Within this paper we focus on the third class of users. We will identify the rationale for some of the choices that we offer in FutureGrid and identify how to simplify access to an environment that provides so many choices.

The paper is structured as follows. In Section II, we present an overview of FutureGrid and explain why FutureGrid provides the current services it offers. We also outline some future directions motivated by simple usage patterns observed in FutureGrid. One of the main objectives of this paper is to contrast the different IaaS frameworks we offer and project first results of our qualitative comparison described in Sections III and IV, respectively. Based on our qualitative analysis we strive to provide answers to the following questions:

1. *Which IaaS framework is most suited for me?*
2. *How can I compare these frameworks not just between each other, but also to bare-metal?*

The latter is of special interest as at this time many of the Cloud frameworks are still under heavy development and pathways to utilize multiple of them are of current interest.

In Section V, we impart our thoughts on providing PaaS offerings attractive for our user communities. Next, in Section VI, we discuss what implications this multitude of service offerings has for the user community. In Section VII we introduce rain and in Section VIII we detail the scalability tests performed in different FG infrastructures. Finally, Section IX collects the conclusions of this work.

II. FUTUREGRID

The FutureGrid project is sponsored by NSF and includes partners from Indiana University, University of Chicago, University of Florida, San Diego Supercomputing Center, Texas Advanced Computing Center, University of Virginia, University of Tennessee, University of Southern California, Dresden, Purdue University, and Grid 5000.

Among its sites Futuregrid has a set of distributed resources totaling about 5000 compute cores. Resources include a variety of different platforms allowing users to access heterogeneous distributed computing, network, and storage resources. This variety of resources and services allow interesting interoperability and scalability experiments. Additionally, FG provides a fertile environment to explore a variety of IaaS and PaaS frameworks.

Early on we were faced with two elementary questions: (1) Does a user need access to more than one IaaS framework, and (2) if so, which frameworks should we offer? To answer these questions, we analyzed data gathered from the user community of FutureGrid as part of the project registration process. Each project requestor had to provide us feedback on the technologies that were relevant for the execution of their projects. The result of this information is depicted in Figure 1. Figure 2 shows the demographic of the projects we host on FutureGrid classified by scientific disciplines and goals. We observe the following:

1. Nimbus and Eucalyptus were requested the most. This is not surprising, as we made most advertisement for these systems and recommended them for educational class projects on FG. Originally we had more requests for Eucalyptus but scalability issues of the Eucalyptus install on FutureGrid resulted in a recent increase in Nimbus requests.
2. High Performance Computing (HPC) services are requested as third highest category. This is motivated by our affiliation with traditional HPC communities as well as the strong ties to XSEDE.
3. Hadoop and map/reduce were requested by about 36.78% of the projects. This number is higher than the one reported in Figure 1, as we combined the values for Hadoop and MapReduce while only considering unique entries.
4. We saw recently an increase in requests for OpenStack. It has just become one of the preferred open source solutions for cloud computing within a large number of companies, but also within the research community.
5. We have seen an increased demand for the support of OpenNebula. It is quite popular as part of the European Cloud efforts and has gained substantial backing also by US projects such as Clemson University and Fermi Laboratory as part of their cloud strategies.
6. Not surprisingly the largest contingent of our users is the group of technology experts.

Based on this analysis we spent our effort to enable such services within FutureGrid. As a result we are currently providing the following partitioning between services as listed in Figure 3. However, the number of nodes associated between these services can be changed by request.

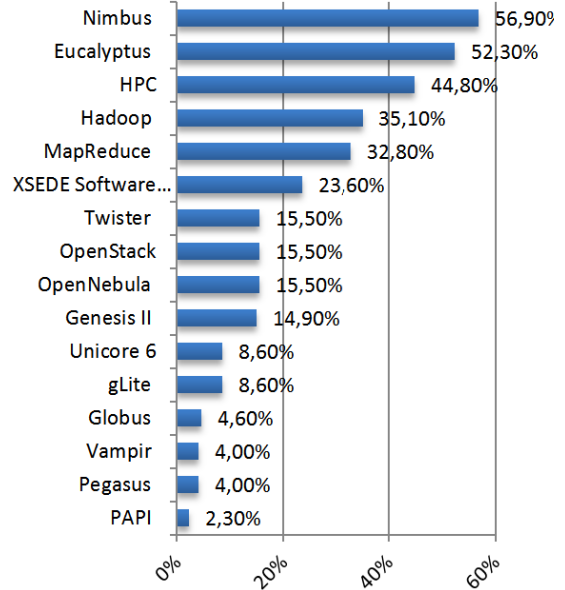


Figure 1: Technology choices made as part of the project application process in FutureGrid. Note that multiple entries could be selected so the total will be more than 100%.

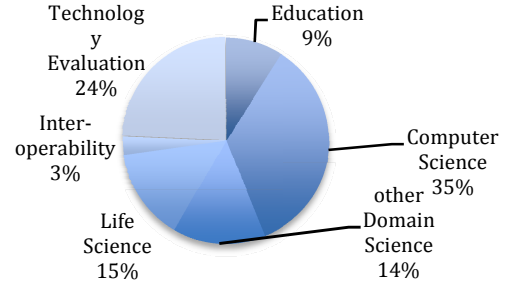


Figure 2: The distribution of the scientific areas that we identified while reviewing the project requests. (Note that 20 project have yet to be integrated into this data).

OpenNebula does not appear on this chart because we have not made it officially accessible. Nevertheless, we have conducted scalability experiments (see Section IV) and monitored usage patterns that could motivate a possible shift in our current deployment strategy. For this decision we are taking into account also other factors, such as what our users do on the portal. An interesting observation we made is that our OpenNebula tutorial that we host on the FG portal is one of our most popular tutorials (see Figure 3) although we do not offer OpenNebula on FutureGrid at this time. This prompts us to identify if we should also offer OpenNebula.

At present, we are integrating mechanisms to float resources between the IaaS frameworks. Thus, today we could decide to assign some resources to Nimbus, while tomorrow the same resources could be assigned to another IaaS framework. Hence, the different infrastructures of our testbed can be resized on-demand to support scalability experiments or scientific problems with high resource requirements.

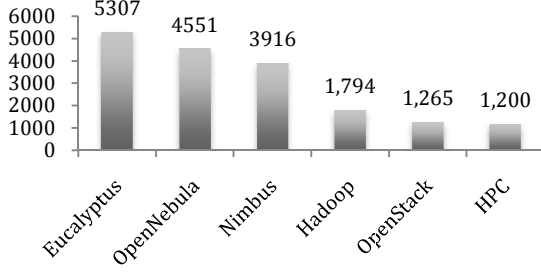


Figure 3: Number of hits on our tutorial pages

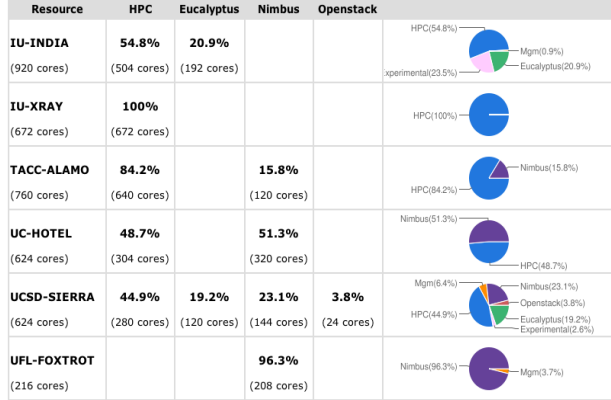


Figure 4: Typical partitioning of FG compute resources by IaaS framework

III. OVERVIEW OF CLOUD IAAS FRAMEWORKS

One fundamental concept in cloud computing is based on providing Infrastructure as a Service (IaaS) to deliver resources to customers and users instead of purchasing and maintaining compute, storage, and network. Typically, this is achieved by offering virtual machines to the users. In order to establish such a service, a number of frameworks are available including Eucalyptus, Nimbus, OpenNebula, OpenStack (see Figure 4). Next, we provide a short discussion about these frameworks and outline some major qualitative differences between them.

A. Nimbus

The Nimbus project [1] is working on two products that they term “Nimbus Infrastructure” and “Nimbus Platform”.

Nimbus Infrastructure: The Nimbus project defines the Nimbus Infrastructure to be “an open source EC2/S3-compatible Infrastructure-as-a-Service implementation specifically targeting features of interest to the scientific community such as support for proxy credentials, batch schedulers, best-effort allocations and others.” To support this mission, Nimbus is providing its own implementation of a storage cloud compatible with S3 compatible and enhanced by quota management, EC2 compatible cloud services, and a convenient cloud client which uses internally WSRE.

Nimbus Platform: The Nimbus platform is targeting to provide additional tools to simplify the management of the infrastructure services and to facilitate the integration with other existing clouds (OpenStack and Amazon). Currently,

this includes the following tools a) cloudinit.d coordinates launching, controlling, and monitoring cloud applications, b) a context broker service that coordinates large virtual cluster launches automatically and repeatedly [1, 2]

B. OpenNebula

OpenNebula [3, 4] is an open source IaaS toolkit. Its design is flexible and modular to allow integration with different storage and network infrastructure configurations, and hypervisor technologies [5]. It can deal with changing resource needs, resource additions, live migration, snapshotting, and failure of physical resources [3, 6]. Furthermore, OpenNebula supports cloud federation to interface with external clouds, which provides scalability, isolation, and multiple-site support. This lets organizations supplement the local infrastructure with computing capacity from public clouds to meet peak demands, or implement high availability strategies. Thus, a single access point and centralized management system can be used to control multiple deployments of OpenNebula.

OpenNebula supports different access interfaces including REST-based interfaces, OGF OCCI service interfaces, and the emerging cloud API standard, as well as the de-facto AWS EC2 API standard.

The authorization is based on passwords, ssh rsa keypairs, X.509 certificates or LDAP. This framework also integrates fine-grained role-based ACLs that as part of its authorization capabilities.

The authorization is based on passwords, ssh rsa keypairs, X.509 certificates or LDAP. This framework also supports fine-grained ACLs to manage users with multiple roles as part of its authorization capabilities.

The storage subsystem supports any backend configuration, from non-shared file systems with image transferring via SSH to shared file systems (NFS, GlusterFS, Lustre) or LVM with CoW (copy-on-write), and any storage server, from using commodity hardware to enterprise-grade solutions.

C. OpenStack

OpenStack [7] is a collection of open source components to deliver public and private clouds. These components currently include OpenStack Compute (called *Nova*), OpenStack Object Storage (called *Swift*), and OpenStack Image Service (called *Glance*). OpenStack is a new effort and has received considerable momentum due to its openness and the support of companies.

Nova is designed to provision and manage large networks of virtual machines, creating a redundant and scalable cloud computing platform. Swift is used to create a redundant, scalable object storage using clusters of standardized servers in order to store petabytes of accessible data. It is not a file system or real-time data storage system, but rather a long-term storage system for more permanent or static data. Glance provides discovery, registration, and delivery services for virtual disk images.

D. Eucalyptus

Eucalyptus [8] promises the creation of on-premise private clouds, with no requirements for retooling the organization's existing IT infrastructure or need to introduce specialized hardware. Eucalyptus implements an IaaS (Infrastructure as a Service) private cloud that is accessible via an API compatible with Amazon EC2 and Amazon S3. It has five high-level components: Cloud Controller (CLC) that manages the virtualized resources; Cluster Controller (CC) controls the execution of VMs; Walrus is the storage system, Storage Controller (SC) provides block-level network storage including support for Amazon Elastic Block Storage (EBS) semantics; and Node Controller (NC) is installed in each compute node to control VM activities, including the execution, inspection, and termination of VM instances.

IV. QUALITATIVE FEATURE COMPARISON OF THE IAAS FRAMEWORKS

All these IaaS frameworks have been designed to allow users to create and manage their own virtual infrastructures. However, these frameworks have differences that need to be considered when choosing a framework. Some qualitative features to consider as part of the selection are summarized in Table 1 and enhance findings from others [9].

Software deployment. An important feature is the ease and frequency of the software deployments. From our own experience the easiest to deploy is OpenNebula because we only have to install a single service in the frontend for a basic configuration while no OpenNebula software is installed in the compute nodes. Nimbus is also relatively easy to install, as only two services have to be configured in the frontend plus the software installation in each compute node. On the other hand, the deployment of Eucalyptus and OpenStack is more difficult due to the number of different components to configure and the different configuration possibilities that they provide. The learning curve for OpenStack is still steep, as the documentation needs some improvements.

In addition to a single install we also have to consider update and new release frequencies. From the release notes and announcements of the framework we observe that major updates happen on a four or six months schedule, with many release candidates and upgrades that fix intermediate issues. Furthermore, we observed that the installation deployment depends on scalability requirements. As such, an OpenStack deployment of four nodes may look quite different from one that utilizes 60 nodes. Hence, it is important that integration with configuration management toolkits exist in order to minimize the effort to repeatedly adapt to configuration requirements for a production system. Tools such as chef and puppet provide a considerable value add in this regards. They serve as a repeatable "template" to install the services in case version dependent performance comparisons or feature comparisons are conducted by the users.

Interfaces. Since Amazon EC2 is a de-facto standard, all of them support the basic functionality of this interface, namely image upload and registration, instantiate VMs, as well as describe and terminate operations. However, the OpenStack project noticed disadvantages due to features that

EC2 is not exposing. Thus, OpenStack is considering to provide interfaces that diverge from the original EC2.

Storage. Storage is very important in cloud because we have to manage many images and they must be available for users anytime. Therefore, most of the IaaS frameworks decided to provide cloud storage. In the case of Nimbus, it is called Cumulus and it is based on the POSIX filesystem. Cumulus also provides a plugin supporting various storage systems including PVFS, GFS, and HDFS (under a FUSE module). Cumulus uses http/s as a communication protocol.

OpenStack and Eucalyptus provide more sophisticated storage systems. In OpenStack it is called Swift, and in Eucalyptus it is called Walrus. Both of them are designed to provide fault tolerance and scalability. OpenStack can store the images in either the POSIX filesystem or Swift. In the first case, images are transferred using ssh while in the second one are transferred using http/s. Finally, OpenNebula does not provide a cloud storage product, but its internal storage system can be configured in different ways. Thus, we can have a shared filesystem between frontend and compute nodes; we can transfer the images using ssh; or we can use LVM with CoW to copy the images to the compute nodes.

Networking. The network is managed differently for each IaaS framework while providing various options in each of them:

- Eucalyptus offers four different networking modes: *managed*, *managed-noLAN*, *system*, and *static* [10]. In the two first modes, Eucalyptus manages the network of the VMs. The modes differ in the network isolation provided by vLAN. In the *system* mode, Eucalyptus assumes that IPs are obtained by an external DHCP server. In the static mode, Eucalyptus manages VM IP address assignment by maintaining its own DHCP server with one static entry per VM.
- Nimbus assigns IPs using a DHCP server that can be configured in two ways: centralized and local. In the first case, a DHCP service is used that one configures with Nimbus-specific MAC to IP mappings. In the second case, a DHCP server is installed on every compute node and automatically configured with the appropriate addresses just before a VM boots.
- OpenStack supports two modes of managing networks for virtual machines: flat networking and vLAN based networking. vLAN based networking requires a vLAN capable managed switch that you can use to setup vLANs for your systems. Flat Networking uses Linux Ethernet bridging to connect multiple compute hosts together.
- The OpenNebula network contains the following options: host-managed vLANs where the network access is restricted through vLAN tagging, which also requires support from the hardware switches; Ebttables to restrict the network access through Ebttables rules; and Open vSwitch to restrict network access with Open vSwitch Virtual Switch.

Hypervisors. All of the IaaS frameworks support KVM and XEN, which are the most popular open source hypervisors. OpenNebula and Openstack also support VMWare. Eucalyptus only supports VMWare in its

Table 1: Feature comparison of IaaS frameworks. ✓ indicates a positive evaluation. The more checkmarks the better.

	OpenStack	Eucalyptus 2.0	Nimbus	OpenNebula
Interfaces	EC2 and S3, Rest Interface. Working on OCCI ✓✓	EC2 and S3, Rest Interface. Working on OCCI ✓✓	EC2 and S3, Rest Interface ✓	Native XML/RPC, EC2 and S3, OCCI, Rest Interface ✓✓✓
Hypervisor	KVM, XEN, VMware Vsphere, LXC, UML and MS HyperV ✓✓✓	KVM and XEN. VMWare in the enterprise edition. ✓✓	KVM and XEN ✓	KVM, XEN and VMWare ✓✓
Networking	- Two modes: (a) Flat networking (b) VLAN networking - Creates Bridges automatically - Uses IP forwarding for public IP - VMs only have private IPs ✓✓✓	- Four modes: (a) managed; (b) managed-novLAN; (c) system; and (d) static - In (a) & (b) bridges are created automatically - IP forwarding for public IP - VMs only have private IPs ✓✓✓	- IP assigned using a DHCP server that can be configured in two ways. - Bridges must exists in the compute nodes ✓✓	- Networks can be defined to support Etable, Open vSwitch and 802.1Q tagging - Bridges must exists in the compute nodes - IP are setup inside VM ✓✓✓
Software deployment	- Software is composed by component that can be placed in different machines. - Compute nodes need to install OpenStack software ✓	- Software is composed by component that can be placed in different machines. - Compute nodes need to install OpenStack software ✓	Software is installed in frontend and compute nodes ✓✓	Software is installed in frontend ✓✓✓
DevOps deployment	Chef, Crowbar, Puppet ✓✓✓	Chef*, Puppet* ✓ (*according to vendor)	no	Chef, Puppet ✓✓
Storage (Image Transference)	- Swift (http/s) - Unix filesystem (ssh) ✓	Walrus (http/s) ✓	Cumulus (http/https) ✓	Unix Filesystem (ssh, shared filesystem or LVM with CoW) ✓
Authentication	X509 credentials, LDAP ✓✓✓	X509 credentials ✓	X509 credentials, Grids ✓✓	X509 credential, ssh rsa keypair, password, LDAP ✓✓✓
Avg. Release Frequency	<4month	>4 month ✓	<4 month	>6 month ✓
License	OpenSource - Apache ✓	OpenSource ≠ Commercial	OpenSource Apache ✓	OpenSource Apache ✓

commercial version. Nimbus does not support VMWare. Additionally, OpenStack also supports LXC, UML and Microsoft's HyperV. This makes OpenStack a quite attractive choice for experimenting with different hypervisors environments.

Authentication. All of the IaaS frameworks support X.509 credentials as authentication method for users. OpenStack and OpenNebula also support authentication via LDAP, although it is quite basic. OpenNebula also support ssh rsa keypair and password authentication. OpenStack focuses recently on the development of a keystore. Nimbus allows interfacing with existing Grid infrastructure authentication frameworks.

V. OVERVIEW OF SELECTED PAAS FRAMEWORKS

A. Platform as a Service

Platform as a Service (PaaS) offers higher-level services to the users that focus on the delivery of a “platform” to develop and reuse advanced services.. Hence, developers can build applications without installing any tools on their computer and deploy those applications without worrying about system administration tasks. On FutureGrid we provide Hadoop and Twister to our users as they provide popular choices in the community. Twister is a product developed at Indiana University (IU) and used as part of

courses taught within the university. This also explains why it is so frequently requested as part of the project registration.

VI. SUPPORTED IAAS AND PAAS FRAMEWORKS IN FUTUREGRID

As already outlined in Section II FutureGrid provides at this time Nimbus, OpenStack, and Eucalyptus on various resources. However, We also experimented with an OpenStack installation with great success. At this time Nimbus is our preferred IaaS framework due to its easy install, the stability, and the support that is provided while including the authors of the Nimbus project as funded partners into the FutureGrid Project. This has a positive impact in support questions, but also in the development of features motivated by FutureGrid. As part of our PaaS offerings, we provide various ways of running Hadoop on FG. This is achieved by either using Hadoop as part of the virtualized environment or exposing it through the queuing system via myHadoop [11]. Twister [12] is contributed through community efforts by IU.

A. Unicore and Genesis II

Additionally, services such as Unicore and Genesis II are available as part of the HPC services.

B. Globus

Originally we expected that the demand for Globus [13] would be higher than what we currently see. However, this is explainable due to the following factors: a) Globus demand has been shrinking because many new projects investigate cloud technologies instead of using Grid toolkits; and b) Grid services have been offered in stable form as part of OSG and XSEDE and thus the demand within a testbed is small as sufficient resources are available to experiment with such systems in production environments. Projects that requested Globus include the SAGA team to investigate interoperability, the EMI project, and the Pegasus Project. Nevertheless, only the later has actively requested a real permanent Globus infrastructure on bare metal resources. The other projects have indicated that their needs can be fulfilled by *rain*ing Globus onto a server based on images that are managed by the projects themselves. Furthermore, we observe that the Globus project has moved over the last year to deliver package based installs that make such deployments easy. We had only one additional user that requested not the installation of Globus, but the installation of GridFTP to more easily move files along. This request could be satisfied while using the CoG Kit [14] and abstracting the need for moving data away from a protocol dependency. Interoperability projects would be entirely served by a virtual Globus installation on images via *rain* (see Section VII). In many of these cases even bare-metal access is not needed. In fact such use cases benefit from the ability to start up many Globus services to as part of the development of advanced workflows that are researched by the community.

At this time we are not supporting any other PaaS such as messaging queues or hosted databases. However, we intend to adapt our strategies based on evaluating user requests with available support capabilities.

VII. RAINING

Due to the variety of services and limited resources provided in FG, it is necessary to enable a mechanism to provision needed services onto resources. This includes also the assignment of resources to different IaaS or PaaS frameworks. We have developed, as first step to address this challenge, a sophisticated image management toolkit that allows us to not only provision virtual machines, but also provision directly onto bare-metal. Thus, we use the term *rain*ing to indicate that we can place arbitrary software stack onto a resource. The toolkit to do so is called *rain*.

Rain makes it possible to compare the benefits of IaaS, PaaS performance issues, as well as evaluating which applications can benefit from such environments and how they must be efficiently configured. As part of this process, we allow the generation of abstract images and universal image registration with the various infrastructures including Nimbus, Eucalyptus, OpenNebula, OpenStack, but also bare-metal via the HPC services. This complex process is described in more detail in [15]. It is one of the unique features about FutureGrid to provide an essential component to make comparisons between the different infrastructures

more easily possible. Our toolkit *rain* is tasked with simplifying the full life cycle of the image management process in FG. It involves the process of creating, customizing, storing, sharing and deploying images for different FG environments. One important feature in our image management design is that we are not simply storing an image but rather focus on the way an image is created through templating. Thus it will be possible at any time to regenerate an image based on the template that is used to install the software stack onto a bare operating system. In this way, we can optimize the use of the storage resources.

Now that we have an elementary design of managing images, we can dynamically provision them to create bare-metal and virtualized environments while *rain*ing them onto our resources. Hence, *rain* will offer four main features:

- Create customized environments on demand.
- Compare different infrastructures.
- Move resources from one infrastructure to another by changing the image they are running plus doing needed changes in the framework.
- Ease the system administrator burden for creating deployable images.

This basic functionality has been tested as part of a scalability experiment motivated by our use case introduced in the introduction. Here, a user likes to start many virtual machines and compare the performance of this task between the different frameworks.

VIII. INFRASTRUCTURE SCALABILITY STUDY

We have performed several tests to study the scalability of the infrastructures installed in our cluster at Indiana University called India. The India cluster is composed by Intel Xeon X5570 with 24GB of memory, a single drive of 500GB with 7200RPM 3Gb/s and an interconnection network of 1Gbps Ethernet.

The idea of these tests is to provision as many physical machines (PM) or virtual machines (VM) at the same time as possible. Obviously this is an important test that is of especial interest to our traditional HPC community that needs many compute resources to operate in parallel. Tests succeed if all the machines have ssh access and we can successfully connect and run commands on all the machines. We measure the time that takes since the request is first issued until we have access to all the machines.

For that purpose we have used our image generator and registration tools and created a CentOS 5 image. We registered it in the different infrastructures: HPC, OpenStack, Eucalyptus and OpenNebula. This process gives us an identical image template. Therefore, the only difference in the image is the version of the 2.6 kernel/ramdisk used. In HPC we use the ramdisk modified by xCAT, in Eucalyptus we use a XEN kernel and in OpenStack or OpenNebula we use a generic kernel with KVM support. The total size of the image without compression is 1.5 GB. In the case of netboot it is compressed to about 300MB. Figure 5 shows the results of the performed tests. In the following sections we describe the software used in each infrastructure, the results obtained and the problems we found.

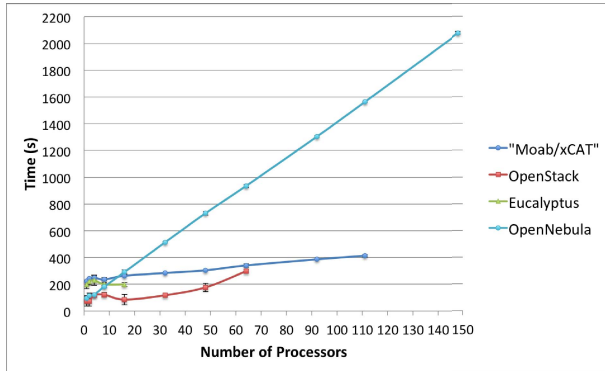


Figure 5: Scalability Experiment of IaaS Frameworks on FutureGrid

A. HPC (Moab/xCAT)

We used Moab 6.0.3 and xCAT 2.6.9 to dynamically provision images to the compute nodes. We had a total of 111 nodes provisioned with the same image. During the tests we did not have any problems and observed very good linear scalability. This behavior is based on the good behavior of execution the provisioning step in parallel. A small bottleneck in our experiments was introduced at the time of retrieving the image from the server. Since the image is compressed and relatively small, the scalability of this setup is preserved. However, provisioning even a single image takes more time in comparison to its virtualized counterparts. This easily explained by the fact that the process of rebooting the physically machine must be called which is naturally not needed in case of using virtualized machines. Here the process of rebooting takes place in memory and is less time consuming.

B. OpenStack

In our experiments, we have used the Cactus version of OpenStack. In order to make an efficient image provisioning, OpenStack caches images in the computes nodes. Thus, it does not need to transfer the image over the network every time it is requested and the instantiation of a VM is faster.

Due to scalability issues by provisioning higher number of VMs, we modified our strategy to submit VM requests in batches of 10 VMs at a time (maximum). This means that in the case of provisioning 32 VMs, we have requested 3 times 10 VMs and once 2 VMs. This is a verified bug in OpenStack and is going to be solved in future releases. Or through the method we chose for Cactus [16].

Furthermore, we observed that if the image is not cached in the compute nodes, the scalability of this framework is very limited. In fact, we started to have problems trying to boot 16 VMs and observed a failure rate of about 50% for our test. We observed that VMs got *stuck* in the launching status (this is the status where the image is transferred to the compute node).

On the other hand, once we had the image cached in most of the compute nodes, we were able to boot up to 64 VMs simultaneously. However, placing the images into the cache was not an easy task, as we had to deal with a 50% failure

rate. Besides observing that the image got stuck in the launching status, we also observed that other images were running, but we could not access them via ssh. The later problem is mainly related to the fact that OpenStack does not inject the public network configuration inside the VM. Alternatively, it creates bridges and conducts IP forwarding to direct all the traffic toward the private IP. At times we observed that the bridges, the iptables entries or both were not created properly. In particular, the iptables issue is important because we observed that sometimes OpenStack gets confused and duplicate entries in the iptables using old information that was not properly cleaned up. This erroneous information makes the VMs inaccessible. We observed that this problem is persistent and one needs to manually modify the database to remove the wrong entries or restart OpenStack.

Finally, another known problem of OpenStack Cactus is that it is not able to automatically assign public IPs to the VMs, so we have to look into the pool and assign one to each VM manually. This feature has been added in the Diablo version, though. Our intention is to rerun our experiments also with Diablo once it is available on FutureGrid.

C. Eucalyptus

We use version 2.03 of Eucalyptus, which is the latest open source version. Eucalyptus also caches the images in the compute nodes. Just like in OpenStack, we observed similar issues with Eucalyptus while requesting larger numbers of VMs. Thus, we modified our initial experiment plan and also introduced batched requests (10 at a time) but with a delay of 6 seconds. This is due to an annoying feature of Eucalyptus that prevents users from executing the same command several times in a short period of time.

The tests with this infrastructure were quite disappointing because we could only get 16 VMs running at the same time. Even though, the failure rate was very high. Eucalyptus, like OpenStack, configures the public network with IP forwarding and creates the bridges at running time. Thus, this creates problems, like in the case of OpenStack, we got similar errors like missing bridges and iptables entries. Additionally, we had problems with the automatic assignment of public IPs to the VMs. This means that some VMs did not get public IPs and therefore they were not accessible for users.

D. OpenNebula

We used OpenNebula version 3.0.0. By default OpenNebula does not cache images in the compute nodes. It supports three basic transfer plugins named nfs, ssh and lvm. NFS has a terrible performance because the VMs are reading the image through the network. SSH was the one that we used because is still easy to configure and has a better performance. The last one seems more difficult to configure, but it should provide the best performance, because it is the one selected by the OpenNebula team to perform their experiments [17, 18]. We were able to instantiate 148 VMs at the same time with almost a 100% of success. In fact, we only got one error in the case of 148 VMs. Since our configuration does not use an image cache, it was very slow

and limited by the network, as each image must be copied to each VM. This is done in a sequential process. However, this can be improved by introducing caches as others have showcased [19].

IX. CONCLUSIONS

This paper provides evidence that offering not one but many IaaS frameworks is needed to allow users to answer the question which IaaS framework is right for them. We have to be aware that users requirements may be very different from each other and the answer to this question cannot be universally cast.

We identified an experiment to be executed on FutureGrid to provide us with feedback on a question many of our users have: Which infrastructure framework should I use under the assumption I have to start many instances. Our close ties to the HPC community and researchers utilizing parallel services and programming methodologies naturally motivate this experiment. However, such users tend not to be experts in IaaS frameworks and a tool is needed to allow the comparison of their use cases in the different frameworks. In this paper, we have demonstrated that we achieved a major milestone towards this goal and proven that our efforts can lead to significant comparisons between deployed frameworks. This also includes the ability to identify a set of benchmarks for such infrastructures in order to further identify which framework is best suited for a particular application. Our test case must be part of such a benchmark.

Concretely, we found challenges in our scalability experiments while raining images on them. This was especially evident for Eucalyptus and even OpenStack. As many components are involved in the deployment they are also not that easy to deploy. Tools provided as part of developments such as Chef [20] and Puppet [21] can simplify deployments especially if they have to be done repeatedly or require modifications to improve scalability. We claim that the environment to conduct an OpenStack experiment with just a handful VMs may look quite different from a deployment that uses many hundreds of servers on which Openstack may be hosted. This is also documented nicely as part of the OpenStack Web pages that recommends more complex service hierarchies in case of larger deployments.

On the other hand, we have seen that OpenNebula is very reliable and easy to deploy. Although in our experiments we found it is quite slow due to the lack of caching images. Nevertheless, we think that this problem is easy to solve. In fact, after we finished the tests we also found other community members having the same problem. A plugin to provide caching support is already available, which can significantly improve managing images across many nodes in OpenNebula.

Through the ability of rain it will become easier for us to deploy PaaS on the IaaS offerings, as we will be able to create "templates" that facilitate their installation and potentially their upgrade. Due to this ability it is possible to replicate the environments and introduce reproducible environment.

The answer to the question, which IaaS question should one use, depends on the requirements of the applications and the user community. OpenNebula and Nimbus were easier to install, but we observed that OpenStack and Eucalyptus have considerably worked on these issues in newer versions to be released shortly. Nimbus has turned out to be very reliable, but our OpenStack and Eucalyptus Clouds have been extremely popular resulting in resource starvation. We will address this issue by assigning dynamically resources between the clouds. From our own observations working with applications it is clear that we do need a testbed, such as FutureGrid, to allow users to cast their own evaluations.

ACKNOWLEDGEMENTS

We would like to acknowledge the rest of the FG team for its support in the preparation and execution of these tests. This material is based upon work supported in part by the National Science Foundation under Grant No. 0910812

REFERENCES

- [1] *Nimbus Project Web Page*. <http://www.nimbusproject.org>, May 2012.
- [2] K. Keahey, I. Foster, T. Freeman, and X. Zhang, "Virtual Workspaces: Achieving Quality of Service and Quality of Life in the Grid," *Scientific Programming Journal*, vol. 13, pp. 265-276, 2005.
- [3] *OpenNebula Web Page*. <http://www.opennebula.org>, May 2012.
- [4] I. M. Llorente, R. Moreno-Vozmediano, and R. S. Montero, "Cloud computing for on-demand grid resource provisioning," *Advances in Parallel Computing*, vol. 18, pp. 177-191, 2009.
- [5] R. S. Montero, R. Moreno-Vozmediano, and I. M. Llorente, "An Elasticity Model for High Throughput Computing Clusters," *J. Parallel and Distributed Computing*, 2010.
- [6] B. Sotomayor, R. S. Montero, I. M. Llorente, and I. Foster, "Virtual Infrastructure Management in Private and Hybrid Clouds," *IEEE Internet Computing*, vol. 13, pp. 14-22, Sep.-Oct 2009.
- [7] *OpenStack Web Page*. <http://www.openstack.org>, May 2012.
- [8] *Eucalyptus Web Pages (Open Source)*. <http://open.eucalyptus.com/>, May 2012.
- [9] P. Sempolinski and D. Thain, "A Comparison and Critique of Eucalyptus, OpenNebula and Nimbus," presented at the IEEE 2nd Int. Conf. on Cloud Computing Technology and Science, 2010.
- [10] *Eucalyptus Network Configuration 2.0*. http://open.eucalyptus.com/wiki/EucalyptusNetworkConfiguration_v2.0, May 2012.
- [11] *myHadoop*. <http://sourceforge.net/projects/myhadoop/>, May 2012.
- [12] *Twister*. <http://www.iterativemapreduce.org/>, May 2012.
- [13] *Globus Project*. <http://www.globus.org/>, May 2012.
- [14] G. von Laszewski, I. Foster, J. Gawor, and P. Lane, "A Java Commodity Grid Kit," *Concurrency and Computation: Practice and Experience*, vol. 13, pp. 643-662, 2001.
- [15] J. Diaz, G. v. Laszewski, F. Wang, and G. Fox, "Abstract Image Management and Universal Image Registration for Cloud and HPC Infrastructures," *IEEE Cloud*, 2012.
- [16] *Running 200 VM instances on OpenStack*. <http://blog.griddynamics.com/2011/05/running-200-vm-instances-on-openstack.html>, May 2012.
- [17] U. Schwickerath, "CERN Cloud Computing Infrastructure," presented at the ICS Cloud Computing Conference, 2010.
- [18] *CERN Scaling up to 16000 VMs*. <http://blog.opennebula.org/?p=620>, May 2012.
- [19] *Cache optimization in OpenNebula*. <http://dev.opennebula.org/issues/553>, May 2012.
- [20] *Chef*. <http://www.opscode.com/chef/>, May 2012.
- [21] *Puppet*. <http://puppetlabs.com/>, May 2012.

Infrastructure as a Service (IaaS): A Comparative Performance Analysis of Open-Source Cloud Platforms

Sonia Shahzadi*, Muddesar Iqbal†, Zia Ul Qayyum‡, Tasos Dagiuklas§

*Email: s.shahzadi@swanmesh.com †Email: vc@uog.edu.pk ‡§Email: {m.iqbal,tdagiuklas}@lsbu.ac.uk

* Swan Mesh Networks Ltd

‡Pak-UK Institute of Emerging Technologies for Disaster Management

†§School of Engineering, London South Bank University, UK

Abstract—Recently, cloud computing has been recognized as effective computing model, being adopted by both large corporations as well as SMEs (Small and Medium Enterprises). Due to its on-demand provisioning nature number of cloud based services are growing gradually and rapidly. The availability of open-source cloud computing platforms plays a vital role to enable researchers to go beyond state of the art for development of cloud applications. In this paper, we have provided an overview of cloud computing along with its services. We have presented a comparative performance analysis of different cloud IaaS platforms while covering different aspects of their application and service models. We have also proposed cloud test-bed using open-source cloud platform. A comparative performance analysis of open-source cloud platforms will assist researchers to select an appropriate tool to validate their research and experimentation.

Keywords—IaaS, PaaS, SaaS, CSP, NASA, OpenStack, DevStack, Oracle VM Virtual Box, Rally.

I. INTRODUCTION

Cloud computing is a dynamic field with a lot of flexibility and availability of very economical and reliable on-demand computing resources. The consumer can simply connect anytime from anywhere to access all the resources that are available through the cloud. These resources are managed and utilized in a distributed manner. Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g. networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction [1]. Essential characteristics of cloud computing are: on-demand self-service, broad network access, resource pooling, rapid elasticity and service measurement [1].

The main cloud computing deployment models are public, private, hybrid and community clouds [2]–[8]. By using these models, cloud computing provides different services such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS) and Software as a Service (SaaS) [1]–[4], [7]–[11].

In IaaS, Cloud Service Providers (CSPs) offer the infrastructure as a service where a set of virtual computing resources, e.g. CPU, storage and network components are provided to the consumer [1]. Cloud users can deploy and run their own system and the software applications using these virtual resources [1] with access to the underlying hardware resources. The clients of IaaS have a reliable control on their infrastructures.

Examples of IaaS providers are Drop Box, Amazon EC2 and S3, Google Compute Engine, Rackspace, IBM Smartcloud, GoGrid and Akamai [2], [12], [13].

In PaaS, CSPs offer the platform as a service where customers can develop and deploy their own applications using development environment, application server technology, different supportive libraries and programming languages [1]. PaaS can be helpful where multiple clients use the same development environment. Integration with the databases and web services through some standards is also a feature of PaaS. The testing and the development can easily be automated for programmers in PaaS. The clients have full access and control over the tools and the developmental languages. PaaS provider's examples includes Windows Azure, Google App Engine, Red Hat OpenShift, Engine Yard, Heroku, AppFog, Caspio, MBaaS providers and Aptana Cloud [12].

SaaS is a demand based application that are hosted, maintained and managed by cloud service providers through a central location. Examples of SaaS providers are Zoho, Google Docs and Salesforce.com [9].

In the remainder of this paper, Section II present a comparisons of open-source cloud infrastructure platforms. In Section III, implementation of cloud testbed are described. Performance evaluation of two different OpenStack cloud implementations as a testbed are presented in Section IV. The paper is concluded in Section V.

II. LITERATURE REVIEW

A. Cloud IaaS Frameworks

Today, many cloud frameworks exist that are providing IaaS for both private and public clouds. Several, frameworks are available for this purpose including, Nimbus, Eucalyptus, OpenNebula and OpenStack. A taxonomy is proposed on selected cloud providers and survey is also given in [14]. From our literature analysis, we have identified some well-known cloud platforms that are providing IaaS. A brief overview of these platforms is given below:

1) *Nimbus*: Nimbus provides IaaS and it is divided into two phases that are: nimbus infrastructure and nimbus platform. Nimbus infrastructure is providing IaaS implementation that is compatible for EC2/S3. While Nimbus platform is providing additional tools for simplifying the infrastructure management.

It is mostly considered for scientific cloud computing solutions [15].

2) *Eucalyptus*: Eucalyptus provides IaaS for private cloud. It has following five components [16], [6], [17]:

- *Cloud Controller (CLC)*: It manages the virtual resources.
- *Cluster Controller (CC)*: It controls the execution of VMs.
- *Storage Controller (SC)*: It provides storage system which is called walrus.
- *Elastic Block Storage (EBS)*: It provides Amazon EBS semantics support.
- *Node Controller (NC)*: It controls VM activities including VM termination and VM execution.

3) *OpenNebula*: OpenNebula is an open source IaaS toolkit [18]. It can deal dynamic resource needs, snapshotting and live migration [6]. It supports different access interfaces, including OCCI service interface, REST-based interfaces and emerging cloud API standards.

4) *OpenStack*: OpenStack provides IaaS for private and public clouds. OpenStack is a combination of multiple open-source projects to develop your own cloud infrastructure and it is initiated by Rackspace and NASA in 2010 [19]. OpenStack main characteristics are: scalable, compatible and flexible, and open source [13]. It exploits various open-source projects and manages both compute and storage resources in the cloud [20]. OpenStack has mainly following components: Keystone (Authentication Service), Glance (Image Service), Nova (Compute Service), Neutron (Network Service) Horizon (Dashboard Service), Cinder (Block Storage) and Swift (Object Storage) [21], [16], [22], [23].

5) *CloudStack*: CloudStack originally developed by cloud.com in 2010 under GPL v3 license. In 2011, Citrix purchased Cloud.com and later on, in 2012, its license changed into Apache 2.0 when Citrix donated CloudStack to the Apache Software Foundation [16]. It consists of the following components [3], [16].

- *Pods*: Hardware configuration into a form of clusters and a pod can consist of one or more clusters.
- *Clusters*: A group of multiple identical hosts and specifies for primary storage.
- *Primary Storage*: Used in each cluster for host VM instances.
- *Secondary Storage*: Used for VM images and snapshots storage.
- *Availability Zones*: Used to represent geographical locations which are helpful for VM allocation.

6) *AbiCloud*: AbiCloud provides IaaS and mainly it consists of following three components [3], [4]:

- *AbiCloud Server*: Used for computer management.
- *AbiCloud Web Services (WS)*: Used to manage virtual applications.

- *Virtual System Monitor (VWS)*: Used to monitor virtual infrastructure.

7) *XEN Cloud Platform (XCP)*: XCP is an open-source solution for server virtualization and cloud computing platform includes the XEN hypervisor [24]. It doesn't provide a complete cloud infrastructure solution, but it is helpful for cloud management system [25].

B. Comparison of IaaS Frameworks

In order to get comparison of these platforms, we have derived certain assessment criteria from different cloud features e.g. structure, supported cloud models and programming languages. To perform networking tasks in cloud computing is also a significant part so it's essential to understand which network service is offered in each platform. Another important aspect is to check these platform compatibility with other hypervisors, and CSP's as well as scalability, load balancing and fault-tolerance. In Table I, we have done some features comparison that is derived from literature analysis [3], [4], [6], [15]–[18], [24]–[36]. The comparison provides the knowledge and understanding which makes it much easier for users, developers and the administrator's to choose the best suited framework that is compatible for their environment. Therefore, the presented review covers different characteristics and implementation aspects to help in developing better understanding of these platforms.

III. IMPLEMENTATION OF CLOUD TESTBED

In this section we have selected two of the OpenStack implementations i.e. deployment using DevStack and native 3-tier OpenStack deployment. Many researchers use DevStack to quickly create an OpenStack development environment for experimentation, whereas 3-tier OpenStack implementations are used for production of IaaS platform to implement as a research testbed to perform a performance comparison analysis. Using DevStack to substitute the OpenStack implementation can produce different results using the deployment environment. This comparative performance analysis will enable researchers to understand this difference among both the options available for testbed deployment in terms of their performance in lab environment for experimentation. Following tools are used to develop these testbeds.

- Cloud IaaS Platform: OpenStack and DevStack.
- Host OS: Ubuntu Server 14.04.
- Hypervisor: Oracle VM Virtual Box.

IV. PERFORMANCE EVALUATION

We have used Rally benchmark tool to measure the performance of different cloud services to test and verify the scalability of cloud. Although, DevStack is not applicable for production environment, but if a user want to deploy a mini private cloud to seek certain cloud performance tests, then this work is beneficial for the purpose where user can get an indication on how well these cloud platform services can perform in different environments and use-cases. Furthermore, it will help the researchers to choose the right platform to setup their testbed in order to validate their hypothesis. We have performed test using same scenarios configuration for both testbeds to generate the results.

Scenario 1: In scenario 1, we have run 100 iterations with 8 atomic actions to measure certain keystone services correctness and performance. As, shown in Figure 1, x-axis represents number of iterations while the y-axis represents the total duration times to complete the specific iteration in seconds and maximum time is 64.95 (sec) in DevStack case while 19.21 (sec) in OpenStack. As, this show that OpenStack performance is much better than as compare to DevStack in terms of keystone services.

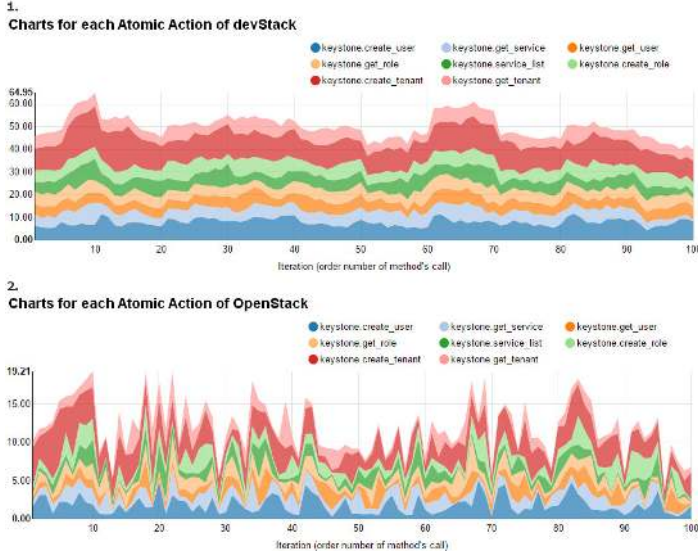


Figure 1. Charts for each Atomic Action in Scenario-1

Scenario 2: In scenario 2, we have run 100 iterations with 2 atomic actions to measure certain glance services correctness and performance. As, shown in Figure 2, x-axis represents number of iterations while the y-axis represents the total duration times to complete the specific iteration in seconds and maximum time is 87.88 (sec) in DevStack case while 44.90 (sec) in OpenStack. As, this scenario also show that OpenStack performance is much better than as compare to DevStack in terms of creating and listing images.

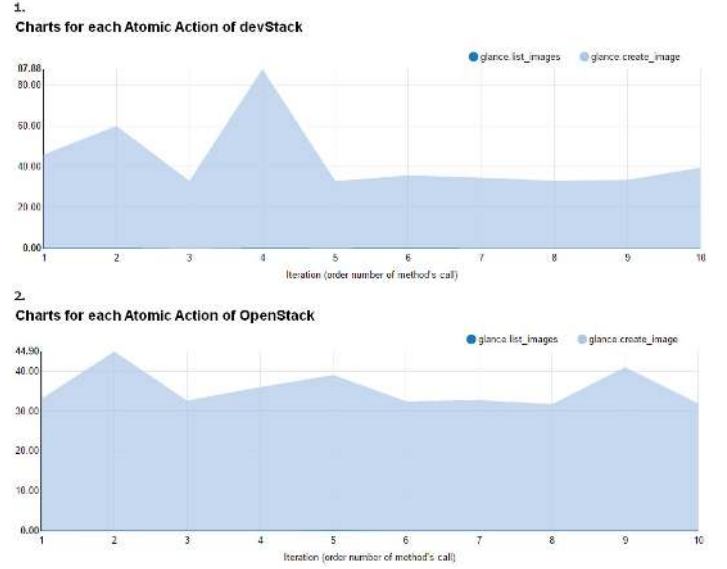


Figure 2. Charts for each Atomic Action in Scenario-2

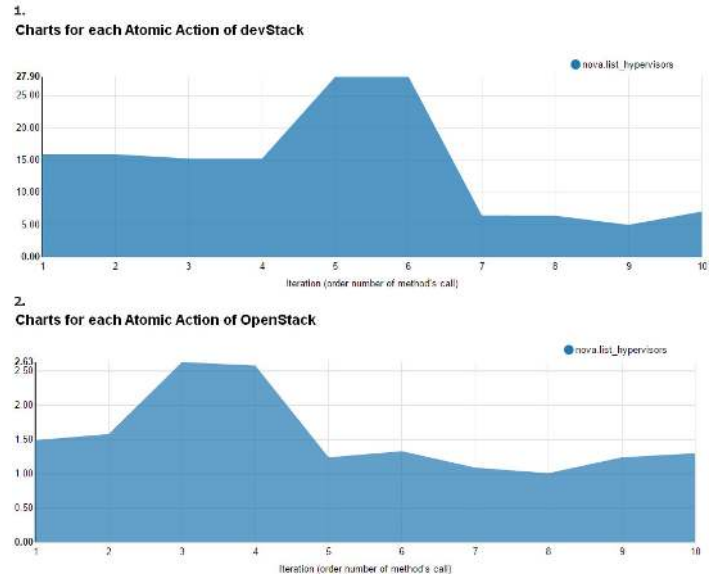


Figure 3. Charts for each Atomic Action in Scenario-3

Scenario 3: In scenario 3, we have run 100 iterations with single atomic actions to measure specific nova service correctness. As, shown in Figure 3, x-axis represents number of iterations while the y-axis represents the total duration times to complete the specific iteration in seconds and maximum time is 27.90 (sec) in DevStack case while 2.63 (sec) in OpenStack. As, this scenario also show that OpenStack performance is much better than as compare to DevStack while listing hypervisors.

Analysis of results After observing these scenarios performance, we conclude that although DevStack installer provides quick deployment of OpenStack and it's also much easier to deploy as compare to manual solution but its performance and response time of cloud services is much slower as compare to native OpenStack approach. As, we have tested multiple scenarios from different OpenStack services (e.g. Keystone, Glance, Nova) but response time in each scenario of DevStack is much lower than manual OpenStack. The results shows a noticeable difference in performance for both the testbed platforms which we believe will help the researchers to select

the most appropriate platform for their testbed deployments.

V. CONCLUSION AND FUTURE WORK

Although many cloud platforms are available for both academic and industrial deployments but selecting a most appropriate platform based on user requirements is major challenge. This paper presents a detailed comparison of these platforms while covering different deployment aspects and characteristics such as architecture, networking, supported platforms, hypervisors and languages. Although, each of these platforms has its own strengths and limitations but the presented comparison will help the researchers to select most appropriate platform as per requirement. In this paper, we have also implemented open-source cloud testbed using both 3-tier OpenStack and DevStack platforms and presented performance comparison for both implementations using different

services scenarios. The results shows that the performance of OpenStack 3-tier implementation is much better as compare to auto deployment method using DevStack. Therefore, the experimentation performed on both the tools using the same hardware and test scenarios may produce different results, which is not acceptable to validate a research hypothese in most of the cases and may raise concerns.

In future work, we will explore the use of light-weight container approaches which is a rapidly growing area of research. We plan to study the performance aspects of using containers and compare it with virtual machines to provide cloud services.

REFERENCES

- [1] P. Mell and T. Grance, "The nist definition of cloud computing," 2011.
- [2] S. N. R. J. S. Vanishree and P. Karthick, "A prototype for private cloud implementation using open-source platform,"
- [3] J. Peng, X. Zhang, Z. Lei, B. Zhang, W. Zhang, and Q. Li, "Comparison of several cloud computing platforms," in *Information Science and Engineering (ISISE), 2009 Second International Symposium on*, pp. 23–27, IEEE, 2009.
- [4] S. Wind, "Open source cloud computing management platforms: Introduction, comparison, and recommendations for implementation," in *Open Systems (ICOS), 2011 IEEE Conference on*, pp. 175–179, IEEE, 2011.
- [5] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, et al., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [6] G. Von Laszewski, J. Diaz, F. Wang, and G. C. Fox, "Comparison of multiple cloud frameworks," in *Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on*, pp. 734–741, IEEE, 2012.
- [7] R. Buyya, S. Pandey, and C. Vecchiola, "Cloudbus toolkit for market-oriented cloud computing," in *IEEE International Conference on Cloud Computing*, pp. 24–44, Springer, 2009.
- [8] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: state-of-the-art and research challenges," *Journal of internet services and applications*, vol. 1, no. 1, pp. 7–18, 2010.
- [9] A. Marinos and G. Briscoe, "Community cloud computing," in *Cloud Computing*, pp. 472–484, Springer, 2009.
- [10] C. Vecchiola, S. Pandey, and R. Buyya, "High-performance cloud computing: A view of scientific applications," in *Pervasive Systems, Algorithms, and Networks (ISPAN), 2009 10th International Symposium on*, pp. 4–16, IEEE, 2009.
- [11] D. Durkee, "Why cloud computing will never be free," *Queue*, vol. 8, no. 4, p. 20, 2010.
- [12] H. BH, L. Suresh, and K. Radhika, "A survey on cloud computing," *INTERNATIONAL JOURNAL OF MATHEMATICS*, vol. 1, no. 11, 2013.
- [13] O. Sefraoui, M. Aissaoui, and M. Eleuldi, "Openstack: toward an open-source solution for cloud computing," *International Journal of Computer Applications*, vol. 55, no. 3, 2012.
- [14] R. Prodan and S. Ostermann, "A survey and taxonomy of infrastructure as a service and web hosting cloud providers," in *Grid Computing, 2009 10th IEEE/ACM International Conference on*, pp. 17–25, IEEE, 2009.
- [15] M. Mahjoub, A. Mdhaflar, R. B. Halima, and M. Jmaiel, "A comparative study of the current cloud computing technologies and offers," in *Network Cloud Computing and Applications (NCCA), 2011 First International Symposium on*, pp. 131–134, IEEE, 2011.
- [16] A. Beloglazov, S. F. Piraghaj, M. Alrokayan, and R. Buyya, "Deploying openstack on centos using the kvm hypervisor and glusterfs distributed file system," *University of Melbourne*, 2012.
- [17] G. Raj, C. Kapoor, and D. Singh, "Comparative cloud deployment and service orchestration process using juju charms,"
- [18] X. Wen, G. Gu, Q. Li, Y. Gao, and X. Zhang, "Comparison of open-source cloud management platforms: Openstack and opennebula," in *Fuzzy Systems and Knowledge Discovery (FSKD), 2012 9th International Conference on*, pp. 2457–2461, IEEE, 2012.
- [19] "OpenStack." <http://www.openstack.org>. Accessed: 2017-01-01.
- [20] A. Corradi, M. Fanelli, and L. Foschini, "Vm consolidation: A real case based on openstack cloud," *Future Generation Computer Systems*, vol. 32, pp. 118–127, 2014.
- [21] F. Wuhib, R. Stadler, and H. Lindgren, "Dynamic resource allocation with management objectives—implementation for an openstack cloud," in *Network and service management (cnsm), 2012 8th international conference and 2012 workshop on systems virtualization management (svm)*, pp. 309–315, IEEE, 2012.
- [22] R. Kumar, "Openstack juno release includes features of nfv, big data," *Nova*, vol. 1, p. 21, 2010.
- [23] T. Yanagawa, "Openstack-based next-generation cloud resource management," *Fujitsu Sci. Tech. J.*, vol. 51, no. 2, pp. 62–65, 2015.
- [24] M. Bist, M. Wariya, and A. Agarwal, "Comparing delta, open stack and xen cloud platforms: A survey on open source iaas," in *Advance Computing Conference (IACC), 2013 IEEE 3rd International*, pp. 96–100, IEEE, 2013.
- [25] T. Cordeiro, D. Damalio, N. Pereira, P. Endo, A. Palhares, G. Gonçalves, D. Sadok, J. Kelner, B. Melander, V. Souza, et al., "Open source cloud computing platforms," in *Grid and Cooperative Computing (GCC), 2010 9th International Conference on*, pp. 366–371, IEEE, 2010.
- [26] Q. Huang, C. Yang, K. Liu, J. Xia, C. Xu, J. Li, Z. Gui, M. Sun, and Z. Li, "Evaluating open-source cloud computing solutions for geosciences," *Computers & Geosciences*, vol. 59, pp. 41–52, 2013.
- [27] N. G. Bachiega, H. P. Martins, R. Spolon, M. A. Cavenaghi, R. S. Lobato, and A. Manacero, "Open source cloud computing: Characteristics and an overview," in *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA)*, p. 237, The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp), 2013.
- [28] H. A. Albaroodi, S. Manickam, and M. F. Aboalmaaly, "The classification and arts of open source cloud computing: A review," *Advances in Information Sciences and Service Sciences*, vol. 5, no. 16, p. 16, 2013.
- [29] F. Meireles and B. Malheiro, "Integrated management of iaas resources," in *Euro-Par 2014: Parallel Processing Workshops*, pp. 73–84, Springer, 2014.
- [30] B. P. Rimal, E. Choi, and I. Lumb, "A taxonomy and survey of cloud computing systems," in *2009 Fifth International Joint Conference on INC, IMS and IDC*, pp. 44–51, Ieee, 2009.
- [31] I. Nwobodo, "A comparison of cloud computing platforms," 2015.
- [32] M. Alam and K. A. Shakil, "Recent developments in cloud based systems: state of art," *arXiv preprint arXiv:1501.01323*, 2015.
- [33] B. Mohammed and M. Kiran, "Analysis of cloud test beds using open-source solutions," in *Future Internet of Things and Cloud (FiCloud), 2015 3rd International Conference on*, pp. 195–203, IEEE, 2015.
- [34] F. Cocozza, G. López, G. Marin, R. Villalón, and F. Arroyo, "Cloud management platform selection: A case study in a university setting," *CLOUD COMPUTING 2015*, p. 92, 2015.
- [35] P. T. Endo, G. E. Gonçalves, J. Kelner, and D. Sadok, "A survey on open-source cloud computing solutions," in *Brazilian Symposium on Computer Networks and Distributed Systems*, pp. 3–16, 2010.
- [36] S. Kumar and R. Goudar, "Cloud computing-research issues, challenges, architecture, platforms and applications: A survey," *International Journal of Future Computer and Communication*, vol. 1, no. 4, p. 356, 2012.
- [37] L. Wang, G. Von Laszewski, A. Younge, X. He, M. Kunze, J. Tao, and C. Fu, "Cloud computing: a perspective study," *New Generation Computing*, vol. 28, no. 2, pp. 137–146, 2010.

Table I: Comparison of IaaS Frameworks

Features	Nimbus	Eucalyptus	Open-Nebula	OpenStack	CloudStack	Abicloud	XEN Cloud Platform
Year	2009	2008	2005	2010	2010	2009	2012
Developed by	University of Chicago	University of California, Santa Barbara, Eucalyptus system company	European Union	Rackspace, NASA	Cloud.com	Abiquo	Citrix Xen Server
Main purpose	Scientific solution	Private Cloud	Private Cloud	Provides Cloud Computing services.	—	Tool for automatic cloud management	Manage cloud computing platform
Architecture /Structure	Centralized	Hierarchical	Centralized	Distributed	Hierarchical	Centralized	Centralized
Cloud Service Model	IaaS	IaaS	IaaS	IaaS	IaaS	IaaS	IaaS
Managed by	—	Eucalyptus Systems	C12G Labs	OpenStack Foundation	Apache Software Foundation	—	—
Hypervisors	KVM, XEN	KVM, XEN, VMWare in the enterprise edition.	KVM, VMWare, XEN	KVM, VMware, XEN, LXC, QEMU, Vsphere, HyperV, Microsoft Hyper-v, UML, Virtual Box	KVM, VMWare, XEN, Oracle VM	KVM, VMWare, XEN, Virtual Box	Xen
Interface	CLI	CLI	CLI	CLI	—	Web interface	CLI
Compatible with	AWS EC2, S3	AWS EC2, S3	AWS EC2, S3, Native XML/RPC, OCCI	AWS EC2, S3, EBS and OCCI	—	AWS EC2	—
Supported Platform	Linux	Linux	Linux	Linux	Linux	Linux	Linux
Supported Cloud Types	Public	Private and hybrid	Private, public, and hybrid	Private, public, and hybrid	—	Private, public, and hybrid	Public and private clouds
Programming Language	Java, Python	Java, Python	C, C++, Java, Ruby	Python	Java	C++, Java, Ruby, Python	C, Python
VMs location	Physical nodes	Node controller	Cluster node	Compute Node	—	Clouds nodes	XCP Host
Networking	IP assigned using a DHCP server that can be configured in two ways i.e. centralized and local.	Managed, managed-novLAN, system, static	VLAN	Flat, Flat DHCP, VLAN DHCP	VLAN	WSManagemen	VLAN, Open vSwitch

Storage	—	Volume and Object storage	Volume storage	Volume and Object storage	Volume and Object storage	NFS	Shared storage
DevOps Deployment	No	Chef, Puppet, Crowbar	Chef, Puppet	Chef, Puppet, Crowbar	—	—	—
Authentication	X509, Grids	X509	X509, LDAP, SSH RSA, key pair, Password	X509, LDAP	—	Code Access System(CAS)	—
Users	Scientific communities	Enterprise	Researchers	Enterprises, researchers and developers.	—	Enterprise	—
Scalability	Yes	Yes	Yes	—	—	Yes	Yes
Storage (Image Transfer-ence)	Cumulus	Walrus	NFS	Swift	—	ZFS	—
License	Apache License 2.0	GPL	Apache License 2.0	Apache License 2.0	Apache License 2.0	GPL	GPL
VM Migration/Live Migration	No	No	Yes	Yes	Yes	No	Yes
Load Balancing	Yes	—	Yes	Yes	Yes	No	Yes
Fault Tolerance	Yes	—	Yes	Yes	Yes	No	Yes

OpenStack and CloudStack: Open Source Solutions for Building Public and Private Clouds

Amine Barkat, Alysson Diniz dos Santos, Thi Thao Nguyen Ho

Politecnico di Milano

Dipartimento di Elettronica Informazione e Bioingegneria

Piazza L. da Vinci, 32 - 20133 Milano, Italy

Email: amine.barkat@polimi.it, alysson@virtual.ufc.br, thithao.ho@polimi.it

Abstract—Cloud computing is continuously growing as a prominent technology for enterprises. While several giant public cloud providers, such as Amazon, Microsoft, IBM, Google are competing to extend their market, there is still a large number of organizations asking higher level of privacy and control over cloud solutions. Therefore, the need to have private cloud solutions is obvious. To overcome this need there are several on-going open source software frameworks for building public and private clouds. Among them, OpenStack and CloudStack are growing at fast pace and gaining more attention. An analysis on these software stacks is necessary in order to choose the most suitable solution that matches an enterprise's requirements. This paper main contribution is an in depth study and comparison of the cloud properties of these two open source frameworks, providing useful information on open source cloud solutions that are not available elsewhere.

Keywords—OpenStack, CloudStack, cloud computing, open source, public cloud, private cloud, IaaS

I. INTRODUCTION

Cloud computing is a new computing model that brings together all disciplines, technologies and business models to deliver Information Technology (IT) resources on-demand. This is a new trend that well fits in an environment where resources are provisioned dynamically and exposed as a service on the Internet [1]. In this context, open source cloud technologies such as OpenStack, CloudStack, OpenNebula, Eucalyptus, OpenShift, and Cloud Foundry have gained significant momentum in the last few years. For a researcher and practitioner, they present a unique opportunity to analyze, contribute, and innovate in new services using these technologies [2].

Cloud computing consists basically of three levels of offerings [2]:

- 1) Infrastructure as a Service (IaaS), where the equipments are provided in the form of virtual machines. The client maintains the applications, runtime, integration SOA (Service Oriented Architecture), databases, server software while the supplier maintains the virtualization layer, server, storage, and network hardware. Among the main actors of IaaS, we find Amazon EC2, Rackspace, GoGrid.
- 2) Platform as a Service (PaaS), user can develop his own applications using the services provided. The client maintains only his applications, while the supplier maintains all the cloud stack from hardware up to application containers. We have among the key players: Google Apps Engine, Windows Azure.

- 3) Software as a Service (SaaS), entire applications are available remotely. Among the providers we have GoogleApps, Salesforce, and Facebook.

At the three levels of cloud offerings are shown in Fig. 1, the lowest level is the focus of this paper. The IaaS manages computing resources (computing, storage, network), and the virtualization layer that allows the access to the physical resources (e.g, processor, memory and other devices) providing resource isolation and security.

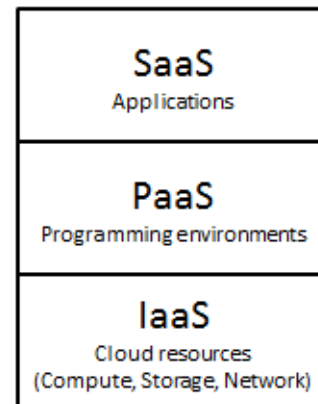


Fig. 1: Offering levels in cloud computing

In the need of having IaaS solutions that can give more privacy and control over the system, open source clouds are born to build private clouds. Eventually, these open source solutions can be used to set up public clouds, private clouds or a mix of them, i.e, hybrid clouds. With the emergence of different open-source cloud solutions, the decision to choose the most suitable one that meets users needs becomes a difficult task, because every platform has its specific characteristics [3]. Moreover, since hybrid clouds are the most widely used nowadays, surveying open source middlewares that simplify cluster management and the creation of private clouds enabled for cloud bursting is an important matter. In this sense, several papers begin to analyze and compare each platform, trying to establish a starting point to look when deciding which open source cloud technology should be adopted. [4]–[6] give essentially an overview of Eucalyptus, Nimbus and OpenNebula solutions, highlighting their different characteristics. [2], [7]–[10] conduct surveys, classify and compare different open source solutions. Concerning specifically OpenStack and

CloudStack, [11] presents briefly both solutions and does some general features comparison, but enter in more details only for OpenStack. Notwithstanding, several updates implemented recently to evolve and improve cloud softwares make this kind of study quickly outdated.

With this in mind, in this paper we present the general features of the newest versions of OpenStack and CloudStack and compare their general features and important properties, trying to provide useful information for users that need to choose an open source cloud software.

The paper is organized as follows: Section II describes the architecture of OpenStack and its important properties, Section III presents CloudStack, Section IV performs comparisons between the two platforms. Conclusions are finally drawn in Section V.

II. OPENSTACK

OpenStack is a cloud software that offers capability to control large pools of compute, storage and networking resources. It also empowers users providing on-demand resources [12]. Starting from 2010, OpenStack was developed by Rackspace Hosting and NASA [13] aimed to provide open source cloud solution to build public or private clouds. The mission of OpenStack is to enable any organization to create and offer cloud computing services running on standard hardware. Provisioned as open source solution, OpenStack is built keeping these core principles in mind: (1) Open source: all code will be released under the Apache 2.0 license allowing the community to use it freely; (2) Open design: every 6 months the development community will hold a design summit to gather requirements and write specifications for the upcoming releases; (3) Open development: maintains a publicly available source code repository through the entire development process; (4) Open community: produces a healthy, vibrant development and user community through an open and transparent process.

A. General Architecture

As in any cloud platform, the infrastructure underneath OpenStack is standard hardware, which can contain any pieces of physical devices such as servers, disks or network devices. In order to provide cloud services, OpenStack develops virtualization layers giving the abstract view of physical infrastructure to end users. These virtualization layers are built up by various components as described in Fig. 2. The OpenStack architecture consists of three main components: Compute (Nova), Network (Quantum) and Storage (Swift). Beside these three pillars, OpenStack has been developing many other services, each of those designed to work together to provide a complete IaaS solution. The integration of these services is facilitated through public application programming interfaces (APIs) offered by each service [13].

In the following, the detailed description of each component is provided.

1) *Compute (Nova)*: Compute is the heart of OpenStack (codename is Nova and it is written in Python), which is the computing fabric controller responsible for managing large networks of virtual machines (VMs), and eventually to properly schedule VMs among available physical machines (PMs)

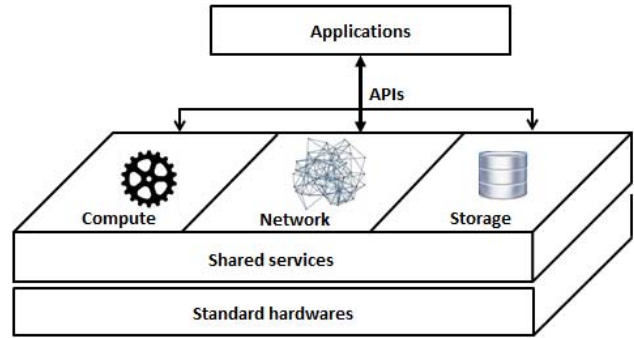


Fig. 2: OpenStack general architecture

[13]. Compute is a distributed application that consists of six components: Nova-api, Message Queue, Nova-Compute, Nova-Network, Nova-Volume and Nova-Scheduler as shown in Fig. 3. Nova supports the complete life-cycles of an instance in the cloud, starting from the request to initialize a VM until its termination. It follows this architecture:

- **Nova-api**: accepts and responds to end user compute API calls. Beside providing its own OpenStack Compute API, Nova-api is compatible with Amazon EC2 API, offering the potential to integrate with Amazon cloud services. It has another special Admin API reserved for privileged users to perform administrative actions. The orchestration activities such as running an instance, or enforcing the policies such as quota checks are initiated by this component.
- **Nova-compute**: is primarily a worker daemon that creates and terminates VM instances via hypervisor APIs. In order to do so, it accepts actions from the queue and performs system commands to fulfill them, while updating the database state accordingly. OpenStack supports several standard hypervisors (listed in Section IV) while keeping the openness that allows to interface other hypervisors through its standard library.
- **Nova-volume**: manages the creation, attaching and detaching of persistent volumes to compute instances. There are two types of block devices supported for an VM instance: (1) **Ephemeral Storage**: is associated to a single unique instance. Its life-cycle exists together with the instance life-cycle, which means when the instance is terminated, data on this storage will also be deleted; (2) **Volume Storage**: is persistent and independent from any particular instance. This storage can be used as external disk device where the data stored on it still remain even when the instance is terminated.
- **Nova-network**: is a worker daemon that handles network-related tasks. It accepts and performs networking tasks from the queue to manipulate the network such as setting up bridging interfaces or changing iptable rules.
- **Nova-schedule**: handles the scheduling of VMs among PMs. It takes a virtual machine instance request from

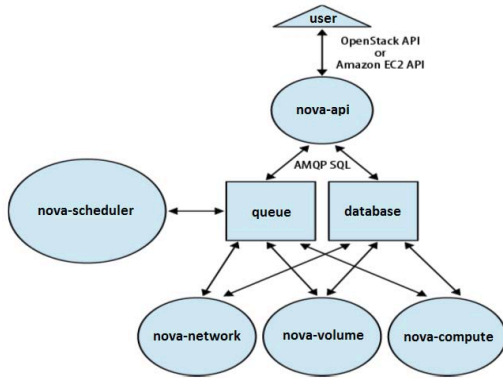


Fig. 3: Compute component architecture [13]

the queue and determines the physical host it should place the instance on. While the scheduling algorithms can be defined by users, Nova-schedule supports by default three algorithms: (1) Simple: attempts to find least loaded host, (2) Chance: chooses random available host from service table, (3) Zone: picks random host from within an available zone. By allowing users to define their own scheduling algorithms, this component is important for building fault tolerant and load-balanced system.

- Queue: provides a central hub for passing messages between daemons. This is usually implemented with-RabbitMQ today, but can support any AMPQ message queue.
- Database: stores most of the build-time and run-time state of a cloud infrastructure. For example, it provides information of the instances that are available for use or in use, networks availability or storage information. Theoretically, OpenStack Nova can support any SQL-based database but the most widely used databases currently are sqlite3, MySQL and PostgreSQL.

Given this architecture, all its components follow a shared-nothing and messaging-based policy. Shared-nothing means that each component or each group of components can be installed on any server, in a distributed manner; while the messaging-based policy ensures the communication among all components such as volume, network and scheduler is performed via Queue Server.

2) *Network (Quantum)*: Network is the key of cloud computing for several reasons: (1) Offered resources and services must be accessible; (2) Address binding between different services is essential to support multi-tier applications; (3) Automatic network configuration capability is important, especially in scenarios where auto-scaling installations evolves. The OpenStack Networking component gives operators the ability to leverage different network technologies to power their cloud networking through a rich set of APIs, multiple networking models (e.g, flat or private network) and flexible plug-in architecture. Especially, the plug-in architecture - with the plug-in agent - enables, not only capability of using various network technologies, but also the ability to handle user workloads. It means, at network level, that developers

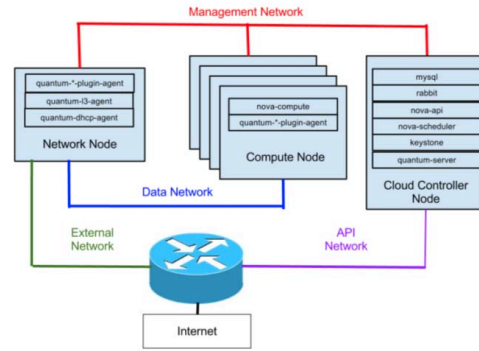


Fig. 4: Networking component architecture [13]

can implement their own load balancing algorithms and plug it in the platform to achieve better workload control. The architecture of Network component is shown in Fig. 4

The Network architecture consists of four distinct physical data center networks:

- Management network: used for internal communication between OpenStack components. The IP addresses on this network should be reachable only within the data center.
- Data network: used for VM data communication within the cloud deployment. The IP addressing requirements of this network depend on the OpenStack Networking plug-in in use.
- External network: used to provide VMs with Internet access in the deployment scenarios. The IP addresses on this network should be reachable by anyone on the Internet.
- API network: exposes all OpenStack APIs, including the OpenStack Networking API, to tenants. The IP addresses on this network should be reachable by anyone on the Internet.

3) *Storage*: The Storage component, one of three main pillars of OpenStack architecture, is used to manage storage resources. OpenStack has support for both Object Storage and Block Storage, with many deployment options for each, depending on the use case.

Object Storage (codename Swift) is a scalable object storage system. It provides a fully distributed, API-accessible storage platform that can be integrated directly into applications or used for backup, archiving, and data retention [13]. In Object Storage, data are written to multiple hardware devices, with the OpenStack software responsible for ensuring data replication and integrity across clusters. Object storage clusters are scaled horizontally while adding new nodes. If a node fails, OpenStack replicates its content from other active nodes. Because OpenStack uses software logic to ensure data replication and distribution, inexpensive commodity hard drives and servers can be used instead of expensive equipments. Therefore, Object Storage is ideal for cost effective, scale-out storage [13].

Block Storage (codename Cinder), is the storage system that allows block devices to be exposed and connected to compute instances for expanded storage, better performance and integration with enterprise storage platforms, such as NetApp, Nexenta and SolidFire [13]. By managing the storage resources in blocks, Block Storage is appropriate for performance sensitive scenarios such as database storage, expandable file systems, or providing a server with access to raw block level storage.

4) *User interface - Dashboard*: The OpenStack dashboard provides to administrators and users a graphical interface to control their compute, storage and networking resources. Through the dashboard, administrators can also manage users and set limits on resources access for each user.

5) *Shared Services*: OpenStack Shared services are a set of several services that span across three pillars of compute, storage and networking, making it easy to perform cloud management operations. These services include [13]:

- Identity Service (code-named Keystone): is the security service to protect resources access and usage. This service provides a central directory management, mapping users to OpenStack accessible services. It acts as a common authentication system across the cloud operating system. It supports multiple forms of authentication including standard username and password credentials, token-based systems and AWS-style logins.
- Image Service (code-named Glance): is the repository for virtual disk and server images used by the VMs. In OpenStack, user can copy or snapshot a server image and immediately store it away. Stored images can be used as a template to get new servers up and running quickly and consistently.
- Telemetry Service: aggregates resources usage and performance data of the services deployed in OpenStack cloud. This powerful capability provides visibility into the usage of the cloud infrastructure and allows cloud operators to view metrics globally or individually.
- Orchestration Service: is a template-driven engine that allows application developers to describe and automate the deployment of the cloud infrastructure as well as detailed post-deployment activities of infrastructure, services and applications.
- Database Service: allows users to quickly and easily utilize the features of a relational database. Cloud users and database administrators can provision and manage multiple database instances as needed.

B. Properties

Provisioned as IaaS, OpenStack is built following an open philosophy: avoid technology lock-ins by not requiring specific technologies and providing user freedom to choose the best slot that matches its needs [13]. In this section, we will analyze some important properties of OpenStack.

- Live migration: is the process of moving a running VM from one PM to another, while the VM is still powered-on. It is important to remember that memory, network connectivity and storage of the migrated VM are also transferred to the destination PM. This capability provides efficient online system maintenance, reconfiguration, load balancing and fault tolerance. OpenStack supports two types of live migration: (i) Shared storage based live migration, and (ii) Block live migration. The former supports live migration scenarios where the source and destination hypervisors have access to the shared storage, while the latter does not require shared storage.
- Load balancing: is the capability that allows to dynamically control the workloads among VMs or physical servers in order to achieve better performance. OpenStack supports load balancing at different scales. First of all, the supporting feature of live migration has enabled system administrators to distribute application workloads among physical servers by means of adjusting VM placement. Moreover, it is possible to control application workloads at VM level, service provided by OpenStack Network layer, controlled by Network component. This component, with a flexible plug-in architecture allows the development of runtime custom algorithms to distribute workloads among VMs. Indeed, OpenStack has an on-going project called Load Balancing as a Service (LBaaS) that is aimed to provide load balancing service to end users. This service has monitoring feature to determine whether the VMs are available to handle user requests and take routing decisions accordingly. Several routing policies are supported such as round robin (i.e., rotates requests evenly between multiple instances), source IP (i.e., requests from a unique source IP address are consistently directed to the same instance) and least connections (i.e., allocate requests to the instance with the least number of active connections).
- Fault tolerance: Within the flexible architecture of OpenStack, fault tolerance can be handled at different levels. These levels depend on the way the IaaS system is configured and deployed. At the VMs level, in order to prevent failures, users can develop scheduling algorithms (besides the three already supported algorithms by OpenStack) for placing the VMs that best fits to his use cases. Some scheduling algorithms have been designed at the present time, such as: group scheduling (i.e., VMs that provide the same functionalities are grouped and placed to separate PMs) and rescheduling (i.e., rescheduling of VMs from failed host to surviving hosts using live-migration). At storage or database level, fault tolerance is achieved by using replication and synchronization to ensure that a failure occurred at one device will not break the whole system.
- Availability: this property seeks to minimize system down time and data loss. In OpenStack, high availability can be achieved through different setups depending on types of services, i.e., stateless or stateful services. Stateless services can provide answer to a request without requiring further information of other

services or historical data. OpenStack stateless services include nova-api, nova-scheduler, etc. For these services, high availability is achieved by providing redundant instances and load balance them. In the opposite, stateful services are ones that requires other information to answer a request, which makes them difficult to obtain high availability. These services, e.g. database or storage, can be highly available by using replication but at the same time the system has to maintain the synchronization between the main version and replicated versions in order to keep the system consistent [14].

- **Security:** OpenStack has a separated service (Identity service) which provides a central authentication management across the cloud operating system and users. The possibility to set up VPNs and firewalls is also available.
- **Compatibility:** OpenStack is highly compatible with Amazon EC2 and Amazon S3 and thus client applications written for Amazon Web Services can be used with OpenStack with minimal porting effort [13]. In terms of hypervisors, OpenStack supports multiple hypervisors, e.g. Xen, KVM, HyperV, VMWare, etc. Other hypervisors with existing standard drivers can also be interfaced with OpenStack through standard library, e.g. libvirt library.

III. CLOUDSTACK

CloudStack [15] is an open source software platform, written in Java, designed for development and management of cloud Infrastructure as a Service. It aggregates computing resources for building private, public or hybrid clouds. CloudStack is a turnkey technology that brings together the "Stack" of features requested by companies and users, like data centers orchestration, management and administration of users and NaaS (Network as a Service).

The start of CloudStack was with Cloud.com in 2008. In May 2010, it was open source under GNU General Public License. Citrix bought CloudStack in July 2011, then in April 2012, Citrix donated CloudStack to Apache Software Foundation (ASF) where it was relicensed under Apache 2.0 and accepted as an incubation project. Since March 2013, CloudStack became a Top Level Project of Apache. Many companies are basing on CloudStack for building and managing their cloud infrastructures. Among these companies, there are: Nokia, Orange, Apple, Disney and many others.

A. CloudStack Architecture

In this section, we break down the logical architecture of CloudStack, shown in Fig. 5. In CloudStack, physical resources are organized and managed in a hierarchical structure. The lowest level contains computational devices such as host and primary storage. Hosts are attached together and access shared storage to form a Cluster. The next level consists of clusters which are combined by a layer 2 switch to form a Pod. Go up to higher level, Pods are grouped together with Secondary storage by layer 3 switch to form a Zone. At the highest level, zones are grouped to create a Region. All these resources are managed by a Management Server. In the

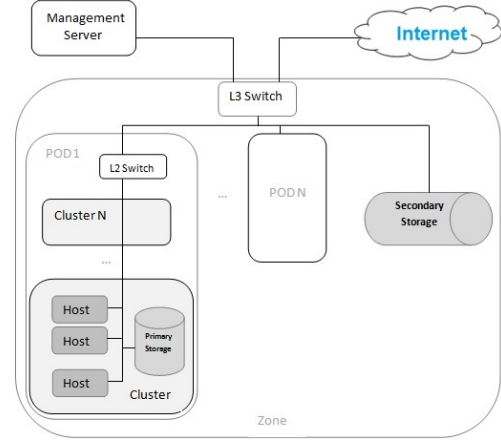


Fig. 5: CloudStack architecture

following, we describe in details each component that forms the whole architecture.

1) *Host*: A host represents a physical computational machine that contains local storage. The physical hosts are virtualized by hypervisors. CloudStack supports many hypervisors for VMs management such as Xen, KVM, vSphere, Hyper-V, VMWare, etc. as well as bare metal provisioning. All hosts within a cluster must be homogeneous in terms of the hypervisor, with the possibility of having heterogeneous hypervisors in different clusters.

2) *Cluster*: Within a cluster, hosts are tied together into the same computational pool with the primary storage and have the same IP subnet. The primary storage can be any kind of storage supported by the hypervisor. One cluster can have more than one primary storage device.

3) *Pod*: A Pod is a collection of different clusters linked with a layer 2 switch. Hosts in the same Pod are in the same subnet. Pod is not visible to the end user.

4) *Zone*: The benefit of using Zone is for isolation and redundancy. Often, it corresponds to a data center; although if a data center is large enough, it can have multiple zones. A zone contains Pods that are attached to the secondary storage using a layer 3 switch. Zones are visible to the end user and they can be private or public. Public zones are visible to all users in the cloud while private zones are visible only to users from a particular domain.

5) *Region*: A region is the largest organizational unit in CloudStack. A region contains multiple zones distributed in geographic locations close to each other.

6) *Management Server*: A Management Server is used to manage all resources in cloud infrastructure through APIs or UI. One management server can support around 10K hosts and can be deployed on a physical server or a VM. In case we have more than one management server, user interaction to either of them will return the same result. This ensures high availability of CloudStack. A database is required for management servers to be persistent. In order to prevent single point of failure, we

can have one primary database and several database replica which always stay synchronized with the primary copy.

7) *Storage*: In addition to the host local storage, CloudStack manages two main types of storage: primary storage and secondary storage.

- Primary storage: is a storage associated with a cluster or a zone. In the same cluster, we can deploy multiple primary storages. This kind of storage is basically used to run VMs and stores application data. Since this storage interacts directly with applications deployed in VMs, it can be expensive in terms of I/O operations, this is the reason why it is placed physically near to the hosts.
- Secondary storage: is used to store ISO images, templates, snapshots, etc. It supports two different types, NFS and Object Storage.
 - ISO image: is used when user wants to create a VM.
 - Template: is the base operating system image that the user can choose when creating new instance. It may also include additional configuration information such as installed applications.
 - Snapshot: is used as backup for data recovery service. CloudStack supports two types of snapshot: individual snapshot and recurring snapshot. The former is one-time full snapshot, while the latter is either one-time full snapshot or incremental snapshot.

8) *Networking*: CloudStack supports the use of different physical networking devices (e.g. NetScaler, F5 BIG-IP, Juniper SRX, etc). In CloudStack, users have the ability to choose between two types of networks scenarios: basic and advanced. The basic scenario is for an AWS-style networking. It provides a single network where guest isolation is done through the layer 3 switch. The advanced scenario is more flexible in defining guest networks [15]. For example, the administrator can create multiple networks for use by the guests. CloudStack provides many networking services. Among them we cite:

- Isolation: CloudStack assures the isolation of networks, by allowing the access to the isolated network only by virtual machines of a single account.
- Load Balancing: to balance the traffic in the cloud, the user can create a rule to control and distribute the traffic and apply it to a group of VMs. Within the defined rule, user can choose a load balancing algorithm among the supported ones.
- VPN: for accessing to the VM using CloudStack account, users can create and configure VPNs. Each network has its own virtual router, so VPNs are not shared across different networks. Using VPN tunnels, hosts in different zones are allowed to access each other.
- Firewall: hosts in the same zone can access to each other without passing through the firewall. Users can use external firewalls.

B. Properties

Cloudstack has many properties that motivate companies to use it to manage their infrastructure. The main properties of CloudStack are [15]:

- Live migration: A live migration of running VMs between hosts is allowed in CloudStack through the Dashboard. Depending on the VMs hypervisor, migration conditions can be different. For example, live migration using KVM hypervisor will not support the use of local disk storage, and source and destination hosts have to be in the same cluster; while Xen and VMWare support local disk storage and allow to migrate between different clusters [16].
- Load balancing: a Load balancer is an optional component of CloudStack that allows to distribute the traffic among different management servers [17]. In addition to creating rules and using load balancing algorithms, CloudStack offers the possibility to integrate with external load balancers such as Citrix NetScaler [18].
- Fault tolerance: in CloudStack, fault tolerance is achieved at different scales. In order to prevent failures of management server, the server can be deployed in multi-node configuration. Should one management node fail, other nodes can be used without affecting cloud functioning. Failures at database level are handled by using one or more replication of the database linked to the management server. For host's fail-over, CloudStack recovers the VM instances by taking the images from secondary storage and using application data in primary storage.
- Availability: CloudStack ensures high availability of the system by using multiple management server nodes which may be deployed with load balancers.
- Security: in addition to isolation using different accounts, VPNs and firewalls, CloudStack offers the isolation of traffic using the strategy of security groups which are sets of VMs that filter the traffic on the basis of configuration rules. CloudStack provides a default security group with predefined rules, however they can be modified if necessary.
- Compatibility: CloudStack is built based on a plugable architecture, one cloud can support different hypervisor implementations including: Hyper-V, KVM, LXC, vSphere, Xenserver, Xen Project and also bare metal provisioning. Moreover, CloudStack is compatible with Amazon API and enables the integration of these two platforms.
- Scalability: CloudStack has the ability to manage thousands of servers distributed in different data centers and different locations thanks to the management server capability (one management server node can manage a big pool of physical resources), and the possibility of using multiple management servers for reducing VMs downtime.
- API extensibility: The CloudStack APIs are very powerful and allows developers to create new command line tools and UIs, and to plug them into CloudStack

architecture. If the developer wants to use new hypervisor, new storage system or new networking service, he just needs to write a new plug-in in Java and integrate it.

IV. CLOUD FRAMEWORKS COMPARISON

In this section, we provide a comparison between OpenStack and CloudStack, looking at three different levels:

- 1) General comparison aimed at providing high level comparison in terms of model, policy and architecture
- 2) Functional comparison whose goal is to compare supported functionalities, and
- 3) Property comparison which finally considers cloud properties implemented in these platforms.

It is important to highlight that, since the two platforms are under continuous development, there are not complete documentation and technical reviews provided to fully understanding their technical aspects. Hence, it is difficult to perform a reasonable comparison between the two. However, we attempt to provide a comparison from user perspective, considering the properties that a user needs to look at when choosing a IaaS cloud solution.

A. General Comparison

The general comparison is provided in Table I, considering general aspects: licensing, cloud model compatibility, business model, architecture, etc.

	OpenStack	CloudStack
Open Source License	Apache 2.0	Apache 2.0
Commercial model	Free	Free
Compatibility with	Private, public and hybrid clouds	Private, public and hybrid clouds
Easy installation	Difficult (many choices, not enough automation)	Medium (Few parts to install)
Architecture	Fragmented into many pieces	Monolithic controller
Large organizations adopters	Yahoo, IBM, VMWare, Rackspace, Redhat, Intel, HP, etc.	Nokia, Orange, Apple, Citrix, Huawei, TomTom, Tata, etc.

TABLE I: General comparison between OpenStack and CloudStack

As it can be seen, the two frameworks are generally equal with respect to business model, licensing policy and cloud models. Each of them has been adopted by large organizations. Nevertheless, a big difference is spotted from the architecture viewpoint. While OpenStack is fragmented into modules, CloudStack has a monolithic central controller. This difference is explained by the open philosophy of Open Stack which tries to avoid technology lock-ins and provides high degree of flexibility and extension. As a consequence, OpenStack is characterized by increasing complexity of installation and configuration.

B. Functional Comparison

The functional comparison looks at the offered functionalities or technical aspects of the two solutions, as described in Table II. Supported hypervisors lists all hypervisors that are currently available in these two platforms, even though there are ways to interface with non-supported hypervisors.

Administration feature is the interface available to interact with these platforms. Both solutions present Web interfaces (Web UI) and command line interfaces (CLI). Also user management is provided in both platforms. Concerning the monitoring feature, both OpenStack and CloudStack can capture system states and resources usage in order to identify and adapt to occurring problems or perform optimization.

Functionality	OpenStack	CloudStack
Supported hypervisors	Xen, KVM, HyperV, VMWare, LXC, vSphere	Xen, KVM, HyperV, VMWare, LXC, vSphere
Administration	Web UI, CLI	Web UI, CLI
User management	yes	yes
Monitoring	Health monitoring, Resources usage trend monitoring	Health monitoring and usage data monitoring

TABLE II: Functional comparison between OpenStack and CloudStack

Other important functional aspects of open source cloud solutions are related to the activeness of its community. A solution that is always seeing new releases is constantly evolving. An active community, with well documented wiki, good bug reporting and fixing system and active users support are fundamental features for the success of an open source solution.

About the releases, after its insertion on Apache Incubator, CloudStack made its first major release (4.0.0-incubating) on November of 2012. Since then, there were three major releases (4.1 and 4.2 and 4.3) and four minor ones. Meanwhile, OpenStack does, since 2012, two big releases per year. In 2012, there were Essex and Folsom, in 2013 Grizzly and Havana and in 2014 Icehouse and the now under development Juno. In OpenStack, the number of minor releases varies from version to version. From this we can conclude that, even though OpenStack is showing more updates per year, both solutions are regularly updated.

About the activeness of the community, OpenStack remains the largest and most active open source cloud computing project [19]. Nevertheless, CloudStack is growing and has an important participation in the market. Both platforms have well updated wikis and rely on summits to get together their developers. An active forum community for bug reporting and user support can be also found for both solutions. In this point, OpenStack has a bigger and more active community but CloudStack has gained momentum in the last years.

C. Properties Comparison

Properties comparison gives deeper insights into the two platforms, considering some important properties that an IaaS has to provide. The comparison is given in Table III.

Property	OpenStack	CloudStack
Live migration	Yes	Yes
Load balancing	VM level, PM level	VM level, host level
Fault tolerance	VM scheduling, replication	VM scheduling, replication
Availability	Redundancy, load balancer	Redundancy, load balancer
Security	VPNs, firewall, user authentication, others	VPNs, firewall, user management, others
Compatibility	Amazon EC2, Amazon S3	Amazon EC2, Amazon S3

TABLE III: Properties comparison between OpenStack and CloudStack

Live migration is an important property in IaaS solution. Both platforms provide different ways to support live migration. OpenStack offers two types of live migration, while CloudStack has migration conditions depending on the hypervisors.

The two platforms support flexible load balancing mechanisms even though they are made through different strategies. Load balancing at host level is implemented in OpenStack through live migration, which is the same in CloudStack; while load balancing at VM level is achieved in OpenStack through the flexible plug-in architecture of Network component. This capability can be also achieved at network layer in CloudStack with ability of using external load balancers.

Fault tolerance mechanisms are provided in both platforms under the policies to schedule VM placement or services replication.

High availability can be achieved in both platforms by means of using redundant service instances and load balancing to distribute workloads among those instances.

Security is provided by the two platforms through setting up VPNs, firewall and other services. A central user authentication system is available in OpenStack, while CloudStack has user account management. OpenStack has an extensible architecture which allows other systems such as intrusion detection system to be plugged in and deployed; instead, CloudStack offers security group to isolate VMs.

OpenStack and CloudStack are also compatible with Amazon APIs that enable the potential integration with this dominant commercial public cloud solution.

V. CONCLUSIONS

OpenStack and CloudStack are two prominent open source platforms that offer tools and services for building private, public, and hybrid clouds. In this paper, we have presented the up-to-date architecture of the two platforms, looking into the details of the provided functionalities and their properties. We conclude the work with the comparison between the two. From the comparison, we realize that it is not easy to say which is the best between them, since they are both stable and used by many large companies, and they continue to evolve. These two solutions support different hypervisors, storages and use different strategies to build a IaaS system. The final choice is for the user who decides to move his business to the cloud, which hypervisors he wants to use, which storage style is more adaptable for his applications, how he wants to deploy his infrastructure, etc. Since the two platforms are on-going developing software, it is necessary to continue updating their development path before selecting an IaaS solution to adopt.

REFERENCES

- [1] L. Schubert, K. G. Jeffery, and B. Neidecker-Lutz, *The Future of Cloud Computing: Opportunities for European Cloud Computing Beyond 2010:—expert Group Report*. European Commission, Information Society and Media, 2010.
- [2] O. Sefraoui, M. Aissaoui, and M. Eleuldj, "Comparison of multiple iaas cloud platform solutions," in *Proceedings of the 7th WSEAS International Conference on Computer Engineering and Applications (Milan-CEA13)*, ISBN, 2012, pp. 978–1.
- [3] X. Wen, G. Gu, Q. Li, Y. Gao, and X. Zhang, "Comparison of open-source cloud management platforms: Openstack and opennebula," in *Fuzzy Systems and Knowledge Discovery (FSKD), 2012 9th International Conference on*. IEEE, 2012, pp. 2457–2461.
- [4] B. Sotomayor, R. S. Montero, I. M. Llorente, and I. Foster, "Virtual infrastructure management in private and hybrid clouds," *Internet Computing, IEEE*, vol. 13, no. 5, pp. 14–22, 2009.
- [5] D. Nurmi, R. Wolski, C. Grzegorzczak, G. Obertelli, S. Soman, L. Youseff, and D. Zagorodnov, "The eucalyptus open-source cloud-computing system," in *Cluster Computing and the Grid, 2009. CCGRID'09. 9th IEEE/ACM International Symposium on*. IEEE, 2009, pp. 124–131.
- [6] J. Peng, X. Zhang, Z. Lei, B. Zhang, W. Zhang, and Q. Li, "Comparison of several cloud computing platforms," in *Information Science and Engineering (ISISE), 2009 Second International Symposium on*. IEEE, 2009, pp. 23–27.
- [7] P. T. Endo, G. E. Gonçalves, J. Kelner, and D. Sadok, "A survey on open-source cloud computing solutions," in *Brazilian Symposium on Computer Networks and Distributed Systems*, 2010.
- [8] P. Sempolinski and D. Thain, "A comparison and critique of eucalyptus, opennebula and nimbus," in *Cloud Computing Technology and Science (CloudCom), 2010 IEEE Second International Conference on*. Ieee, 2010, pp. 417–426.
- [9] M. Mahjoub, A. Mdhaffar, R. B. Halima, and M. Jmaiel, "A comparative study of the current cloud computing technologies and offers," in *Network Cloud Computing and Applications (NCCA), 2011 First International Symposium on*. IEEE, 2011, pp. 131–134.
- [10] I. Voras, B. Mihaljevic, M. Orlic, M. Pletikosa, M. Zagar, T. Pavic, K. Zimmer, I. Cavrak, V. Paunovic, I. Bosnic *et al.*, "Evaluating open-source cloud computing solutions," in *MIPRO, 2011 Proceedings of the 34th International Convention*. IEEE, 2011, pp. 209–214.
- [11] S. A. Baset, "Open source cloud technologies," in *Proceedings of the Third ACM Symposium on Cloud Computing*. ACM, 2012, p. 28.
- [12] O. C. Software, "Chapter 1. introduction to openstack," in <http://docs.openstack.org/training-guides/content/module001-ch001-intro-text.html>. OpenStack.
- [13] (2014) Openstack history. [Online]. Available: <https://www.openstack.org/>
- [14] OpenStack, "Introduction to openstack high availability," in <http://docs.openstack.org/high-availability-guide/content/stateless-vs-stateful.html>. OpenStack.
- [15] (2014) Cloudstack website. [Online]. Available: <https://cloudstack.apache.org/docs/>
- [16] (2014) Cloudstack live migration. [Online]. Available: http://cloudstack.apache.org/docs/en-US/Apache_CloudStack/4.1.0/html/Admin_Guide/manual-live-migration.html
- [17] (2014) Management server load balancing. [Online]. Available: https://cloudstack.apache.org/docs/en-US/Apache_CloudStack/4.0.2/html/Installation_Guide/management-server-lb.html
- [18] P. N. Sabharwal, "Integrating netScaler with cloudstack," in *Apache CloudStack Cloud Computing*. Packt Publishing.
- [19] networkworld. (2014, July). [Online]. Available: <http://www.networkworld.com/article/2166407/cloud-computing/stack-wars—openstack-v—cloudstack-v—eucalyptus.html>

Private IaaS Clouds: A Comparative Analysis of OpenNebula, CloudStack and OpenStack

Adriano Vogel*, Dalvan Griebler^{*†}, Carlos A. F. Maron[†], Claudio Schepke[‡] and Luiz Gustavo Fernandes[†]

* Laboratory of Advanced Researches for Cloud Computing (LARCC),

Três de Maio Faculty (SETREM), 2405, Santa Rosa Av.– Três de Maio – RS – Brazil

[†]PUCRS, Department of Informatics, Computer Science Graduate Program, 6681, Ipiranga Av. – Porto Alegre – RS – Brazil

[‡]Federal University of Pampa (UNIPAMPA), Laboratory of Advanced Studies (LEA) – Alegrete – RS – Brazil

Email: adrianovogel03@gmail.com, {dalvan.griebler, carlos.maron}@acad.pucrs.br,
claudioschepke@unipampa.edu.br, luiz.fernandes@pucrs.br

Abstract—Despite the evolution of cloud computing in recent years, the performance and comprehensive understanding of the available private cloud tools are still under research. This paper contributes to an analysis of the Infrastructure as a Service (IaaS) domain by mapping new insights and discussing the challenges for improving cloud services. The goal is to make a comparative analysis of OpenNebula, OpenStack and CloudStack tools, evaluating their differences on support for flexibility and resiliency. Also, we aim at evaluating these three cloud tools when they are deployed using a mutual hypervisor (KVM) for discovering new empirical insights. Our research results demonstrated that OpenStack is the most resilient and CloudStack is the most flexible for deploying an IaaS private cloud. Moreover, the performance experiments indicated some contrasts among the private IaaS cloud instances when running intensive workloads and scientific applications.

I. INTRODUCTION

Scientific researches are becoming even more necessary for cloud computing to find out new questions and current problem answers. The cloud popularity is a consequence of the well-consolidated architecture of service (IaaS, PaaS and SaaS) and deployment (private, public, hybrid and community) models [1]. This scenario also has changed the way as information technology deals with business and research, where resources are easily available by a shared pool and on-demand usage [2]. Moreover, although it has become a suitable model for enterprise and scientific applications, such environment integrates several technologies and distributed systems paradigms that makes it complex to manage and evaluate [3], [4].

Cloud computing has challenges for various reasons. We highlight software abstraction layer, deployment model, and different technology integration as the main aspects to focus our studies. The first aspect is the virtualization that abstracts the hardware layer [5]. The second is the workload impact and cloud performance variation, which may affect the overall system results (latency, throughput and energy consumption) [3]. The last are the Service Agreement Level (SLA) issues between the provider and client. This relationship refers for a correct combination and tools compatibility between the target application and cloud environment in a cloud deployment [6].

To increase performance, contingency, security and availability, many companies and research centers are also looking

for private cloud deployment solutions. In the IaaS model, several open source tools provide elementary management capabilities over a data center infrastructure [7]. This interest motivates our study towards a comparative analysis of performance, flexibility and support for resiliency.

Choose an appropriate IaaS management tool for deploying a private cloud is important to achieve successful results. This decision is made according to the tool's features and imposed application constraints. The key points are support for flexibility and resiliency because it allows to estimate the tool robustness level. In this paper, we measured these aspects with our case study tools by using and extending the [7]'s taxonomy, which classifies IaaS management tools. Therefore, we can point out differences and challenges that match the application's needs and help the decision maker.

Another critical point for providing efficient services on cloud computing is the performance. In the last years, only few research papers investigated this topic, concerning IaaS and private deployment models. The literature (Section II) provides a small number of evaluation reports which puts such topic as a research challenge. Our experiments are the starting point towards a comparative evaluation and analysis of the tools.

The main goal of this paper is to discuss and compare IaaS management tools for addressing the challenges of private cloud deployments as well as to address flexibility and resiliency differences. Consequently, other tools can be analyzed in the future using our methodology. The secondary goal is to provide performance insights through intensive workloads and some scientific applications when running in the deployed environments. Therefore, the main contributions are:

- Support for Flexibility and resiliency evaluation of three IaaS management tools. This contribution extends the evaluations of [7], including the resiliency and CloudStack tool not considered in their taxonomy.
- A comparison of a private IaaS cloud deployment. Unlike previous work, this paper evaluates the tools considering three key aspects of cloud computing: flexibility, resiliency and performance.
- Performance evaluation of three private IaaS cloud environments. Considering the related work, we con-

tribute with OpenStack performance view and scientific applications insights for these cloud pools.

Our paper is organized as follows. Section II discusses the closest related work. Section III performs a survey and measures IaaS tool's flexibility and resiliency. Section IV presents the performance experiments and analyzes the results. Finally, Section V provides the conclusion of the paper.

II. RELATED WORK

The related work is classified in tool features evaluation and performance analysis.

A. Surveys for Tool Features

In this section, we investigate papers concerning the evaluation of IaaS cloud features.

Regarding cloud infrastructure provision and web hosting, [8]'s paper proposed a taxonomy to identify and classify cloud computing aspects (service type, resource deployment, hardware, runtime tuning, security, business model, middleware, and performance). They considered seven IaaS, Web hosting and PaaS providers. The paper also introduced a performance measurement (CPU) on cloud instances, which was continued in [9], using a larger number of applications and test scenarios. The results emphasized the weak performance (with variations) and SLA support on cloud providers.

[7]'s study proposed seven conceptual layers for a management tool of cloud infrastructure. In their classification, the IaaS management layer is composed by Command Line Interface Tools, APIs, Dashboard and Orchestrator. Such services perform essential tasks (instantiate, delete, and manage VMs, monitor resources, and power management APIs) on cloud environments for users and administrators. The authors surveyed private and public IaaS providers to demonstrate their taxonomy. As a result, it distinguishes support and capabilities for adapting the most suitable technology.

A survey of the state-of-the-art efforts was conducted by [10]. It investigated interoperability as well as practical cloud technologies of both the cloud provider's and user's perspectives to enable inter-operation. They concluded that innovations on a virtualized network should exploit software-defined networking capabilities to better facilitate inter-cloud cooperation. Advanced SLA mechanisms may be needed to constantly incorporate user feedback and customized features into the SLA evaluation framework. Finally, the question remains of how to implement cost analysis and pricing of various resources in interoperable clouds.

Considering these related works, [8]'s paper provides a survey of IaaS taxonomy for public clouds while our work concentrates on private clouds. [7]'s work creates a taxonomy for flexibility evaluation that we use as a baseline in our paper. On the other hand, [10]'s study provides a taxonomy for IaaS interoperability. **Therefore, we complement flexibility and contribute with resiliency and CloudStack tool analysis.**

B. Experiments for Performance Evaluation

For each performance related works, we analyzed aspects such as objectives, execution environment, adopted workloads, numeric and qualitative conclusions and limitations.

[9] and [3] analyze usefulness and performance of scientific applications for current cloud computing services. They analyzed Amazon EC2 platform using micro-benchmarks and synthetic kernels. The results indicated current public cloud services need performance improvements for scientific applications. Also, [3] concluded performance is a challenge for cloud environments due to resource sharing.

The goal in [11] was to evaluate HPC applications in the main public cloud providers, considering performance, cost efficiency and development. The article suggests cloud computing as an alternative for eliminating the maintenance of standard clusters. They ran NAS benchmarks for evaluating development and cost-effectiveness. The results show that clouds are a viable to run HPC applications with some development disadvantages such as the support for libraries. Considering performance and the cost-effectiveness, the authors estimated cloud environments 27% and 41% more efficient than clusters.

[5]'s paper studied six IaaS public providers using standard benchmark applications. The objective was to analyze performance and scalability variations when a multi-tier application is migrated from a traditional data-center environment to a cloud infrastructure. Also, they evaluated the performance on three mainstream hypervisors: XEN, KVM and CVM. The results show significant performance variations among the hypervisors. More precisely, Xen outperforms the commercial hypervisor (CVM) by 75% on the read-write RUBBoS workload and the commercial hypervisor outperforms Xen by over 10% on the Cloudstone workload.

All these papers validated the benefits to assess virtualization technologies or public IaaS clouds. [11] point out HPC challenges and usage on public cloud computing. By the other hand, [9], [5] and [3] contributed for public IaaS cloud provider performance evaluation when running scientific applications. **Our work differs from these previous studies on a performance analysis for three private IaaS cloud deployments.** Moreover, this paper contributes for the literature with new performance challenges and insights when running intensive workloads and scientific applications.

III. ANALYZING FLEXIBILITY AND RESILIENCY

The IaaS cloud comprises different layers, as illustrated in Figure 1. At the bottom are the physical resources (hardware). Above there is a Virtual Machine Manager (VMM) that supports the abstraction needed for the upper layers. The VMM is a hypervisor running within the native operating system (full virtualization) or a hypervisor running directly in the hardware (bare metal). Also, the Virtual Infrastructure Manager (VIM) is part of the IaaS cloud tools that takes advantage of virtual resources for offering services. The main tasks of resources scheduler, images, networks, volumes, templates and VMs

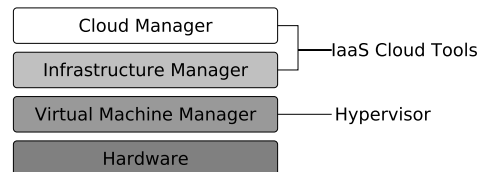


Fig. 1. IaaS overview.

creation are performed by the VIM, which are related to virtual instances and appliances [12]. Finally, the cloud manager can be a part of an IaaS tool or a separate cloud layer. This layer controls users, groups, permissions, quotas and quality of service issues of the cloud environment.

Almost all cloud management tools support administrative tasks through a graphical User Interface (UI), which is named dashboard by the tool's nomenclature. This interface is available through a web browser, providing virtual infrastructure control. There is also a Command Line Interface (CLI) which is used by cloud administrators to faster and easier terminal control. The last user interactions are the APIs for controlling the tool's infrastructure and integrating them among different services. The next sections will present and discuss OpenNebula, CloudStack and OpenStack.

A. IaaS Tools in a Nutshell

OpenNebula software is designed to work with driver concept for implementing the IaaS layer [13]. Those familiar with Linux systems will feel comfortable to deploy and control OpenNebula cloud. Mostly because it does not create a proper nomenclature, integrating default Linux-based services like scripts and components. When building a cloud environment, a machine must be configured as front-end, which will host the cloud manage, and all the other machines will be slave nodes. Storage, virtual machine, data transfer, network management, and job scheduler are configured by OpenNebula services management, using a compatible technology [14].

CloudStack is conceptually built on stacks of services and internal agents. The first service is the cloud controller. The second is the storage server for hosting volumes. [15] The third service (secondary storage) is for handling ISO images, templates and snapshots. It is implemented using a distributed file system service. The last one is a network service that implements several advanced features like virtual LAN, Virtual Private Network and Generic Routing Encapsulation [16].

OpenStack builds on API stacks. These APIs are used to communicate its services using the RabbitMQ. Such messenger runs as an interface among APIs and offers infrastructure services for the cloud system. There are forty individual components available, where the essential are: orchestrator, ceilometer (metering), swift (object storage support), Neutron (networking), Keystone and hybrid support [17]. The storage may be deployed in different ways and stored locally or distributed. OpenStack is considered a modular and granular solution because enables to implement individually all infrastructure services[18]. The next sections will present and discuss more details about the tools.

B. Flexibility Evaluation

The majority of cloud tools are robust and complex technologies. In recent years, cloud applications and usage have been investigated by the scientific community [3], [11], [10]. The architecture and flexibility of IaaS cloud tools for a customizable deployment are still under research [7], [8], [10]. Therefore, we present a comparative analysis among three open source solutions to deploy an IaaS cloud. We based our flexibility evaluation on [7]'s taxonomy which is divided into seven layers: core service, support, value-added services,

control, management, security, and resource abstraction. Also, we extend the analysis to cover resiliency.

Open source IaaS cloud tools present relevant and advanced options both for the cloud administrator and users. Table I shows the IaaS tools equivalency for flexibility. We surveyed all tabulated data by deploying these tools as well as looking for their official documentation. As presented in the Table I, the

TABLE I. TOOL'S FLEXIBILITY.

Support	OpenNebula	CloudStack	OpenStack
Resource abstraction layer			
Compute	Oned	Libcloud	Nova
Storage	Internal	Internal	Object storage (Swift)/Block Storage(Cinder)
Volume	Internal	Internal	Nova-Volume
Network	Virtual Network Manager	Internal	Neutron/Nova-network
Core service layer			
Identity service	IAM plugin	IAM API	Keystone
Scheduling	Scheduler	Internal	Nova-scheduler
Image repository	Internal	Internal	Glace
Charging and billing	Internal	CloudStack Usage	Ceilometer
Logging	Internal	Internal	Internal
Support layer			
Message bus	Internal/RabbitMQ	internal/RabbitMQ	RabbitMQ
Database	sqlite/MySQL	MySQL	MySQL/Galera/MariaDB/MongoDB
Transfer service	Internal	Internal	Nova Object store/cinder
Management layer			
Resource management	Internal	Internal	Nova
Federation management	/	/	/
Elasticity management	Auto-scaling	Elastic Load Balancing	Elastic Recheck
User/group management	Internal	Internal	Internal
SLA definition	/	/	/
Monitoring	probe/ssh/OneGate	External	External
Reporting	code reporting	/	/
Incident management	/	Internal	External
Power management	External	External	Blueprint driver
Lease management	External	External	External
Management tools			
CLI tools	OpenNebula CLI	cloudmonkey	OpenStack (CLI)
APIs	Public cloud and Plugins	Public cloud and Plugins	Public cloud and Plugins
Dashboard	Sunstone(Admin UI, User UI)	Admin UI	Horizon(Admin UI)
Orchestrator	Oneflow	Cloudstack Cookbook	Heat
Security layer			
Authentication	Basic Auth/Open Nebula Auth/x 509 Auth/LDAP	SAML/LDAP	LDAP/Tokens(APIs)/X.509/HTTPD
Authorization	Auth driver	SAML	Keystone
Security groups	Internal	Internal	Internal
Single sign-on	/	External	/
Security monitoring	External	External	External
Control layer			
SLA enforcement	/	/	/
SLA monitoring	/	/	/
Metering	External	Usage plugin	Ceilometer
Policy control	/	/	/
Notification service	/	Internal	/
Orchestration	Internal	Internal	Internal
Value-added services			
Availability zones	Internal	Internal	Internal
High availability	External	External	External
Hybrid support	Microsoft Azure AmazonEC2/IBM	Amazon EC2	HP Helion/Amazon EC2/IBM
Live migration	Internal	Internal	Internal
Portability support	/	/	/
Image contextualization	One-context	/	/
Application support	/	/	/

IaaS solutions have different features and goals. Some features are supported inside the core solution (“internal”), others easy achieve the demand when integrating with an external and fully compatible component/framework (“external”), while a few are not supported (“/”).

The resource abstraction layer is the closest to the virtualization. OpenNebula uses the Oned to provide computer resources (CPU, memory). OpenStack takes advantage of the Nova API to process compute requests, which have advanced tasks/resources like scheduling and networking. CloudStack uses the libcloud for attending the tool’s services. On the storage and volume, while OpenStack uses specific components, OpenNebula and CloudStack embedded it internally. In contrast, just CloudStack handled itself the network feature. Considering that all tools are proving at least some resource abstraction, they have similar flexibility on this layer.

In the core service layer, the identity service is used on complex cloud deployments to guarantee authorization. For example, OpenStack keystone API manages all its components and users while other tools uses an external plugin or manages it internally. The scheduler prepares all the management jobs according to the resources availability and load balance, where each one have its own scheduler. The image repository provides a catalog of pre-built OS ISOs to facilitate the VMs launching. All tools enable charging and billing services. Such feature is useful for public cloud providers to control the tenants resources usage. The logging are stored internally in plain text files or database. Thereby, specific tools are necessary to audit them.

The support layer comprises additional services on a cloud system. The message bus is used to communicate among the independent services and APIs on a cloud environment. RabbitMQ is unanimous used for transferring messages among independent components. The database is used to control cloud operations and store user credentials and system logs. All tools support at least one database, where OpenStack has more compatibility options. Transfer services are an alternative to communicate or send files among management services. Each solution performs this job using its most convenient protocol.

Management layer is responsible for controlling or centralizing user operations. The monitoring feature is fundamental for checking the hosts and instance status. All three tools implement the resources functions, elasticity, groups management, and cloud virtual environment controlling. However, they presented a low flexibility concerning federation, SLA, incident and lease management. Consequently, we identified open challenges to be considered in the tools’ design and their poor capabilities on this layer.

In the management tools layer are the interfaces to control a cloud infrastructure either for administrators and end users. As previously described, CLIs and APIs play on all the tools. However, each tool presents these interfaces more or less fragmented. Also, the web interface helps to manage the cloud environment by the dashboard front-end and orchestrator feature, which are useful for large cloud deployments.

Many aspects are related to the security layer. In our table, the authentication is the process for transferring credentials to control the system access. Authorization is related to the user privileges and authentication. A security group corresponds

to a set of users or projects sharing affinities and privileges. The single sign-on is related to the identities sharing and authentication among distinct/independent systems. Finally, the security monitoring is a complex system that manages and works based on predefined actions to maintain elevated security levels (authorization, high availability, among others). Despite the general poor security monitoring and single sign-on (only CloudStack supports), the tools enable authentication, authorization and security groups for increase the privacy and isolation. These insights are basic security aspects which demonstrates open challenges for OpenStack and OpenNebula to integrate security monitoring and deal with single sign-on.

The control layer presented a weak coverage of the tools. The poor SLA support indicates that they are most suitable for private cloud instead of public cloud. The metering service is for monitoring users’ cloud utilization. CloudStack has the advantage of offering a notification service, facilitating the cloud administration. The policy control feature is a set of capabilities related to security and quotas. Through our studies was possible to find out their no support in this category. The orchestration system is for enabling easy integration of services, being internally implemented.

The last layer is the value-added, which covers additional services for a cloud system. For instance, the high availability feature is complex to achieve in the cloud. One way towards high availability is implement redundancy on every level such as disks (e.g., RAID arrangement), services and database (replication) and install more than one cloud management server. The support of hybrid clouds enable to integrate a private cloud with services running on public clouds. These features and live migration are present in the surveyed tools. However, they are still lacking on virtual applications and portability, which is relevant for advanced cloud deployments. For additional service, only OpenNebula supports image contextualization to deploy VMs.

The support for flexibility of a computer system is the capability to support multiple components and applications settings [2]. It is needed in order to meet the different clients and application demands. In our approach, we used flexibility to analyze the enabling technologies and components supported by the cloud tools. Figure 2 summarizes the support of the tools’ flexibility levels. Such measurement is computed using the tabulated data from Table I. As the taxonomy layers are

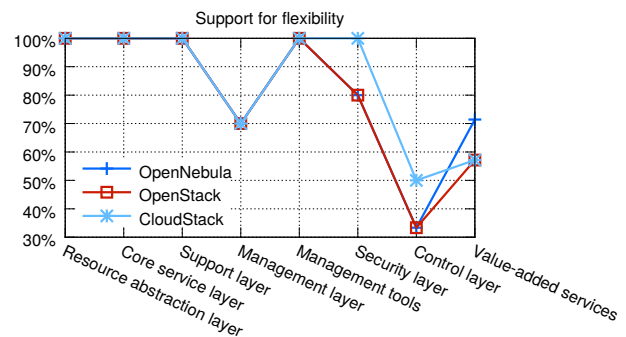


Fig. 2. Tools’ flexibility levels.

shaped by items, we calculated the percentage rate in separate for each one of the layers, where an item must have at least one possible setting to be counted. Consequently, a tool achieves 100% of support for flexibility in one of the surveyed layers when it has compatibilities with all the items.

When analyzing the flexibility results, it becomes evident the tools' poor support for the control, services and management layers. The way resources abstraction technologies behave impact on cloud infrastructure services. Consequently, the resources are effecting on core service, support, control and management layers will indeed affect cloud operations and services for clients. For example, cloud security implementation will result in how reliable and confidential is the service provision. Also, the value-added services will offer more options and customization for a cloud.

Due to the relevance of flexibility for deploying a cloud environment, the tools are expected to continue increasing and improving its features. Through our comparative analysis, we can highlight that there are many studies to do for improving management and control layer in these tools (previously discussed). Such lack will impact directly to provide services for end users. Moreover, there are research challenges in security and value-added services to offer robust cloud deployments.

C. Support for Resiliency Evaluation

The computer system support for resiliency is the ability for adapting the system for constant changes or failures maintaining the availability [19]. It is also the compatibility of multiple configurations and customizations such for load balance, high performance and fault tolerance [4]. We understand that support for resiliency when referring to IaaS tools aggregate the aspects mentioned by both authors ([19] and [4]). Consequently, this is not only the ability of the system to recover in case of failures, but also the tool mechanisms and technologies that make it possible. Thus, a variety of infrastructure technologies is expected to increase the resiliency of a private cloud environment such a way that it can look for the most efficient alternative. In Table II, we surveyed these information on the tools official documentation and the deployed environments.

Table II fulfills IaaS tools support for enabling resiliency. The Full and Bare Metal virtualization refers to the virtual-

ization for running multiple machines on the same hardware. These virtualization architectures share the hypervisor usage that emulates hardware resources to the virtual machines.

The cloud storage supports many technologies. Table II refers to used solutions on each cloud tool. For example, it is fundamental to fit the applications performance needs because storage technologies and local/distributed deployment have performance contrasts. Also, the compatibility vary among hypervisors and workloads for disk formats. They may impact on performance [20] due to the different supported hypervisor and I/O scheduler algorithms. Therefore, the most appropriate disk format vary when heterogeneous workloads are considered.

Depending on the hypervisor and virtualization technologies, the network compatibility may impact on the application's performance. The OS support refers to the number of host OS coverage on full virtualization deployments. A combination of OS and hypervisor also implies on different performance results, resources consumption, and system resiliency. For example, the containers' virtualization are built inside a native OS to simplify the virtualization layer and reduce the overhead. Finally, the Object storage is a robust and new way for offering data storage, dealing with disks as objects [21].

Figure 3 presents the tools' support for resiliency. The results are computed based on Table II data, measuring the amount of supported technologies by the cloud tools for each item. Using such quantitative results of the survey, each item' percentage is calculated considering only the number of available technologies supported for the three cloud tools. The total number of technologies (among the tools) supported by each item is considered 100% and each tool supports a specific number of technologies that reflects in a percentage value.

We identified that OpenStack presents the best percentage of resiliency, followed by CloudStack. These two solutions received the best averages because of the large support for storage, host OS, and hypervisors. All tools have Linux Containers (LXC) compatibility, enabling the deployment of OS level virtualization. As object storage is recommended for advanced storage systems to treat huge amounts of data (e.g., data mining, big data). Consequently, we can highlight challenges to increase it for better resiliency. Our insights are demonstrating challenges for the network, OS, and disk format.

TABLE II. IaaS TOOLS SUPPORT FOR RESILIENCY.

Support	OpenNebula	OpenStack	CloudStack
Full and Bare Metal virtualization	Xen, KVM, Vmware	Hyper-V, VMware, Xen, KVM, VirtualBox	Hyper-V, Xen, KVM, VMware, VirtualBox
Storage technology	NFS, SSH(transfer), Ceph	LVM, Ceph, Gluster, NFS, ZFS, Sheepdog	NFS, SMB, SolidFire, NetApp, Ceph, LVM
Disk Formats	qcow2, vmfs, ceph, lvm, fslvm, raw, dev	LVM, qcow2, raw, vhd, vmdk, vdi	LVM, VMDK, VHD, qcow2
Network	dummy, ebttables, VLAN, OVS, vmware	Neutron, and B.Switch, Brocade, OVS, NSX, PLUMgrid	bridge, VLAN, DHCP, DNS, NVP, BigSwitch, OVS
OS	Ubuntu, Debian, RedHat, SUSE, CentOS	Debian, Ubuntu, RHEL, CentOS, Fedora, Suse	Debian, Ubuntu, RHEL, CentOS
Container Virtualization	LXC	LXC	LXC
Object Storage	Ceph	Cinder, Ceph	Cinder, Ceph

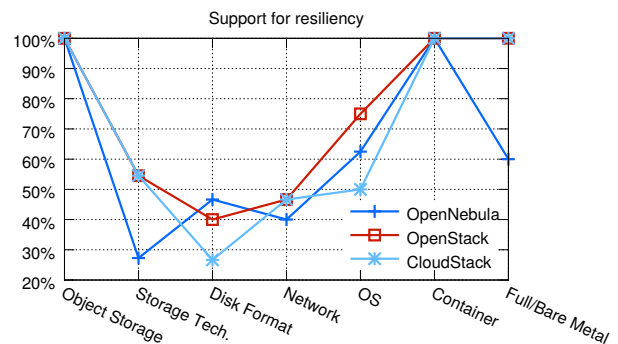


Fig. 3. Tools' support for resiliency levels.

D. Remarks

Our analysis clarified the robustness of cloud tools, detailing with their support as well as an estimation of their support for flexibility and resiliency. While flexibility may affect end users, the resiliency may impact the cloud system performance. As there are several solutions available, sometimes with poor documentation about the capabilities, it also becomes difficult for decision-making and requires a deep learning.

To discuss performance system aspects, the next section (IV) presents a performance overview on private IaaS cloud deployments. These tools can reduce the overall performance when the recommended deployment is not followed, or tools that do not support application needs. Therefore, robustness is important to enable a scalable and customizable environment for clients running their application in a cloud virtual pool.

IV. PERFORMANCE ON IAAS PRIVATE CLOUDS

This section evaluates cloud system's performance using intensive micro-benchmark and scientific applications. As presented on Section III-C, the IaaS cloud tools support several infrastructure technologies. Our goal is to discover performance insights when cloud tools are similarly deployed. Consequently, our test scenario is described on methodology section (IV-B). Therefore, we concentrate in the performance differences and variations on our three private clouds. Such results will give an overview of the performance on cloud virtualized systems compared to the native environment.

A. Cloud Deployment

We deployed the cloud pools following tools' official documentation. Our environment consist of two elements: cloud controller (also called as front-end or cloud management) and node. The controller configures an IaaS tool and distributed storage. All nodes are slaves of the controller for allocating instances, dealing with the virtualization layer (KVM, Libvirt, QEMU). The nodes communicate among each other and with the controller by a network switch.

Concerning the network, we installed a flat implementation using Linux bridges. Such configuration is justified by the small number of hosts on each cloud pool, which does not requires advanced features (eg., OpenStack Neutron). Additionally, the distributed storage was chosen due to the reliability of the network and full compatibility with the tools. The hosts communicate and transfer volumes using the SSH protocol or via Remote Procedure Call (RPC).

B. Methodology

Our methodology is based on those used in related work's performance analysis. We chosen well known micro-benchmarks and applications as the baseline. The average performance is computed over 40 samples for each one of the tests. We intentionally deploy identical clouds, using a mutual hypervisor as done by [22]. The cloud tools were the only differences. Such environments are built in order to compare the fairest possible.

1) *Intensive workloads*: are run to evaluate individual intensive computations. The following benchmarks will stress CPU, memory, storage and networking:

- **LINPACK**: performs CPU-intensive computations for solving linear equations [23]. In the experiments, we set up a matrix with dimensions of 8000x8000 running on a single CPU core.
- **STREAM**: measures the memory bandwidth for four synthetic vector kernels (Add, Copy, Scale and Triad) by using larger data set than the system cache size. By default it uses three arrays, each one with 610.4 MiB, requiring a total of 1.8 GiB. [24].
- **IOzone**: stresses I/O performance by Write, Read, Re-Read and Rewrite disk operations for measuring the throughput with a 5GB files [25].
- **IPerf**: evaluates network bandwidth [26]. We used a default TCP window size of 85.0 KByte, transferring in an interval of 10 seconds.

2) *Scientific Applications*: are evaluated in the three deployed clouds by the following NAS [27] kernels:

- **Embarrassingly Parallel (EP)**: is an application that does not have dependency between tasks.
- **Multi Grid (MG)**: stresses memory and communicates [27] by mesh operation sequences.
- **Integer Sort (IS)**: performs random memory access by sorting operations.
- **Fourier Transform (FT)**: communicates one to all for fast Fourier transformation.

3) *Environment of Tests*: For deploying our three cloud tools, we used 4 identical Supermicro blades. Each one has 24 GB of RAM over a speed of 1333 MHz, a CPU socket Intel Xeon X5560 (quad-core 2.80GHz), 3 physical disks (SATA II 7200 RPM) organized on a RAID5 and interconnected over a gigabit (10/1000) network. The virtual environment was configured using QEMU/KVM 2.0.0 hypervisor to offer virtual resources. Our IaaS cloud deployed were the CloudStack 4.5.2, OpenNebula 4.12 and Openstack Kilo. As native OS environment for both deploying cloud tools and running experiments, we used Ubuntu Server 14.04 with 3.19.0 kernel. Each physical server hosted one large virtual instance, which had available the full machine resources and used the same OS version.

C. Intensive Workloads Performance

Figure 4 presents the micro-benchmarks' performance. We ran experiments in the four scenarios: Native, OpenNebula, OpenStack, and CloudStack to compare their differences.

The results demonstrates small overheads between the native and virtualized environments. LINPACK results demonstrated a similar performance between the cloud instances and close to Native. By other hand, IOzone results had more variation. Such performance is related to the workload's behavior. When comparing among the tools, the native outperform them using IOzone. On Read and ReRead operations, the cloud instances had similar performance to Native. Otherwise, it is

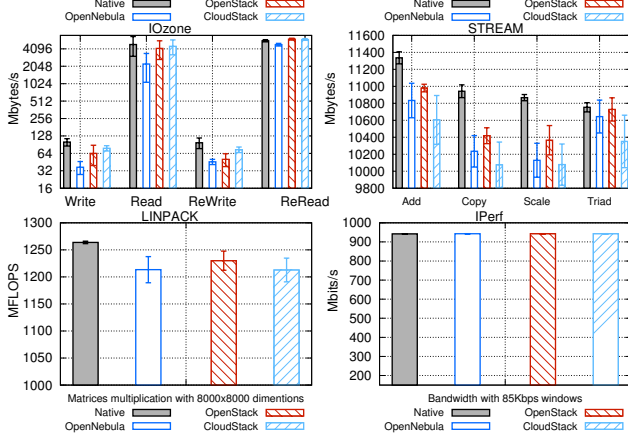


Fig. 4. Intensive workloads' performance.

possible to realize that OpenNebula had poor disks throughput, mainly on Write and ReWrite operations.

The memory experiments presented the OpenStack instances slightly better than the OpenNebula and Cloudstack because of the less variation. For the network results, the instances and native environment had a very similar throughput.

Several researches analyzed the performance variation on public cloud systems such [28] and [5]. In our experiments, we also found high variation on some workloads either on native and cloud environments. In general, OpenStack instances presented less variation in respect to other tools. For example, the overall disks performance presented high standard deviation (of a total of 40 tests). Even the native environment, which is supposed to have less variation. We observed an unfavorable deviation on disks workloads of 14.5% on write, 20.9% on ReWrite and 37.4% on disk read operations. Also, it tends to become worst on virtualized environments (*eg.*, cloud instances). On the other hand, the workloads deviation on CPU, memory and network resources presented less variation and are more stable. Our experiments contribute with new insights on these micro-benchmarks. The next sections presents scientific applications performance analysis (Section IV-D).

D. Scientific Applications Performance

Figure 5 demonstrates the NAS-OMP kernels performance results on: OpenNebula, OpenStack, CloudStack and Native. For this kernel suite, we only ran the experiments on a single machine and instance (OpenMP), setting up all the available resources. We can highlight the similar performance results of the deployed cloud environment when comparing to the native. Also, among the tools there were no significant contrasts and performance variations.

In the NAS-MPI kernels, the results were different. Figure 6 shows the NAS-MPI kernels results. The Message-Passing Interface (MPI) had performance variation and contrasts. The native environment presented the best results on scientific applications, followed by the cloud instances. While Openstack cloud shown the highest variations, CloudStack and OpenNebula clouds presented better performances. However, the differences among the deployed environment are not big. Our

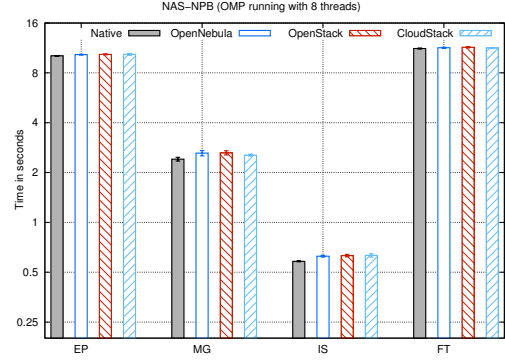


Fig. 5. NAS-OMP performance.

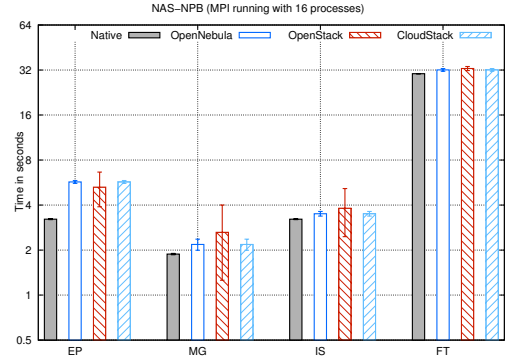


Fig. 6. NAS-MPI performance.

experiments demonstrated that when running scientific applications on private clouds, the performance does not impact for shared memory. However, sometimes the applications running on distributed fashion will have performance degradation.

E. Remarks

The previous subsections (IV-C, IV-D) presented a performance overview of OpenStack, OpenNebula and CloudStack tools. With such analysis, we find out that a implementation of distributed file system does not increases the performance degradation in the cloud. Such event is by the combination of a fair network throughput (Figure 4) and the para-virtualized VirtIO driver. In STREAM benchmark, the results for memory bandwidth showed that performance increases along with the native environment. Similar event occurred with LINPACK due to the KVM hypervisor improvements. In general, the cloud tools presented similar results.

In our work, we found some performance contrasts among the tools. The results may vary on different installations approaches or experiments. For instance, CloudStack requires at least three system VMs (secondary storage, console proxy and virtual router) to work. These VMs are used for robustness and reliability. Consequently, the VMs system reduce the resources available for run instances. In our environment, these VMs were not hosted on the compute nodes to allow full resource accesses. Another example is the high number of OpenStack components, which may overload the controller node when running instances. In this approach, we maintained

the controller dedicated for the cloud management and monitoring, and the openstack compute nodes just had installed the Nova component. In contrast, OpenNebula has a centralized deployment and does not have such limitations because its core is fine-grained.

V. CONCLUSIONS

This paper researched and analyzed IaaS cloud solutions for deploying private clouds. Such study contributes for new insights concerning flexibility and resiliency. We demonstrated their distinct capabilities through an enhanced methodology that can be used for future studies as well as update the information of the surveyed tools. Also, we point out challenges for improving the tools' robustness levels such as SLA, management, advanced security options, and virtualization.

We also introduced performance experiments for OpenNebula, OpenStack, CloudStack and native environments. The goal was to evaluate performance by using intensive workloads and scientific applications. Our work contributed for empirical performance insights on three different IaaS private clouds and well known test scenarios. Through analysis, we evidenced high performance variations on IOzone workload and poor performance on specific resources on OpenNebula instances. The workloads running on OpenStack instances were the most stable when comparing with the other cloud instances (OpenNebula and CloudStack). In addition, in almost all the cases at least one cloud deployment presented results close to native. We noticed a small virtualization overhead.

Moreover, all deployed tools achieved similar performance among them and native environment concerning the scientific applications. We concluded in our analysis that private clouds are a good alternative for intensive workloads and scientific applications. These outcomes were not found on public cloud providers when looking for the performance analysis on related works (Section II). As future work, we intend to: (I) evaluate more applications and benchmarks for VM scheduling, deployment, and image transfer; (II) customize the deployed clouds for testing different network and storage options; (III) continue deploying and analyzing private IaaS cloud tools using our methodology.

ACKNOWLEDGMENT

The authors would like to thank for the partial support in this research from HiPerfCloud project and its supporters: computer network department of SETREM², and CNPq (MCTI/CNPQ/Universal 14/2014 - Process 457684/2014-3).

REFERENCES

- [1] ISO JTC1/SC38 Technical Report, "ISO/IEC 17789:2014. Information Technology - Cloud Computing - Reference Architecture." 2014, last access in Sept, 2015. [Online]. Available: http://www.iso.org/iso/catalogue_detail?csnumber=60545
- [2] R. Buyya, C. Vecchiola, and T. Selvi, *Mastering Cloud Computing*. San Francisco, CA, USA: McGraw Hill, 2013.
- [3] A. Iosup, S. Ostermann, M. Yigitbasi, R. Prodan, T. Fahringer, and D. Epema, "Performance Analysis of Cloud Computing Services for Many-Tasks Scientific Computing," *Parallel and Distributed Systems IEEE Transactions on*, vol. 22, no. 6, pp. 931–945, June 2011.
- [4] ISO JTC1/SC38 Technical Report, "ISO/IEC 17788:2014 Information Technology - Cloud Computing - Overview and Vocabulary," 2014, last access in Sept, 2015. [Online]. Available: http://www.iso.org/iso/catalogue_detail?csnumber=60544
- [5] D. Jayasinghe, S. Malkowski, J. Li, Q. Wang, Z. Wang, and C. Pu, "Variations in Performance and Scalability: An Experimental Study in IaaS Clouds Using Multi-Tier Workloads," *Services Computing, IEEE Transactions on*, vol. 7, no. 2, pp. 293–306, April 2014.
- [6] B. Furth and A. Escalante, *Handbook of Cloud Computing*, ser. Computer Science. Springer, USA, 2010.
- [7] R. Dukaric and M. B. Juric, "Towards a Unified Taxonomy and Architecture of Cloud Frameworks," *Future Gener. Comput. Syst.*, vol. 29, no. 5, pp. 1196–1210, Jul. 2013.
- [8] R. Prodan and S. Ostermann, "A Survey and Taxonomy of Infrastructure as a Service and Web Hosting Cloud Providers," in *Grid Computing, 10th IEEE/ACM International Conference on*. IEEE, Oct 2009, pp. 17–25.
- [9] S. Ostermann, A. Iosup, N. Yigitbasi, R. Prodan, T. Fahringer, and D. Epema, "A Performance Analysis of EC2 Cloud Computing Services for Scientific Computing," vol. 34, pp. 115–131, 2010.
- [10] Z. Zhang, C. Wu, and D. W. Cheung, "A Survey on Cloud Interoperability: Taxonomies, Standards, and Practice," *SIGMETRICS Perform. Eval. Rev.*, vol. 40, no. 4, pp. 13–22, Apr. 2013.
- [11] E. Roloff, M. Diener, A. Carissimi, and P. O. A. Navaux, "High Performance Computing in the Cloud: Deployment, Performance and Cost Efficiency," *CloudCom*, vol. 2012, pp. 371–378, 2012.
- [12] F. Xu, F. Liu, H. Jin, and A. V. Vasilakos, "Managing Performance Overhead of Virtual Machines in Cloud Computing: A Survey, State of the Art, and Future Directions," *Proceedings of the IEEE*, vol. 102, no. 1, pp. 11–31, 2014.
- [13] OpenNebula, "OpenNebula," 2015, last access Aug, 2015. [Online]. Available: <http://opennebula.org>
- [14] G. Toraldo, *OpenNebula 3 Cloud Computing*.
- [15] N. Sabharwal, *Apache CloudStack Cloud Computing*.
- [16] Cloudstack, "Cloudstack," 2015, last access Aug, 2015. [Online]. Available: <https://cloudstack.apache.org>
- [17] Q. Khedher, *Mastering OpenStack*.
- [18] OpenStack, "OpenStack Roadmap," 2015, last access Aug, 2015. [Online]. Available: <http://openstack.org/software/roadmap>
- [19] V. Castano and I. Schagayev, *Resilient Computer System Design*. Springer International Publishing, 2015.
- [20] A.-C. Bujor and R. Dobre, "KVM IO Profiling," in *Networking in Education and Research, 2013 RoEduNet International Conference 12th Edition*, Constanta, Romania, Sept 2013, pp. 1–6.
- [21] L. Coyne, S. Gopalakrishnan, J. Sing, and I. Redbooks, *IBM Private, Public, and Hybrid Cloud Storage Solutions*. IBM Redbooks, 2014.
- [22] A. Paradowski, L. Liu, and B. Yuan, "Benchmarking the Performance of OpenStack and CloudStack," in *17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing (ISORC)*. Virginia St Reno, NV, USA: IEEE, June 2014, pp. 405–412.
- [23] J. J. Dongarra, P. Luszczek, and A. Petitet, "The LINPACK Benchmark: Past, Present, and Future," *Concurrency and Computation: Practice and Experience*, vol. 15, p. 2003, 2003.
- [24] J. McCalpin, "The Stream Benchmark," 1996, last access in Sept, 2015. [Online]. Available: <http://www.cs.virginia.edu/~mccalpin/papers/bandwidth/node1.htm>
- [25] Iozone, "Iozone File System Benchmark (Official Page)," 2006, last access in Sept, 2015. [Online]. Available: <http://iozone.org>
- [26] Iperf, "What is Iperf?" 2015, last access in Sept, 2015. [Online]. Available: <http://iperf.fr>
- [27] D. H. Bailey et al., "The NAS parallel benchmarks," *International Journal of High Performance Computing Applications*, vol. 5, no. 3, pp. 63–73, 1991.
- [28] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

¹<http://hiperfcloud.setrem.com.br>

²<http://setrem.com.br>